SQL

CHEAT SHEETS

From the First SELECT to Window Functions, Transactions & Tuning

SELECT · Joins · Aggregates · Window Functions · CTEs · DML · DDL · Indexes · Transactions · JSON · PostgreSQL · MySQL · SQL Server · SQLite

82 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP
01 Introduction & History
02 Installing & Connecting
03 Syntax & Statement Structure
04 Relational Model & Sample Schema

WHICH TOOL WHEN
05 Which Clause When
06 Function Directory

DATA TYPES
07 Numeric Types
08 Character Types
09 Date & Time Types
10 Boolean, Binary, UUID & Enum Types
11 NULL & Three-Valued Logic
12 Type Conversion & Casting

SELECT FUNDAMENTALS
13 SELECT Basics
14 Filtering with WHERE
15 Sorting with ORDER BY
16 Limiting & Pagination
17 DISTINCT & Duplicates

OPERATORS & EXPRESSIONS
18 Comparison & Range Operators
19 Pattern Matching
20 Arithmetic & Concatenation Operators
21 CASE Expressions

BUILT-IN FUNCTIONS
22 String Functions — Transform
23 String Functions — Search & Extract
24 Numeric Functions
25 Date & Time Functions
26 Conditional & NULL Functions

AGGREGATION & GROUPING
27 Aggregate Functions
28 GROUP BY
29 HAVING & Conditional Aggregation
30 ROLLUP, CUBE & GROUPING SETS
31 String & Array Aggregation

JOINS
32 INNER JOIN
33 Outer Joins
34 CROSS JOIN & Self Joins
35 Multi-Table Joins
36 Semi & Anti Joins
37 LATERAL & APPLY

SUBQUERIES, CTES & SET OPERATIONS
38 Subqueries — Scalar & IN
39 Correlated Subqueries
40 Derived Tables
41 Common Table Expressions
42 Recursive CTEs
43 Set Operations

WINDOW FUNCTIONS
44 Window Function Basics
45 Ranking Functions
46 Offset & Value Functions
47 Running Totals & Moving Averages
48 Window Frames
49 Window Function Patterns

MODIFYING DATA — DML
50 INSERT
51 UPDATE
52 DELETE & TRUNCATE
53 Upsert & MERGE
54 Bulk Loading & Export

DEFINING SCHEMA — DDL
55 CREATE TABLE
56 Primary & Foreign Keys
57 Constraints
58 Auto-Increment & Sequences
59 ALTER & DROP
60 Databases, Schemas & the Catalog

DATABASE DESIGN
61 Normalization
62 Relationships & ER Modeling
63 Design Conventions

VIEWS, FUNCTIONS, PROCEDURES & TRIGGERS
64 Views & Materialized Views
65 User-Defined Functions
66 Stored Procedures
67 Triggers

TRANSACTIONS & CONCURRENCY
68 Transactions
69 Isolation Levels
70 Locking & Deadlocks

INDEXING & PERFORMANCE
71 Index Basics
72 Index Types & Strategies
73 Reading Query Plans
74 Query Optimization

SECURITY & ADMINISTRATION
75 Users, Roles & Privileges
76 SQL Injection & Parameterized Queries
77 Backup & Restore

SEMI-STRUCTURED DATA & SEARCH
78 JSON
79 Arrays & Generated Series
80 Full-Text Search

QUICK REFERENCE
81 Quick Reference: Query Syntax
82 Quick Reference: Dialect Differences

SQL

Table of Contents

82 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 04

- 01 Introduction & History
- 02 Installing & Connecting
- 03 Syntax & Statement St…
- 04 Relational Model & Sa…

WHICH TOOL WHEN 05 – 06

- 05 Which Clause When
- 06 Function Directory

DATA TYPES 07 – 12

- 07 Numeric Types
- 08 Character Types
- 09 Date & Time Types
- 10 Boolean, Binary, UUID …
- 11 NULL & Three-Valued …
- 12 Type Conversion & Ca…

SELECT FUNDAMENTALS 13 – 17

- 13 SELECT Basics
- 14 Filtering with WHERE
- 15 Sorting with ORDER BY
- 16 Limiting & Pagination
- 17 DISTINCT & Duplicates

OPERATORS & EXPRESSIONS 18 – 21

- 18 Comparison & Range …
- 19 Pattern Matching
- 20 Arithmetic & Concate…
- 21 CASE Expressions

BUILT-IN FUNCTIONS 22 – 26

- 22 String Functions — Tra…
- 23 String Functions — Sea…
- 24 Numeric Functions
- 25 Date & Time Functions
- 26 Conditional & NULL Fu…

AGGREGATION & GROUPING 27 – 31

- 27 Aggregate Functions
- 28 GROUP BY
- 29 HAVING & Conditional …
- 30 ROLLUP, CUBE & GROU…
- 31 String & Array Aggrega…

JOINS 32 – 37

- 32 INNER JOIN
- 33 Outer Joins
- 34 CROSS JOIN & Self Joi…
- 35 Multi-Table Joins
- 36 Semi & Anti Joins
- 37 LATERAL & APPLY

SUBQUERIES, CTES & SET OPERATIONS 38 – 43

- 38 Subqueries — Scalar &…
- 39 Correlated Subqueries
- 40 Derived Tables
- 41 Common Table Expres…
- 42 Recursive CTEs
- 43 Set Operations

WINDOW FUNCTIONS 44 – 49

- 44 Window Function Basi…
- 45 Ranking Functions
- 46 Offset & Value Functio…
- 47 Running Totals & Movi…
- 48 Window Frames
- 49 Window Function Patt…

MODIFYING DATA — DML 50 – 54

- 50 INSERT
- 51 UPDATE
- 52 DELETE & TRUNCATE
- 53 Upsert & MERGE
- 54 Bulk Loading & Export

DEFINING SCHEMA — DDL 55 – 60

- 55 CREATE TABLE
- 56 Primary & Foreign Keys
- 57 Constraints
- 58 Auto-Increment & Seq…
- 59 ALTER & DROP
- 60 Databases, Schemas &…

DATABASE DESIGN 61 – 63

- 61 Normalization
- 62 Relationships & ER Mo…
- 63 Design Conventions

VIEWS, FUNCTIONS, PROCEDURES & TRIGGERS 64 – 67

- 64 Views & Materialized V…
- 65 User-Defined Functions
- 66 Stored Procedures
- 67 Triggers

TRANSACTIONS & CONCURRENCY 68 – 70

- 68 Transactions
- 69 Isolation Levels
- 70 Locking & Deadlocks

INDEXING & PERFORMANCE 71 – 74

- 71 Index Basics
- 72 Index Types & Strategi…
- 73 Reading Query Plans
- 74 Query Optimization

SECURITY & ADMINISTRATION 75 – 77

- 75 Users, Roles & Privileges
- 76 SQL Injection & Param…
- 77 Backup & Restore

SEMI-STRUCTURED DATA & SEARCH 78 – 80

- 78 JSON
- 79 Arrays & Generated Se…
- 80 Full-Text Search

QUICK REFERENCE 81 – 82

- 81 Quick Reference: Quer…
- 82 Quick Reference: Diale…

Introduction & History

SQL · relational model · ISO/ANSI standard · RDBMS · dialects · declarative

WHAT SQL IS

SQL (Structured Query Language) is the language of relational databases. It is **declarative**: you describe the result you want and the database's query planner decides how to get it. It grew out of E. F. Codd's 1970 relational model and IBM's SEQUEL prototype, became an ANSI standard in 1986 and an ISO standard in 1987, and the current edition is **SQL:2023** (ISO/IEC 9075). Every RDBMS speaks a **dialect**: the core is portable, the edges are not.

SQL FACTS

- Say "S-Q-L" or "sequel"; both are common
- A table is an **unordered** set of rows; without `ORDER BY` row order is not guaranteed
- This set uses PostgreSQL 17+ and notes MySQL 8.4, SQL Server 2022+ and SQLite 3.45+

NOSQL COMPARISON

- **MongoDB** stores JSON-like documents in collections; no fixed columns
- Queries are method calls and pipelines, not one declarative statement
- `SELECT name FROM customers WHERE city = 'Oslo'` ≈ `db.customers.find({ city: "Oslo" }, { name: 1 })`

A FIRST QUERY

```sql
-- Which customers spent the most?
SELECT   c.name,
         SUM(o.total) AS spent
FROM     customers AS c
JOIN     orders    AS o ON o.customer_id = c.customer_id
WHERE    o.status <> 'cancelled'
GROUP BY c.name
HAVING   SUM(o.total) > 500
ORDER BY spent DESC
LIMIT    5;

-- The five sublanguages
SELECT name FROM products;                -- DQL
DELETE FROM orders WHERE order_id = 9;    -- DML
CREATE TABLE tags (tag_id INT);           -- DDL
GRANT SELECT ON products TO analyst;      -- DCL
COMMIT;                                   -- TCL
```

CODE EXPLAINED

- → `JOIN … ON` — pairs each order with its customer (sheet 32)
- → `WHERE` filters rows, `HAVING` filters groups (sheet 29)
- → `SUM(…) AS spent` — aggregate with an alias
- → `ORDER BY … LIMIT 5` — sort, then keep the top five

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	Closest to the standard; rich types (`jsonb`, arrays, ranges)
MySQL	Most-deployed open source; backtick identifiers; InnoDB engine
SQL Server	T-SQL dialect: `TOP`, `+` concatenation, procedural batches
SQLite	Embedded library, one file per database, flexible typing

IN AN ORM

```ts
await prisma.order.groupBy({
  by: ['customerId'],
  _sum: { total: true },
  orderBy: { _sum: { total: 'desc' } },
  take: 5,
});
```

MILESTONES

YEAR	EVENT
1970	Codd publishes the relational model
1986	SQL-86, first ANSI standard (ISO 1987)
1992	SQL-92: `JOIN` syntax, `CASE`, `CAST`
1999	Recursive CTEs, triggers, `BOOLEAN`
2003	Window functions, `MERGE`, sequences
2023	SQL/PGQ graph queries, `JSON` type

SUBLANGUAGES

GROUP	STATEMENTS	SHEET
DQL	`SELECT`	13
DML	`INSERT UPDATE DELETE MERGE`	50
DDL	`CREATE ALTER DROP TRUNCATE`	55
DCL	`GRANT REVOKE`	75
TCL	`BEGIN COMMIT ROLLBACK SAVEPOINT`	68

⚠ COMMON MISTAKES

- Expecting rows back in insert order without `ORDER BY`
- Assuming a feature works everywhere because one engine has it
- Writing loops over rows when one set-based statement would do

✓ TIPS & BEST PRACTICES

- Learn standard SQL first, then your engine's extras
- Read a query in logical order: `FROM` first, `SELECT` near the end (sheet 03)
- Develop against the same engine and version you run in production

QUICK SUMMARY

- SQL — declarative language for relational data
- Standard — ISO/IEC 9075, latest SQL:2023
- Dialects — PostgreSQL, MySQL, SQL Server, SQLite
- Statements — DQL, DML, DDL, DCL, TCL

Installing & Connecting

GETTING A DATABASE

The quickest way to get a server engine is a **Docker** container: one command, nothing installed on your machine, and you can throw it away. You then connect with the engine's **command-line client** (`psql`, `mysql`, `sqlcmd`) or a GUI, using a host, port, user, password and database name, often packed into one **connection string** (URL). **SQLite** has no server at all: the client opens a file and the file is the database.

CLIENTS

- Cross-engine GUIs: DBeaver, DataGrip, TablePlus, Beekeeper Studio
- Engine GUIs: pgAdmin, MySQL Workbench, SSMS, DB Browser for SQLite
- Keep passwords in env vars (`PGPASSWORD` , `.env`), not in scripts

NOSQL COMPARISON

- MongoDB: `docker run -d -p 27017:27017 mongo:8`
- Client: `mongosh "mongodb://localhost:27017/shop"`
- Redis: `docker run -d -p 6379:6379 redis` then `redis-cli`

START & CONNECT

```
# PostgreSQL 17
docker run -d --name pg -p 5432:5432 \
  -e POSTGRES_PASSWORD=secret postgres:17
psql "postgresql://postgres:secret@localhost:5432/postgres"

# MySQL 8.4 LTS
docker run -d --name my -p 3306:3306 \
  -e MYSQL_ROOT_PASSWORD=secret mysql:8.4
mysql -h 127.0.0.1 -P 3306 -u root -p

# SQL Server 2022
docker run -d --name ms -p 1433:1433 -e ACCEPT_EULA=Y \
  -e MSSQL_SA_PASSWORD='Str0ng!Passw0rd' \
  mcr.microsoft.com/mssql/server:2022-latest
sqlcmd -S localhost -U sa -P 'Str0ng!Passw0rd' -C

# SQLite: no server; creates the file if missing
sqlite3 shop.db < schema.sql
```

CODE EXPLAINED

- → `-p 5432:5432` — publish the container port to localhost
- → `-p` with no value — `mysql` prompts for the password
- → `-C` — `sqlcmd` trusts the self-signed certificate
- → `< schema.sql` — run a file of statements

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	port `5432` · `psql` · user `postgres`
MySQL	port `3306` · `mysql` / `mysqlsh` · user `root`
SQL Server	port `1433` · `sqlcmd` · user `sa`; end batches with `GO`
SQLite	no port · `sqlite3 file.db` · no users

IN AN ORM

```
// prisma.config.ts
datasource: { url: env('DATABASE_URL') },
// app: a driver adapter opens the connection
const url = process.env.DATABASE_URL;
const adapter = new PrismaPg({ connectionString: url });
const prisma = new PrismaClient({ adapter });
```

CONNECTION URLS

ENGINE	FORMAT
PostgreSQL	`postgresql://user:pass@host:5432/db`
MySQL	`mysql://user:pass@host:3306/db`
SQL Server	`sqlserver://host:1433;database=db`
SQLite	`file:./shop.db` (Prisma) · path to the file
JDBC (Java)	`jdbc:postgresql://host:5432/db`
MongoDB	`mongodb://user:pass@host:27017/db`

CLIENT COMMANDS

TASK	PSQL	MYSQL	SQLITE3
list databases	`\l`	`SHOW DATABASES;`	`.databases`
switch database	`\c shop`	`USE shop;`	`.open shop.db`
list tables	`\dt`	`SHOW TABLES;`	`.tables`
describe table	`\d orders`	`DESCRIBE orders;`	`.schema orders`
run a file	`\i f.sql`	`source f.sql`	`.read f.sql`
quit	`\q`	`exit`	`.quit`

⚠ COMMON MISTAKES

- Connecting to `localhost` from inside another container; use the service name
- Forgetting `GO` in `sqlcmd` , so nothing runs
- Committing a connection string with a real password to git

✓ TIPS & BEST PRACTICES

- Name containers and add a volume (`-v pgdata:/var/lib/postgresql/data`) to keep data
- Pin the image tag (`postgres:17`) so upgrades are deliberate
- Use a read-only user for GUI browsing of production

QUICK SUMMARY

- Docker — fastest way to run a server engine
- CLI — `psql` , `mysql` , `sqlcmd` , `sqlite3`
- URL — `engine://user:pass@host:port/db`
- SQLite — the file is the database

Sheet 03

Syntax & Statement Structure

clauses · ; terminator · keyword case · "quoted identifiers" · -- comment · /* */ · logical query order

HOW A STATEMENT IS BUILT

A statement is a sequence of **clauses** (SELECT, FROM, WHERE …), each starting with a keyword, ended by a **semicolon**. Whitespace and line breaks don't matter. **Strings** use single quotes; **double quotes** wrap identifiers that need spaces, capitals or reserved words. Clauses are written in one order but **evaluated in another**: FROM runs first and SELECT near the end, which explains most "why can't I use my alias here?" errors.

RULES OF THUMB

- Keywords in UPPER, names in `snake_case` — a convention, not a rule
- `'text'` is a string; `"text"` is a name
- Double a quote to escape it: `'O''Brien'`
- `;` is required between statements in a script

WRITTEN VS EVALUATED

```sql
SELECT   category,                -- 5. compute output columns
         COUNT(*) AS n
FROM     products                 -- 1. pick the source rows
WHERE    price > 10               -- 2. filter rows
GROUP BY category                 -- 3. form groups
HAVING   COUNT(*) >= 3            -- 4. filter groups
ORDER BY n DESC                   -- 6. sort (aliases OK here)
LIMIT    10;                      -- 7. keep the first 10

/* Literals */
SELECT 'O''Brien'        AS surname,  -- escaped quote
       1.5e3             AS big,      -- 1500
       DATE '2026-01-31' AS due,      -- typed literal
       TRUE              AS ok;

-- Identifiers
select * from PRODUCTS;          -- same as products
CREATE TABLE "Order Notes" (     -- quoted: must be
  "Note Text" TEXT);             -- quoted forever
```

CODE EXPLAINED

→ Numbers on the right — the logical evaluation order

→ `n` works in `ORDER BY` because sorting runs after `SELECT`

→ `n` fails in `WHERE` or `HAVING`: they run before it exists

→ `"Order Notes"` — legal, but every query must quote it

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`"name"`; unquoted names fold to lower case
MySQL	`` `name` ``; `"name"` only with `ANSI_QUOTES` mode
SQL Server	`[name]` or `"name"`; batches end with `GO`
SQLite	`"name"`, also accepts `` `name` `` and `[name]`

IN AN ORM

```
const prisma = new PrismaClient({
  adapter,
  log: ['query'],   // print every SQL statement sent
});
```

- SQLAlchemy: `create_engine(url, echo=True)`

LOGICAL ORDER

#	CLAUSE	DOES	SEES ALIASES?
1	`FROM` / `JOIN`	build the row source	no
2	`WHERE`	filter rows	no
3	`GROUP BY`	form groups	no (MySQL, PG: yes)
4	`HAVING`	filter groups	no
5	`SELECT`	compute columns	defines them
6	`DISTINCT`	remove duplicates	—
7	`ORDER BY`	sort	yes
8	`LIMIT` / `OFFSET`	cut rows	—

TOKENS

WRITE	MEANING
`-- note`	comment to end of line
`/* … */`	block comment (can span lines)
`'text'`	string literal
`"Name"`	quoted identifier
`42 3.14 1e3`	numeric literals
`DATE '2026-01-31'`	typed date literal
`$1 ? @p`	parameter placeholders (sheet 76)

⚠ COMMON MISTAKES

- Using `"double quotes"` for strings; PostgreSQL reads them as column names
- Referencing a `SELECT` alias in `WHERE`
- Creating mixed-case quoted names, then forgetting the quotes

✓ TIPS & BEST PRACTICES

- One clause per line, aligned, makes diffs and reviews easier
- Use `snake_case` names so you never need quotes
- Comment the why, not the what: `-- exclude test accounts`

QUICK SUMMARY

- Clauses — written `SELECT … FROM …`, run `FROM` first
- `'string'` vs `"identifier"`
- `--` and `/* */` comments
- `;` ends each statement

Relational Model & Sample Schema

table · row · column · primary key · foreign key · sample schema

THE RELATIONAL MODEL

Data lives in **tables**: each **row** is one fact, each **column** one typed attribute. A **primary key** identifies a row; a **foreign key** points at another table's key, and links to missing rows are rejected. Every later example uses this **shop schema**: customers place orders of products; employees belong to departments and report to a manager.

TERMS

- **Table** = relation · **row** = tuple · **column** = attribute
- **Primary key** — unique and never NULL
- **Foreign key** — must match a parent key
- **Composite key** — 2+ columns (`order_items`)

NOSQL COMPARISON

- MongoDB would **embed** order items inside each order document
- Customers and products stay separate, **referenced** by `_id`
- No foreign keys: the app keeps references valid

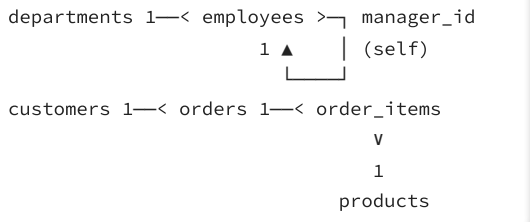
SCHEMA SCRIPT (POSTGRESQL)

```
CREATE TABLE departments (
  dept_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name    VARCHAR(50) NOT NULL UNIQUE,
  budget  NUMERIC(12,2)
);
CREATE TABLE employees (
  emp_id  INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  first_name VARCHAR(50)  NOT NULL,
  last_name  VARCHAR(50)  NOT NULL,
  email      VARCHAR(255) NOT NULL UNIQUE,
  dept_id    INT REFERENCES departments,
  manager_id INT REFERENCES employees,
  hire_date  DATE NOT NULL,
  salary     NUMERIC(10,2)
);
CREATE TABLE customers (
  customer_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(255) UNIQUE,
  city        VARCHAR(50),
  country     CHAR(2),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

SCHEMA SCRIPT (CONTINUED)

```
CREATE TABLE products (
  product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name       VARCHAR(100)  NOT NULL,
  category   VARCHAR(50),
  price      NUMERIC(10,2) NOT NULL CHECK (price >= 0),
  stock      INT NOT NULL DEFAULT 0
);
CREATE TABLE orders (
  order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id INT NOT NULL REFERENCES customers,
  order_date  DATE NOT NULL DEFAULT CURRENT_DATE,
  status      VARCHAR(20) NOT NULL DEFAULT 'pending',
  total       NUMERIC(10,2)
);
CREATE TABLE order_items (
  order_id   INT REFERENCES orders ON DELETE CASCADE,
  product_id INT REFERENCES products,
  quantity   INT NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
```

ER DIAGRAM



DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
MySQL	`INT AUTO_INCREMENT`; `DATETIME` for `TIMESTAMPTZ`
SQL Server	`INT IDENTITY(1,1)`; `DATETIMEOFFSET`
SQLite	`INTEGER PRIMARY KEY`; run `PRAGMA foreign_keys = ON`

IN AN ORM (PRISMA SCHEMA)

```
model Order {
  id         Int      @id @default(autoincrement()) @map("order_id")
  customerId Int      @map("customer_id")
  customer   Customer @relation(fields: [customerId], references: [id])
  items      OrderItem[]
  @@map("orders")
}
```

⚠ COMMON MISTAKES

- Storing a list in one column (`'1,4,9'`)
- Assuming SQLite enforces foreign keys by default
- Using a changeable value (email) as the primary key

✓ TIPS & BEST PRACTICES

- `orders.total` is a stored copy of the item sum; sheet 61 covers the trade-off
- `status` values: `pending` , `paid` , `shipped` , `cancelled`
- ORM boxes use camelCase Prisma fields mapped to these columns

QUICK SUMMARY

- Row = fact, column = attribute
- PK identifies, FK links
- 6 tables, used by every later sheet

Sheet 05

Which Clause When

WHERE vs HAVING · GROUP BY vs ORDER BY vs PARTITION BY · DISTINCT vs GROUP BY · JOIN vs subquery vs CTE · IN vs EXISTS · UNION vs UNION ALL · DELETE vs TRUNCATE vs DROP

FILTERING

PICK	USE IT WHEN	#
`WHERE`	filter **rows** before grouping; no aggregates allowed	14
`HAVING`	filter **groups** after `GROUP BY`, using aggregates	29
`FILTER (WHERE …)`	count or sum only some rows inside one aggregate	29
`CASE in SELECT`	label or transform values, not remove rows	21
`QUALIFY`	filter on a window result (DuckDB, Snowflake); elsewhere wrap in a CTE	49

GROUPING & SORTING

PICK	USE IT WHEN	#
`GROUP BY`	collapse rows to **one per group** with aggregates	28
`PARTITION BY`	compute per group but **keep every row** (window)	44
`ORDER BY`	sort the final output; the only way to guarantee order	15
`DISTINCT`	drop duplicate output rows; no aggregates needed	17
`GROUP BY vs DISTINCT`	same rows without aggregates; prefer `DISTINCT` for intent	17
`ROLLUP / CUBE`	add subtotal and grand-total rows	30

PAGING

PICK	USE IT WHEN	#
`LIMIT / OFFSET`	first pages, small offsets (PG, MySQL, SQLite)	16
`FETCH FIRST n ROWS`	standard form; also SQL Server with `OFFSET`	16
`TOP (n)`	SQL Server only	16
`keyset (WHERE id > last)`	deep pages and infinite scroll; stays fast	16

COMBINING TABLES

PICK	USE IT WHEN	#
`INNER JOIN`	need columns from both tables, matches only	32
`LEFT JOIN`	keep every left row, even without a match	33
`scalar subquery`	need **one value** from another query	38
`derived table`	a one-off step in `FROM (…) AS t`	40
`CTE (WITH)`	name steps, reuse one, or read top to bottom	41
`recursive CTE`	walk a hierarchy or generate a series	42
`LATERAL / APPLY`	run a subquery **per row** (top-N per customer)	37
`view`	save a query for many callers	64

EXISTENCE TESTS

PICK	USE IT WHEN	#
`IN (subquery)`	match against a list; fine for semi-joins	38
`EXISTS`	"has at least one"; stops at the first match	36
`NOT EXISTS`	"has none"; NULL-safe anti-join	36
`NOT IN`	only when the list **cannot contain NULL**	36
`LEFT JOIN … IS NULL`	anti-join when you also need left columns	36

STACKING RESULTS

PICK	USE IT WHEN	#
`UNION ALL`	stack two queries' rows, keep duplicates (fast)	43
`UNION`	stack and remove duplicates (sorts or hashes)	43
`INTERSECT`	rows present in both queries	43
`EXCEPT`	rows in the first query but not the second	43

CHANGING DATA

PICK	USE IT WHEN	#
`INSERT … VALUES`	add rows you have in hand	50
`INSERT … SELECT`	copy rows from a query	50
`UPDATE`	change columns in existing rows	51
`ON CONFLICT / MERGE`	insert or update in one statement (upsert)	53
`COPY / LOAD DATA`	load thousands of rows from a file	54

REMOVING DATA

PICK	USE IT WHEN	#
`DELETE … WHERE`	remove **some** rows; triggers fire; can roll back	52
`TRUNCATE`	empty a whole table fast; resets identity with `RESTART IDENTITY`	52
`DROP TABLE`	remove the table itself: data, indexes, grants	59
`soft delete`	`UPDATE … SET deleted_at = now()`; keep history	52

Function Directory

string · numeric · date & time · conversion · NULL handling · aggregate · window · what it does / use it for

STRING

FUNCTION	DOES	#
`LENGTH(s)`	character count	22
`UPPER(s) / LOWER(s)`	change case	22
`INITCAP(s)`	Title Case (PG)	22
`TRIM(s)`	strip spaces / chars	22
`LPAD(s, n, c)`	pad to width	22
`REPEAT(s, n)`	repeat n times	22
`CONCAT(a, b)`	join, NULL → ''	20
`CONCAT_WS(sep, …)`	join with separator	20
`a || b`	join (standard)	20
`SUBSTRING(s, i, n)`	slice	23
`LEFT(s, n) / RIGHT`	first / last n	23
`POSITION(x IN s)`	index, 0 = absent	23
`REPLACE(s, a, b)`	replace all a	23
`SPLIT_PART(s, d, n)`	nth delimited field	23
`REVERSE(s)`	reverse text	23
`REGEXP_REPLACE(…)`	regex replace	19

NUMERIC

FUNCTION	DOES	#
`ROUND(x, d)`	round to d places	24
`CEIL(x) / FLOOR(x)`	round up / down	24
`TRUNC(x, d)`	cut, no rounding	24
`ABS(x)`	absolute value	24
`MOD(a, b) / a % b`	remainder	24
`POWER(x, y)`	exponent	24
`SQRT(x)`	square root	24
`RANDOM()`	0 ≤ r < 1	24

DATE & TIME

FUNCTION	DOES	#
`NOW()`	date + time now	25
`CURRENT_DATE`	today	25
`EXTRACT(p FROM d)`	year, month, dow…	25
`DATE_TRUNC('month', d)`	round down to unit	25
`d + INTERVAL '7 days'`	add time	25
`AGE(a, b)`	interval between	25
`DATEADD / DATEDIFF`	SQL Server math	25
`TO_CHAR(d, fmt)`	format as text	25
`d AT TIME ZONE 'UTC'`	convert zone	25
`GENERATE_SERIES(a, b)`	one row per step	79

CONVERSION

FUNCTION	DOES	#
`CAST(x AS t)`	standard cast	12
`x::t`	PG shorthand	12
`CONVERT(t, x, style)`	SQL Server cast	12
`TRY_CAST(x AS t)`	NULL on failure	12
`TO_NUMBER / TO_DATE`	parse with format	12

CONDITIONAL & NULL

FUNCTION	DOES	#
`COALESCE(a, b, …)`	first non-NULL	26
`NULLIF(a, b)`	NULL if a = b	26
`GREATEST / LEAST`	max / min of args	26
`IFNULL / ISNULL`	MySQL / SQL Server	26
`CASE WHEN … END`	if / else	21
`IIF(c, a, b)`	inline if	21

AGGREGATE

FUNCTION	DOES	#
`COUNT(*)`	rows in group	27
`COUNT(col)`	non-NULL values	27
`COUNT(DISTINCT c)`	distinct values	17
`SUM(x) / AVG(x)`	total / mean	27
`MIN(x) / MAX(x)`	smallest / largest	27
`STRING_AGG(s, ',')`	join group text	31
`GROUP_CONCAT(s)`	MySQL / SQLite	31
`ARRAY_AGG(x)`	collect to array	31
`JSON_AGG(x)`	collect to JSON	31
`GROUPING(col)`	1 on subtotal row	30

WINDOW

FUNCTION	DOES	#
`ROW_NUMBER()`	1, 2, 3 …	45
`RANK()`	1, 2, 2, 4	45
`DENSE_RANK()`	1, 2, 2, 3	45
`NTILE(n)`	n buckets	45
`PERCENT_RANK()`	rank as 0–1	45
`CUME_DIST()`	share ≤ this row	45
`LAG(x) / LEAD(x)`	previous / next	46
`FIRST_VALUE(x)`	first in frame	46
`LAST_VALUE(x)`	last in frame	46
`NTH_VALUE(x, n)`	nth in frame	46
`SUM(x) OVER (…)`	running total	47

COMMON TASKS → FUNCTION

TASK	REACH FOR	#
clean user input	`LOWER(TRIM(email))`	22
default for a missing value	`COALESCE(phone, 'n/a')`	26
avoid divide-by-zero	`total / NULLIF(qty, 0)`	26
monthly buckets	`DATE_TRUNC('month', order_date)`	25

MORE TASKS → FUNCTION

TASK	REACH FOR	#
top N per group	`ROW_NUMBER() OVER (PARTITION BY …)`	49
change since the previous row	`x - LAG(x) OVER (ORDER BY d)`	46
comma-separated list per group	`STRING_AGG(name, ', ')`	31
parse text that might be bad	`TRY_CAST` / `CASE` + regex check	12

Numeric Types

INTEGER · BIGINT · SMALLINT · NUMERIC(p,s) · DECIMAL · REAL · DOUBLE PRECISION

EXACT VS APPROXIMATE

Integers (`SMALLINT`, `INTEGER`, `BIGINT`) hold whole numbers exactly. **`NUMERIC(p,s)`** (same as `DECIMAL`) holds exact decimals: `p` digits in total, `s` after the point, so it is the type for money. **`REAL`** and **`DOUBLE PRECISION`** are binary floating point: fast and compact but approximate, so `0.1 + 0.2` is not quite `0.3`. A value that doesn't fit raises an **error**; nothing wraps around.

NUMBER FACTS

- `NUMERIC` = `DECIMAL` in every engine here
- Casting to fewer decimals **rounds**
- `int / int` drops the fraction (sheet 20)
- No `UNSIGNED` in PostgreSQL; use `CHECK (x >= 0)`

NOSQL COMPARISON

- MongoDB numbers are `double` unless you say otherwise
- `NumberInt` / `NumberLong` for int32 / int64
- `Decimal128` for money; JSON itself has one number type

NUMERIC TYPES

```
CREATE TABLE measurements (
  id        BIGINT GENERATED ALWAYS AS IDENTITY,
  qty       SMALLINT,          -- ±32,767
  stock     INTEGER,           -- ±2.1 billion
  price     NUMERIC(10,2),     -- max 99,999,999.99
  weight_kg DOUBLE PRECISION   -- ~15 digits, approx
);

SELECT 0.1 + 0.2 = 0.3;                  -- true
SELECT 0.1::float8 + 0.2::float8 = 0.3;  -- false
SELECT 12.345::NUMERIC(5,2);             -- 12.35
SELECT 1234.5::NUMERIC(5,2);
-- ERROR: numeric field overflow
SELECT 2147483647::INT + 1;
-- ERROR: integer out of range
SELECT 7 / 2, 7 / 2.0;            -- 3 | 3.5000000000000000
SELECT pg_typeof(7), pg_typeof(7.0);  -- integer | numeric
```

CODE EXPLAINED

- `NUMERIC(10,2)` — 10 digits total, 2 after the point
- Plain `0.1` is a `numeric` literal in PostgreSQL, so it's exact
- `float8` = `DOUBLE PRECISION`: binary, 15–17 significant digits
- `7 / 2` is integer division; one decimal operand fixes it

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`SMALLINT` `INT` `BIGINT` `NUMERIC` `REAL` `FLOAT8`
MySQL	adds `TINYINT`, `MEDIUMINT`, `UNSIGNED` integers
SQL Server	`TINYINT` is 0–255; avoid `MONEY`; `FLOAT(53)`
SQLite	stores `INTEGER` or `REAL`; `NUMERIC` is only an affinity

IN AN ORM

```
model Product {
  price  Decimal @db.Decimal(10, 2)  // Prisma.Decimal
  stock  Int
  views  BigInt                      // JS bigint
  weight Float
}
```

RANGES

TYPE	BYTES	RANGE / PRECISION
`SMALLINT`	2	-32,768 … 32,767
`INTEGER`	4	-2,147,483,648 … 2,147,483,647
`BIGINT`	8	about ±9.22 × 10^{18}
`NUMERIC(p,s)`	varies	exact; up to 1,000 digits declared (PG)
`REAL`	4	about 6 significant digits
`DOUBLE PRECISION`	8	about 15 significant digits

PICK A TYPE

STORING	USE
`prices, money`	`NUMERIC(12,2)`
`tax or interest rates`	`NUMERIC(6,4)` → `0.0825`
`counts, quantities`	`INTEGER`
`IDs in busy tables`	`BIGINT`
`sensor readings, science`	`DOUBLE PRECISION`
`percent 0-100 with 1 decimal`	`NUMERIC(4,1)`

⚠ COMMON MISTAKES

- Using `FLOAT` for money and getting `19.989999999` totals
- `INTEGER` keys running out at 2.1 billion rows
- SQL Server `AVG(int_col)` returns an integer; cast to decimal first

✓ TIPS & BEST PRACTICES

- Always give `NUMERIC` a precision and scale
- Use `BIGINT` keys for tables that grow without limit
- Compare floats with a tolerance: `abs(a - b) < 1e-9`

QUICK SUMMARY

- Whole numbers — `INTEGER` / `BIGINT`
- Exact decimals — `NUMERIC(p,s)`
- Approximate — `REAL` / `DOUBLE PRECISION`
- Overflow — an error, never a wrap

Character Types

STORING TEXT

CHAR(n) is fixed width and pads with spaces; **VARCHAR(n)** stores up to n characters; **TEXT** has no practical limit. In PostgreSQL all three perform the same, so TEXT (plus a CHECK if you need a limit) is the usual choice. A **collation** decides how text compares and sorts: case and accent sensitivity, and whether `'B'` comes before `'a'`. Use **UTF-8** so every language and emoji fits.

TEXT FACTS

- Strings use single quotes: `'O''Brien'`
- Too-long values are an **error** (PG, MySQL strict mode)
- `CHAR` trailing spaces are ignored when comparing
- MySQL `utf8` is 3-byte; use `utf8mb4`

NOSQL COMPARISON

- MongoDB strings are UTF-8 with no length type; only the 16 MB document cap
- Limit length with `$jsonSchema maxLength`
- Collation is set per collection, index or query

CHARACTER TYPES

```
CREATE TABLE contacts (
  country CHAR(2),          -- always 2: 'US'
  phone   VARCHAR(20),      -- up to 20 characters
  bio     TEXT,             -- no practical limit
  CHECK (length(bio) <= 5000)
);

SELECT 'ab'::CHAR(5) || '|';              -- ab|
SELECT length('ab'::CHAR(5));             -- 2
SELECT length('é'), octet_length('é');    -- 1 | 2
INSERT INTO contacts (country) VALUES ('USA');
-- ERROR: value too long for type character(2)

-- Case-insensitive comparison
SELECT 'abc' = 'ABC';                     -- false
CREATE COLLATION ci (provider = icu,
  locale = 'und-u-ks-level2', deterministic = false);
SELECT 'abc' = 'ABC' COLLATE ci;          -- true
SELECT name FROM customers
ORDER  BY name COLLATE "C";               -- byte order
```

CODE EXPLAINED

- → `CHAR(5)` pads to 5, but the padding is dropped as text
- → `length` counts characters; `octet_length` counts bytes
- → `ks-level2` — ICU strength that ignores case
- → `COLLATE "C"` — plain byte order: fast, case-sensitive

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`TEXT` preferred; `VARCHAR` with no `n` = `TEXT`
MySQL	`VARCHAR(n)` ≤ 65,535 bytes/row; `TEXT` = 64 KB
SQL Server	`VARCHAR(n)` counts bytes; `NVARCHAR(n)` UTF-16; `(MAX)`
SQLite	all are `TEXT` affinity; `n` is not enforced

IN AN ORM

```
model Customer {
  country String  @db.Char(2)
  phone   String? @db.VarChar(20)
  bio     String? @db.Text
}
```

TYPES

TYPE	STORES	USE FOR
`CHAR(n)`	exactly n, space-padded	fixed codes: `'US'`
`VARCHAR(n)`	up to n characters	names, emails
`TEXT`	any length	bodies, notes
`NVARCHAR(n)`	UTF-16 (SQL Server)	Unicode on old collations
`CITEXT`	case-insensitive text (PG ext.)	emails, usernames

COMMON COLLATIONS

COLLATION	BEHAVIOR
`"C" (PG)`	byte order; case-sensitive; fastest
`en-US-x-icu (PG)`	language-aware sorting
`utf8mb4_0900_ai_ci`	MySQL 8 default: ignores accents + case
`SQL_Latin1_General_CP1_CI_AS`	SQL Server default: case-insensitive
`NOCASE (SQLite)`	ignores ASCII case only

⚠ COMMON MISTAKES

- Using `CHAR(n)` for variable data and fighting padding
- MySQL `utf8` columns rejecting emoji; switch to `utf8mb4`
- Assuming `'abc' = 'ABC'` behaves the same on every engine

✓ TIPS & BEST PRACTICES

- PostgreSQL: `TEXT` + `CHECK (length(x) <= n)`
- Choose the collation when creating the database
- Case-insensitive unique email: index `lower(email)`

QUICK SUMMARY

- `CHAR` fixed · `VARCHAR` limited · `TEXT` open
- Collation = compare + sort rules
- UTF-8 everywhere (`utf8mb4` in MySQL)

Date & Time Types

DATE · TIME · TIMESTAMP · TIMESTAMPTZ · INTERVAL · DATETIME2 · time zones

DATES, TIMES & ZONES

DATE is a calendar day, **TIME** a time of day, **TIMESTAMP** a date plus time with **no zone**, and **TIMESTAMPTZ** an **instant**: PostgreSQL converts it to UTC on the way in and back to the session's zone on the way out. **INTERVAL** is a duration. Store moments that happened (orders, logins) as `TIMESTAMPTZ`; store wall-clock plans ("9:00 in Chicago") as `TIMESTAMP` plus a zone name.

TIME FACTS

- Write literals as ISO 8601: `'2026-10-18 14:30'`
- `TIMESTAMPTZ` doesn't store the zone; it converts
- `SET TIME ZONE` changes display, not data
- PG precision is microseconds

NOSQL COMPARISON

- MongoDB `Date` is UTC milliseconds since 1970; no zone kept
- Write `ISODate("2026-10-18T14:00:00Z")`
- DynamoDB has no date type: ISO strings or epoch numbers

DATE & TIME TYPES

```
CREATE TABLE events (
  event_day DATE,          -- 2026-10-18
  starts    TIME,          -- 09:30:00
  local_ts  TIMESTAMP,     -- wall clock, no zone
  happened  TIMESTAMPTZ,   -- an instant
  length    INTERVAL       -- 01:30:00
);

SET TIME ZONE 'America/Chicago';
SELECT TIMESTAMPTZ '2026-10-18 14:00:00+00';
-- 2026-10-18 09:00:00-05

SELECT TIMESTAMP '2026-10-18 14:00' AT TIME ZONE 'UTC';
-- read the wall time as UTC → timestamptz

SELECT DATE '2026-03-31' + 1;          -- 2026-04-01
SELECT DATE '2026-01-31' + INTERVAL '1 month';
-- 2026-02-28 00:00:00 (clamped to month end)
SELECT INTERVAL '90 minutes';          -- 01:30:00
SELECT now()::date, now()::time(0);
```

CODE EXPLAINED

- → `+00` in the literal fixes the instant; display follows the session
- → `AT TIME ZONE` on a `TIMESTAMP` returns a `TIMESTAMPTZ`
- → `date + 1` adds days; `+ INTERVAL` returns a timestamp
- → Month math clamps: Jan 31 + 1 month = Feb 28

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`TIMESTAMPTZ` stores UTC; real `INTERVAL` type
MySQL	`DATETIME` no zone; `TIMESTAMP` UTC, 1970–2038
SQL Server	`DATETIME2(7)`; `DATETIMEOFFSET` keeps the offset
SQLite	no date type: ISO `TEXT`, Unix `INTEGER` or Julian `REAL`

IN AN ORM

```
model Order {
  orderDate DateTime @db.Date
  createdAt DateTime @default(now()) @db.Timestamptz(6)
}
// a JS Date is always an instant (UTC milliseconds)
```

WHICH TYPE?

STORING	USE
`birthday, due date`	`DATE`
`created_at, paid_at`	`TIMESTAMPTZ`
`store opening time`	`TIME`
`meeting at 9:00 local`	`TIMESTAMP` + zone name column
`subscription length`	`INTERVAL '1 year'`
`Unix epoch from an API`	`to_timestamp(1760000000)`

LITERALS

WRITE	MEANS
`DATE '2026-10-18'`	a calendar day
`TIME '14:30:05'`	a time of day
`'2026-10-18T14:30Z'`	ISO instant in UTC
`'2026-10-18 09:30-05'`	instant with an offset
`INTERVAL '2 hours 30 minutes'`	a duration
`'today' · 'now'`	special inputs (PG)

⚠ COMMON MISTAKES

- Using `TIMESTAMP` for instants, then servers in other zones disagree
- `WHERE ts <= '2026-10-18'` misses everything after midnight
- Storing dates as text like `'10/18/2026'`

✓ TIPS & BEST PRACTICES

- Run servers in UTC; convert only for display
- Use half-open ranges: `>= day AND < day + 1`
- Store IANA zone names (`'Europe/Oslo'`), not fixed offsets

QUICK SUMMARY

- `DATE` · `TIME` · `TIMESTAMP` · `TIMESTAMPTZ` · `INTERVAL`
- Instants → `TIMESTAMPTZ`
- ISO 8601 literals everywhere

Boolean, Binary, UUID & Enum Types

THE OTHER TYPES

BOOLEAN holds `true`, `false` or NULL. **Binary** types (BYTEA, BLOB) hold raw bytes such as hashes or small files. **UUID** is a 128-bit identifier you can generate anywhere without coordination: v4 is random, v7 starts with a timestamp so new values sort together. An **enum** limits a column to a fixed list of labels; in PostgreSQL you create it once with `CREATE TYPE` and reuse it.

FACTS

- `WHERE is_active` is enough; no `= true` needed
- Enum values sort in declared order
- PG can add enum values but not drop them
- `uuidv7()` is built in from PostgreSQL 18

NOSQL COMPARISON

- MongoDB has `bool` and `BinData`; `ObjectId` plays the UUID role
- `ObjectId` starts with a timestamp, like UUIDv7
- Enums are enforced with `$jsonSchema enum: [...]`

BOOLEAN, BINARY, UUID & ENUM

```
CREATE TYPE order_status AS ENUM
  ('pending', 'paid', 'shipped', 'cancelled');

CREATE TABLE payments (
  payment_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  status     order_status NOT NULL DEFAULT 'pending',
  is_refund  BOOLEAN NOT NULL DEFAULT false,
  receipt    BYTEA
);

SELECT * FROM payments WHERE is_refund;
SELECT * FROM payments WHERE NOT is_refund;
SELECT 'yes'::boolean, 'f'::boolean;     -- t | f
SELECT * FROM payments ORDER BY status;  -- enum order
ALTER TYPE order_status ADD VALUE 'refunded'
  AFTER 'shipped';

SELECT gen_random_uuid();   -- v4: random
SELECT uuidv7();            -- PG 18+: time-ordered
SELECT decode('DEADBEEF', 'hex');        -- \xdeadbeef
```

CODE EXPLAINED

- `CREATE TYPE … AS ENUM` — reusable type with fixed labels
- `DEFAULT gen_random_uuid()` — the database fills the key
- Boolean input accepts `true/false`, `yes/no`, `t/f`, `1/0`
- `decode(…, 'hex')` turns a hex string into bytes

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`BOOLEAN`, `UUID`, `CREATE TYPE … ENUM`, `BYTEA`
MySQL	`BOOLEAN` = `TINYINT(1)`; inline `ENUM('a','b')`; `BLOB`
SQL Server	`BIT`; `UNIQUEIDENTIFIER` + `NEWID()`; no enum
SQLite	booleans stored as 0 / 1; `BLOB`; UUIDs as `TEXT`

IN AN ORM

```
enum OrderStatus { pending paid shipped cancelled }
model Payment {
  id       String      @id @default(uuid(7)) @db.Uuid
  status   OrderStatus @default(pending)
  isRefund Boolean     @default(false)
}
```

CHOOSING

NEED	USE
yes / no flag	`BOOLEAN NOT NULL DEFAULT false`
short fixed list	enum, or `CHECK (x IN (…))`
list that changes or has data	lookup table + foreign key
public or merge-safe IDs	`UUID` (v7 for insert order)
hash, token, small file	`BYTEA` / `BLOB`
large files	object storage; keep the URL

UUID VERSIONS

VERSION	BUILT FROM	SORTS BY TIME?
v1	timestamp + MAC address	no (fields reordered)
v4	122 random bits	no
v7	Unix ms timestamp + random	yes
nil	all zeros	—

⚠ COMMON MISTAKES

- Random v4 UUID keys scattering inserts across a big index
- Storing UUIDs as `VARCHAR(36)`: 2× the space, slower joins
- Needing to remove an enum value later; PG can't drop one

✓ TIPS & BEST PRACTICES

- Name booleans as questions: `is_active`, `has_paid`
- Prefer `CHECK` or a lookup table when values may change
- Use the native `UUID` type, or `BINARY(16)` in MySQL

QUICK SUMMARY

- `BOOLEAN` — true / false / NULL
- `BYTEA` / `BLOB` — raw bytes
- `UUID` — v4 random, v7 time-ordered
- Enum — fixed label list

Sheet 11

NULL & Three-Valued Logic

NULL · IS NULL · IS NOT NULL · UNKNOWN · IS DISTINCT FROM · NULL in comparisons

UNKNOWN VALUES

NULL means "unknown" or "not applicable". It is not zero and not an empty string. Any comparison with NULL returns **UNKNOWN**, and WHERE keeps only rows that are **TRUE**, so `city = NULL` never matches anything. Test with **`IS NULL`**. Most operators return NULL when an input is NULL. **`IS DISTINCT FROM`** compares values but treats two NULLs as equal.

NULL RULES

- `= NULL` is always UNKNOWN; use `IS NULL`
- `1 + NULL` and `'a' || NULL` are NULL
- `TRUE OR NULL` is TRUE; `FALSE AND NULL` is FALSE
- Aggregates skip NULLs (sheet 27)

NOSQL COMPARISON

- MongoDB separates a **missing** field from `null`
- `{ city: null }` matches both; `{ city: { $exists: false } }` only missing
- No three-valued logic: comparisons are just true or false

NULL IN PRACTICE

```sql
SELECT NULL = NULL;              -- NULL (unknown)
SELECT NULL IS NULL;             -- true
SELECT 1 + NULL, 'a' || NULL;    -- NULL | NULL

-- Customers with no city
SELECT name FROM customers WHERE city IS NULL;
SELECT name FROM customers WHERE city = NULL; -- 0 rows!

-- "Not Oslo" silently skips NULL cities
SELECT count(*) FROM customers WHERE city <> 'Oslo';
SELECT count(*) FROM customers
WHERE  city IS DISTINCT FROM 'Oslo';     -- counts NULLs

-- NULL-safe match between columns
SELECT a.emp_id, b.emp_id
FROM   employees a
JOIN   employees b
  ON   a.manager_id IS NOT DISTINCT FROM b.manager_id
 AND   a.emp_id < b.emp_id;

SELECT TRUE OR NULL, FALSE AND NULL;   -- true | false
```

CODE EXPLAINED

→ `city <> 'Oslo'` is UNKNOWN for NULL cities, so they drop

→ `IS DISTINCT FROM` — NULL vs value is TRUE, NULL vs NULL FALSE

→ The self-join pairs employees with the same manager, NULL too

→ `OR` with a TRUE side is TRUE even if the other is unknown

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`IS [NOT] DISTINCT FROM`; `'a' || NULL` is NULL
MySQL	`<=>` is NULL-safe equals; `CONCAT('a', NULL)` is NULL
SQL Server	`IS [NOT] DISTINCT FROM` from 2022; `+` with NULL is NULL
SQLite	`a IS b` / `a IS NOT b` are NULL-safe

IN AN ORM

```ts
await prisma.customer.findMany({
  where: { city: null },           // IS NULL
});
await prisma.customer.findMany({
  where: { city: { not: null } },  // IS NOT NULL
});
```

TRUTH TABLE

P	Q	P AND Q	P OR Q
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN
NOT UNKNOWN		UNKNOWN	

WHERE NULLS BITE

SITUATION	WHAT HAPPENS
`COUNT(col)`	skips NULL rows; `COUNT(*)` doesn't
`ORDER BY col`	NULLs last in PG, first in MySQL / SQL Server / SQLite
`GROUP BY col`	all NULLs form one group
`UNIQUE col`	many NULLs allowed (SQL Server: only one)
`x NOT IN (…, NULL)`	never TRUE → no rows (sheet 36)
`CHECK (x > 0)`	passes when `x` is NULL

⚠ COMMON MISTAKES

- Writing `= NULL` or `<> NULL`
- Losing NULL rows with `<>` filters
- `NOT IN` against a subquery that can return NULL

✓ TIPS & BEST PRACTICES

- Decide per column: `NOT NULL` unless unknown is real
- Use `COALESCE` for display defaults (sheet 26)
- Reach for `IS DISTINCT FROM` when comparing nullable columns

QUICK SUMMARY

- NULL = unknown, not empty
- Comparisons → UNKNOWN; `WHERE` keeps TRUE only
- `IS NULL` · `IS DISTINCT FROM`

Type Conversion & Casting

CHANGING TYPES

`CAST(x AS type)` is the standard way to convert; PostgreSQL also accepts **`x::type`**. When types differ the engine may convert **implicitly**: PostgreSQL is strict and errors on mismatches it can't resolve, while MySQL and SQLite quietly turn `'12abc'` into 12. SQL Server's **`TRY_CAST`** and **`TRY_CONVERT`** return NULL instead of failing. For formatted text and dates, use **formatting functions** (`to_char`, `to_date`) rather than plain casts.

CASTING RULES

- Numeric → integer **rounds** in PG and MySQL, **truncates** in SQL Server and SQLite
- Text → number fails on bad input (PG, SQL Server)
- A cast on a column in `WHERE` can block an index (sheet 74)
- PG 16+: `pg_input_is_valid(text, type)` tests first

CASTING

```
SELECT CAST('42' AS INTEGER) + 1;          -- 43
SELECT '42'::int, '3.14'::numeric(4,2);    -- PG shorthand
SELECT CAST(9.99 AS INTEGER);              -- 10
SELECT CAST('2026-10-18' AS DATE);
SELECT order_id::text || '-A' FROM orders;
SELECT CAST('abc' AS INTEGER);
-- ERROR: invalid input syntax for type integer

-- Check before casting
SELECT pg_input_is_valid('abc', 'integer');  -- false
SELECT CASE WHEN qty ~ '^[0-9]+$'
            THEN qty::int END AS qty
FROM   staging_rows;

-- Format and parse with functions
SELECT to_char(1234.5, 'FM9,999.00');      -- 1,234.50
SELECT to_number('1,234.50', '9G999D99');  -- 1234.50
SELECT to_date('18/10/2026', 'DD/MM/YYYY');
SELECT to_timestamp(1760000000);           -- epoch → tz
```

CODE EXPLAINED

- → `::` — PostgreSQL-only shorthand for `CAST`
- → `CAST(9.99 AS INTEGER)` rounds to 10; it doesn't truncate
- → `~ '^[0-9]+$'` — regex guard so the cast can't fail (sheet 19)
- → `FM` drops padding; `G` and `D` are locale separators

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`CAST` and `::`; strict about mismatched types
MySQL	`CAST(x AS SIGNED / DECIMAL(10,2) / CHAR)`; loose
SQL Server	`TRY_CAST`, `TRY_CONVERT`, `CONVERT(varchar, d, 23)`
SQLite	type affinity: `CAST` never errors; `'abc'` → `0`

IN AN ORM

```
// Prisma validates input types before SQL runs.
// For casts inside a query, drop to raw SQL:
const rows = await prisma.$queryRaw`
  SELECT order_id::text AS id, total::float8 AS amt
  FROM   orders`;
```

COMMON CASTS (PG)

WANT	WRITE
`text → integer`	`'42'::int`
`numeric → integer`	`round(x)::int` (explicit)
`anything → text`	`x::text`
`text → date`	`'2026-10-18'::date`
`timestamp → date`	`created_at::date`
`boolean → integer`	`is_paid::int` → 0 / 1

IMPLICIT CONVERSION

EXPRESSION	PG	MYSQL	SQLITE
`'5' + 3`	8	8	8
`'abc' + 1`	error	1 + warning	1
`5 / 2`	2	2.5000	2
`1 = '1'`	true	1	0 (false)
`'10' < '9'`	true	1	1

⚠ COMMON MISTAKES

- MySQL `CAST(x AS INT)` ; the target must be `SIGNED`
- Casting a column in `WHERE` and losing the index
- Relying on MySQL turning bad text into 0 instead of failing

✓ TIPS & BEST PRACTICES

- Cast the literal, not the column: `WHERE d = '2026-10-18'::date`
- Validate staging data before casting the whole table
- Prefer `to_char` / `FORMAT` for display, casts for logic

QUICK SUMMARY

- `CAST(x AS t)` · `x::t` · `TRY_CAST`
- Implicit rules differ by engine
- Format with functions, not casts

SELECT Basics

SELECT · FROM · * · column list · AS alias · computed columns · SELECT without FROM

CHOOSING COLUMNS

SELECT lists what to return and **FROM** names the table it comes from. * returns every column; a **column list** returns only the ones you name, in that order. Each item can be an **expression** evaluated once per row (`price * 0.9`), and **AS** gives it an output name (an alias). With no FROM, SELECT just evaluates its expressions once, which is handy for testing functions.

SELECT FACTS

- Output column order = the order you list them
- `AS` is optional: `price * 2 doubled` works
- A missing comma makes an alias: `SELECT name price`
- Table aliases shorten joins: `FROM products p`

NOSQL COMPARISON

- MongoDB **projection** picks fields: `find({}, { name: 1, price: 1, _id: 0 })`
- Computed fields need the aggregation pipeline: `$project`
- `{ sale: { $multiply: ['$price', 0.9] } }`

SELECT

```sql
SELECT * FROM products;                  -- every column

SELECT name, price
FROM   products;                         -- chosen columns

SELECT name            AS product,
       price * 0.9     AS sale_price,    -- computed
       price * stock   AS stock_value,
       upper(category) AS category,
       'USD'           AS currency       -- constant
FROM   products;

SELECT p.name, p.price                   -- table alias
FROM   products AS p;

SELECT first_name || ' ' || last_name AS "Full Name"
FROM   employees;

SELECT 2 + 3 AS five, now() AS at;       -- no FROM
SELECT current_user, version();
```

CODE EXPLAINED

→ `*` is fine for exploring; list columns in application code

→ Each expression runs once per row of `products`

→ `"Full Name"` — quoted alias keeps the space and capitals

→ No `FROM` → exactly one row of computed values

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`SELECT 1;` works; `||` concatenates
MySQL	`SELECT 1;` works; use `CONCAT(a, ' ', b)`
SQL Server	`SELECT 1;` works; `alias = expr` form also allowed
SQLite	`SELECT 1;` works; `||` concatenates

IN AN ORM

```sql
await prisma.product.findMany({
  select: { name: true, price: true },
});
// computed columns: derive in JS, or $queryRaw
```

WHAT CAN GO IN SELECT

ITEM	EXAMPLE
column	`name`
all columns	`* · p.*`
arithmetic	`price * stock`
function call	`upper(name) · now()`
literal	`'USD' · 1`
CASE expression	`CASE WHEN stock = 0 THEN 'out' END` (sheet 21)
scalar subquery	`(SELECT count(*) FROM orders)` (sheet 38)

WHERE AN OUTPUT ALIAS WORKS

CLAUSE	ALIAS USABLE?
`WHERE`	no — runs before `SELECT` (SQLite allows it)
`GROUP BY`	PG, MySQL, SQLite: yes · SQL Server: no
`HAVING`	MySQL, SQLite: yes · PG, SQL Server: no
`ORDER BY`	yes, everywhere
`outer query`	yes, through a derived table or CTE

⚠ COMMON MISTAKES

- `SELECT *` in app code: breaks or slows down when columns change
- A missing comma silently renaming a column
- Using an alias in `WHERE`

✓ TIPS & BEST PRACTICES

- Alias every computed column so callers get a clear name
- Qualify columns (`p.name`) as soon as there's a join
- Use `SELECT` without `FROM` to try out a function

QUICK SUMMARY

- `SELECT cols FROM table`
- `AS` names output columns
- Expressions run per row

Filtering with WHERE

WHERE · AND · OR · NOT · parentheses · precedence · aliases in WHERE

KEEPING SOME ROWS

WHERE keeps only the rows whose condition is **TRUE** (not FALSE, not UNKNOWN). Join conditions with **AND**, **OR** and **NOT**. AND binds tighter than OR, exactly like * before +, so mixing them without **parentheses** is the classic filtering bug. WHERE runs before SELECT, which is why output aliases aren't visible here.

WHERE FACTS

- Rows where the condition is NULL are dropped
- Precedence: `NOT` → `AND` → `OR`
- The planner picks evaluation order, not you
- Test a filter with `SELECT` before `UPDATE` / `DELETE`

NOSQL COMPARISON

- MongoDB filter documents have the same shape:
- `{ $or: [{ category: 'Books' }, { category: 'Games' }], price: { $lt: 10 } }`
- Top-level fields are ANDed together

WHERE

```sql
SELECT name, price FROM products
WHERE  price > 20;

SELECT * FROM orders
WHERE  status = 'paid'
  AND  order_date >= DATE '2026-01-01';

-- AND binds first: not what it looks like
SELECT * FROM products
WHERE  category = 'Books' OR category = 'Games'
  AND  price < 10;   -- Books OR (Games AND < 10)

SELECT * FROM products
WHERE  (category = 'Books' OR category = 'Games')
  AND  price < 10;

SELECT * FROM customers
WHERE  NOT (country = 'US' OR country = 'CA');

-- Output alias isn't visible: repeat the expression
SELECT name, price * 0.9 AS sale FROM products
WHERE  price * 0.9 < 20;
```

CODE EXPLAINED

→ Without parentheses, cheap Books slip through at any price

→ `NOT (a OR b) = NOT a AND NOT b` (De Morgan)

→ `sale` doesn't exist yet when `WHERE` runs

→ Wrap the query in a CTE (sheet 41) to filter on an alias

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`=` on text is case-sensitive; `WHERE is_active` works
MySQL	default collation is case-insensitive: `'a' = 'A'`
SQL Server	case-insensitive by default; bit needs `= 1`
SQLite	`=` case-sensitive; `LIKE` isn't; aliases allowed in `WHERE`

IN AN ORM

```ts
await prisma.product.findMany({
  where: {
    OR: [{ category: 'Books' }, { category: 'Games' }],
    price: { lt: 10 },          // ANDed with the OR
  },
});
```

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
1	`( … )`
2	`* / %` then `+ - ||`
3	`= <> < > LIKE IN BETWEEN IS`
4	`NOT`
5	`AND`
6	`OR`

CONDITION KINDS

KIND	EXAMPLE	SHEET
comparison	`price >= 10`	18
range	`price BETWEEN 10 AND 20`	18
list	`status IN ('paid', 'shipped')`	18
pattern	`name LIKE 'Desk%'`	19
null test	`shipped_at IS NULL`	11
subquery	`EXISTS (SELECT …)`	36

⚠ COMMON MISTAKES

- Mixing `AND` and `OR` without parentheses
- Referencing a `SELECT` alias in `WHERE`
- Forgetting `WHERE` on an `UPDATE` or `DELETE`

✓ TIPS & BEST PRACTICES

- Always parenthesize `OR` groups, even when not required
- Put each condition on its own line, `AND` at the start
- Write date ranges half-open: `>= start AND < end`

QUICK SUMMARY

- `WHERE` keeps TRUE rows only
- `AND` before `OR`; use parentheses
- No output aliases in `WHERE`

Sorting with ORDER BY

ORDER BY · ASC · DESC · NULLS FIRST/LAST · multiple keys · ordinal position · COLLATE

PUTTING ROWS IN ORDER

Without **ORDER BY**, row order is whatever the engine finds fastest and can change between runs. ASC (the default) sorts low to high, DESC high to low. With **several keys**, later keys only break ties in earlier ones. **NULLS FIRST / NULLS LAST** decide where NULLs land. You can sort by aliases, expressions, a CASE for custom orders, or a column's position number (avoid that one).

SORTING FACTS

- `ORDER BY` runs after `SELECT`, so aliases work
- Each key has its own direction: `a DESC, b ASC`
- Text sorts by collation (sheet 08)
- Add a unique key last for repeatable order

NOSQL COMPARISON

- MongoDB: `.sort({ category: 1, price: -1 })`
- `null` and missing fields sort **before** numbers ascending
- No positional sorting; name the field

ORDER BY

```sql
SELECT name, price FROM products
ORDER  BY price DESC;                  -- high → low

SELECT name, category, price FROM products
ORDER  BY category, price DESC;        -- ties → price

SELECT name, city FROM customers
ORDER  BY city NULLS FIRST;            -- PG default: last

SELECT name, price * stock AS value FROM products
ORDER  BY value DESC;                  -- alias OK

SELECT name, price FROM products
ORDER  BY 2 DESC;                      -- 2nd column

-- Custom order with CASE
SELECT order_id, status FROM orders
ORDER  BY CASE status WHEN 'pending' THEN 1
                      WHEN 'paid'    THEN 2
                      ELSE 3 END,
          order_id;

SELECT name FROM customers
ORDER  BY lower(name), customer_id;    -- expression
```

CODE EXPLAINED

- → `category` with no direction is `ASC`
- → `NULLS FIRST` overrides PostgreSQL's NULLs-last default
- → `ORDER BY 2` breaks silently when the column list changes
- → `CASE` maps each status to a sort rank

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	NULLs last ASC; `NULLS FIRST / LAST` supported
MySQL	NULLs first ASC; emulate: `ORDER BY col IS NULL, col`
SQL Server	NULLs first ASC; no `NULLS LAST` → `CASE` key
SQLite	NULLs first ASC; `NULLS FIRST / LAST` since 3.30

IN AN ORM

```ts
await prisma.product.findMany({
  orderBy: [
    { category: 'asc' },
    { price: { sort: 'desc', nulls: 'last' } },
  ],
});
```

ORDER BY FORMS

FORM	EXAMPLE
column	`ORDER BY price`
direction	`ORDER BY price DESC`
several keys	`ORDER BY category, price DESC`
alias	`ORDER BY value`
expression	`ORDER BY lower(name)`
ordinal	`ORDER BY 2` (avoid)
collation	`ORDER BY name COLLATE "C"`

DEFAULT NULL POSITION

ENGINE	ASC	DESC
PostgreSQL	last	first
MySQL	first	last
SQL Server	first	last
SQLite	first	last
Oracle	last	first

⚠ COMMON MISTAKES

- Expecting insertion order without `ORDER BY`
- `ORDER BY` inside a view or subquery: the outer query can ignore it
- Sorting numbers stored as text: `'10'` before `'9'`

✓ TIPS & BEST PRACTICES

- End every paged sort with a unique column (sheet 16)
- An index in the same order can skip the sort step (sheet 71)
- Name columns instead of using ordinals

QUICK SUMMARY

- Only `ORDER BY` guarantees order
- `ASC` default · `DESC` reverse
- `NULLS FIRST / LAST` where supported

Limiting & Pagination

RETURNING A PAGE OF ROWS

`LIMIT n` returns the first n rows of the sorted result; **`OFFSET k`** skips k rows first. The standard spelling is **`OFFSET k ROWS FETCH NEXT n ROWS ONLY`**, and SQL Server also has **`TOP`**. OFFSET still reads every skipped row, so deep pages get slow and rows shift when data changes. **Keyset pagination** instead filters past the last row you saw, so page 1,000 costs the same as page 1.

PAGING FACTS

- Always pair a limit with `ORDER BY`
- Sort by a unique key, or pages overlap
- `WITH TIES` also returns rows tied with the last
- Fetch `n + 1` rows to know if a next page exists

NOSQL COMPARISON

- MongoDB: `.sort({ _id: 1 }).skip(20).limit(10)`
- `skip` has the same deep-page cost as `OFFSET`
- Keyset: `find({ _id: { $gt: lastId } }).limit(10)`

LIMIT & PAGINATE

```sql
SELECT name, price FROM products
ORDER  BY price DESC
LIMIT  10;                              -- top 10

SELECT name, price FROM products
ORDER  BY price DESC, product_id
LIMIT  10 OFFSET 20;                    -- page 3

-- Standard form (PG, SQL Server, Oracle)
SELECT name, price FROM products
ORDER  BY price DESC
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;

SELECT name, price FROM products
ORDER  BY price DESC
FETCH  FIRST 3 ROWS WITH TIES;          -- + ties

-- Keyset: continue after the last row seen
SELECT order_id, order_date FROM orders
WHERE  (order_date, order_id) < ('2026-09-30', 4211)
ORDER  BY order_date DESC, order_id DESC
LIMIT  20;
```

CODE EXPLAINED

- → `product_id` breaks price ties so pages don't overlap
- → Page `p` of size `n` → `OFFSET (p - 1) * n`
- → `WITH TIES` needs `ORDER BY`; may return more than 3
- → Row comparison `(a, b) < (x, y)` resumes after the last row

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`LIMIT / OFFSET`; `FETCH FIRST … WITH TIES`
MySQL	`LIMIT 10 OFFSET 20` or `LIMIT 20, 10`
SQL Server	`TOP (10) [WITH TIES]`; `OFFSET … FETCH` needs `ORDER BY`
SQLite	`LIMIT 10 OFFSET 20`; no `FETCH` or `TOP`

IN AN ORM

```ts
await prisma.order.findMany({
  orderBy: [{ orderDate: 'desc' }, { id: 'desc' }],
  take: 20,
  skip: 1,                  // skip the cursor row
  cursor: { id: lastId },   // keyset-style paging
});
```

OFFSET VS KEYSET

	OFFSET	KEYSET
jump to page N	yes	no — next / previous only
deep page speed	slows as offset grows	constant with an index
rows inserted meanwhile	duplicates or skips	stable
sort key	should be unique	must be unique
best for	admin tables, small sets	feeds, APIs, infinite scroll

PAGE MATH

NEED	EXPRESSION
offset for page p	`(p - 1) * size`
total pages	`ceil(count / size)`
has next page?	fetch `size + 1` rows
SQL Server top-N	`SELECT TOP (10) …`
keyset cursor	last row's sort key values

⚠ COMMON MISTAKES

- `LIMIT` without `ORDER BY`: a different 10 rows each time
- Sorting by a non-unique column, so rows repeat across pages
- Running `count(*)` over a huge table for every page

✓ TIPS & BEST PRACTICES

- Index the sort columns in sort order
- Use keyset for anything users scroll through
- Return the cursor (last key) to the client

QUICK SUMMARY

- `LIMIT` / `OFFSET` · `FETCH FIRST` · `TOP`
- `WITH TIES` keeps ties
- Keyset = `WHERE key > last`

DISTINCT & Duplicates

REMOVING REPEATS

`SELECT DISTINCT` removes duplicate **output rows**, comparing every selected column (two NULLs count as equal here). **`COUNT(DISTINCT x)`** counts unique non-NULL values. PostgreSQL's **`DISTINCT ON (key)`** keeps the first row per key according to `ORDER BY`, a quick way to get "latest order per customer". To **find** duplicate data, group by the columns that should be unique and keep groups with `count(*) > 1`.

DUPLICATE FACTS

- `DISTINCT` applies to the whole row, not one column
- Without aggregates, `DISTINCT` = `GROUP BY` all columns
- Deduplicating sorts or hashes the result: not free
- Prevent duplicates with `UNIQUE` (sheet 57)

NOSQL COMPARISON

- MongoDB: `db.customers.distinct('city')`
- Pipeline: `{ $group: { _id: '$city' } }`
- Unique index: `createIndex({ email: 1 }, { unique: true })`

DISTINCT & DUPLICATES

```sql
SELECT DISTINCT city FROM customers;
SELECT DISTINCT city, country FROM customers;
SELECT count(DISTINCT city) FROM customers;  -- no NULLs

-- Latest order per customer (PostgreSQL)
SELECT DISTINCT ON (customer_id)
       customer_id, order_id, order_date
FROM   orders
ORDER  BY customer_id, order_date DESC;

-- Find duplicate emails
SELECT lower(email) AS email, count(*) AS n
FROM   customers
GROUP  BY lower(email)
HAVING count(*) > 1;

-- Delete duplicates, keep the lowest id
DELETE FROM customers c
USING  customers d
WHERE  lower(c.email) = lower(d.email)
  AND  c.customer_id > d.customer_id;
```

CODE EXPLAINED

- → `DISTINCT city, country` — one row per unique pair
- → `DISTINCT ON` keeps the first row in `ORDER BY` order
- → `ORDER BY` must start with the `DISTINCT ON` columns
- → `USING` self-join deletes every copy but the oldest

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`DISTINCT ON (…)`; `DELETE … USING`
MySQL	no `DISTINCT ON` → `ROW_NUMBER()` (sheet 49)
SQL Server	no `DISTINCT ON`; delete through a CTE
SQLite	no `DISTINCT ON`; `MAX()` picks that row's columns

IN AN ORM

```ts
await prisma.customer.findMany({
  distinct: ['city'],
  select: { city: true },
  orderBy: { city: 'asc' },
});
```

TOOLS

NEED	USE
unique values	`SELECT DISTINCT col`
unique combinations	`SELECT DISTINCT a, b`
count unique	`count(DISTINCT col)`
one row per group	`DISTINCT ON` / `ROW_NUMBER()`
find duplicates	`GROUP BY … HAVING count(*) > 1`
stop duplicates	`UNIQUE` constraint or index

DISTINCT VS GROUP BY

QUERY	RESULT
`SELECT DISTINCT city`	unique cities
`SELECT city … GROUP BY city`	same rows
`GROUP BY city + count(*)`	unique cities with counts
`DISTINCT + aggregate`	usually a sign you want `GROUP BY`

⚠ COMMON MISTAKES

- Adding `DISTINCT` to hide duplicates from a bad join (sheet 35)
- `SELECT DISTINCT *` on wide rows: slow and rarely meaningful
- `DISTINCT ON` without `ORDER BY`: an arbitrary row wins

✓ TIPS & BEST PRACTICES

- Ask why duplicates exist before removing them
- Clean up duplicates, then add the `UNIQUE` constraint
- Normalize before comparing: `lower(trim(email))`

QUICK SUMMARY

- `DISTINCT` — unique output rows
- `count(DISTINCT x)` — unique values
- `DISTINCT ON` — first row per key (PG)

Comparison & Range Operators

= · <> · != · < > <= >= · BETWEEN · IN · NOT IN

COMPARING VALUES

=, **<>** (standard) or **!=**, and **< > <= >=** compare numbers, text (in collation order) and dates. **`BETWEEN a AND b`** is shorthand for `>= a AND <= b`, so **both ends are included**. **`IN (…)`** tests membership in a list or subquery, and `NOT IN` the reverse. Any comparison with NULL yields UNKNOWN, so those rows are filtered out (sheet 11).

COMPARISON FACTS

- `<>` and `!=` are identical
- `BETWEEN 20 AND 10` matches nothing: low bound first
- `IN ()` with an empty list is a syntax error
- `NOT IN (…, NULL)` never matches (sheet 36)

NOSQL COMPARISON

- MongoDB: `$eq` `$ne` `$gt` `$gte` `$lt` `$lte` `$in` `$nin`
- `{ price: { $gte: 10, $lte: 20 } }` = `BETWEEN 10 AND 20`
- `$ne` and `$nin` also match documents **missing** the field

COMPARISONS

```sql
SELECT * FROM products WHERE price = 19.99;
SELECT * FROM orders   WHERE status <> 'cancelled';
SELECT * FROM orders   WHERE status != 'cancelled';

SELECT * FROM products
WHERE  price BETWEEN 10 AND 20;       -- 10 ≤ price ≤ 20
SELECT * FROM products
WHERE  price NOT BETWEEN 10 AND 20;

-- DATE column: BETWEEN includes Sept 30
SELECT * FROM orders
WHERE  order_date BETWEEN '2026-09-01' AND '2026-09-30';

-- TIMESTAMP column: half-open range instead
SELECT * FROM customers
WHERE  created_at >= '2026-09-01'
  AND  created_at <  '2026-10-01';

SELECT * FROM orders
WHERE  status IN ('paid', 'shipped');
SELECT * FROM customers
WHERE  country NOT IN ('US', 'CA');
```

CODE EXPLAINED

→ `BETWEEN` is inclusive on both ends

→ On timestamps, `BETWEEN … '2026-09-30'` stops at midnight

→ `>= start AND < next_start` never misses or double-counts

→ `IN (a, b)` = `x = a OR x = b`

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`BETWEEN SYMMETRIC` accepts bounds in either order
MySQL	`<=>` NULL-safe equals; text compare ignores case
SQL Server	also `!<` and `!>` (nonstandard)
SQLite	`==` also means equals

IN AN ORM

```ts
await prisma.order.findMany({
  where: {
    total: { gte: 10, lte: 20 },          // BETWEEN
    status: { in: ['paid', 'shipped'] },  // IN
    customerId: { notIn: [7, 9] },        // NOT IN
  },
});
```

OPERATORS

OPERATOR	MEANS	EXAMPLE
`=`	equal	`status = 'paid'`
`<>` · `!=`	not equal	`status <> 'paid'`
`<` · `<=`	less than (or equal)	`price < 10`
`>` · `>=`	greater than (or equal)	`stock >= 1`
`BETWEEN a AND b`	a ≤ x ≤ b	`price BETWEEN 5 AND 9`
`IN (…)`	matches any listed value	`id IN (1, 2, 3)`
`NOT IN (…)`	matches none of them	`country NOT IN ('US')`

GOTCHAS

WRITE	RESULT
`BETWEEN 20 AND 10`	no rows
`ts BETWEEN d1 AND d2`	misses the last day after 00:00
`x NOT IN (1, NULL)`	no rows
`'10' < '9'`	true: text compares by character
`x = NULL`	UNKNOWN; use `IS NULL`

⚠ COMMON MISTAKES

- Reversed `BETWEEN` bounds returning nothing
- `BETWEEN` on timestamps losing the final day
- Comparing numbers stored as text

✓ TIPS & BEST PRACTICES

- Prefer half-open ranges for dates and times
- Use `IN` instead of a chain of `OR`s
- Use `NOT EXISTS` over `NOT IN` with subqueries

QUICK SUMMARY

- `=` `<>` `<` `>` `<=` `>=`
- `BETWEEN` — inclusive range
- `IN` / `NOT IN` — list membership

Pattern Matching

MATCHING TEXT PATTERNS

LIKE matches text against a pattern where **%** means "any run of characters" and _ means "exactly one character". Use **ESCAPE** to match a literal % or _. PostgreSQL adds **ILIKE** (case-insensitive) and full **regular expressions** with ~; MySQL has REGEXP. A pattern that **starts with %** can't use a normal B-tree index, so it scans the whole table.

PATTERN FACTS

- `LIKE` must match the **whole** string
- Case sensitivity follows the engine and collation
- `'Desk%'` can use an index; `'%desk'` can't
- For words in prose, use full-text search (sheet 80)

NOSQL COMPARISON

- MongoDB: `{ name: /lamp/i }` or `{ $regex: '^Desk', $options: 'i' }`
- Only an anchored, case-sensitive `^prefix` regex uses an index
- Atlas Search handles fuzzy and full-text matching

LIKE & REGEX

```sql
SELECT name FROM products WHERE name LIKE 'Desk%';
SELECT name FROM products WHERE name LIKE '%lamp%';
SELECT name FROM products WHERE name LIKE '_at';
SELECT email FROM customers
WHERE  email NOT LIKE '%@%.%';

-- Case-insensitive (PostgreSQL)
SELECT name FROM products WHERE name ILIKE '%LAMP%';

-- Match a literal %
SELECT name FROM products
WHERE  name LIKE '%50!%%' ESCAPE '!';   -- contains 50%

-- Regular expressions (PostgreSQL)
SELECT email FROM customers
WHERE  email ~* '^[^@\s]+@[^@\s]+\.[a-z]{2,}$';
SELECT name FROM products WHERE name ~ '^[A-Z]{3}-\d+';
SELECT regexp_replace(name, '[^A-Za-z ]', '', 'g')
FROM   customers;                       -- letters only
SELECT substring(email FROM '@(.*)$')   -- domain
FROM   customers;
```

CODE EXPLAINED

- → `'_at'` — exactly three characters ending in `at`
- → `ESCAPE '!'` makes `!%` a literal percent sign
- → `~` regex · `~*` case-insensitive · `!~` doesn't match
- → `'g'` flag replaces every match, not just the first

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`ILIKE`, `~` `~*`, `SIMILAR TO`, `regexp_*()`
MySQL	`LIKE` ignores case by collation; `REGEXP` / `REGEXP_LIKE()`
SQL Server	`LIKE '[a-c]%'` sets; `REGEXP_LIKE()` from 2025
SQLite	`LIKE` ignores ASCII case; `GLOB` uses `*` `?`

IN AN ORM

```ts
await prisma.product.findMany({
  where: {
    name: { contains: 'lamp', mode: 'insensitive' },
  },  // also startsWith, endsWith
});
```

LIKE PATTERNS

PATTERN	MATCHES
`'Desk%'`	starts with Desk
`'%lamp'`	ends with lamp
`'%lamp%'`	contains lamp
`'_at'`	cat, hat, bat (3 chars)
`'__-%'`	2 chars, a dash, anything
`'%!_%' ESCAPE '!'`	contains an underscore

REGEX QUICK REFERENCE

TOKEN	MEANS
`^` · `$`	start · end of string
`.` · `\d` · `\s`	any char · digit · whitespace
`[abc]` · `[^abc]`	one of · none of
`*` · `+` · `?`	0+, 1+, 0 or 1 of previous
`{2,4}`	between 2 and 4 times
`(a|b)`	a or b, captured

⚠ COMMON MISTAKES

- Forgetting `_` is a wildcard: `LIKE 'user_1'` matches `userX1`
- Leading `%` on a big table: full scan every time
- Assuming `LIKE` is case-insensitive in PostgreSQL

✓ TIPS & BEST PRACTICES

- PG: a `pg_trgm` GIN index speeds up `'%x%'` searches (sheet 72)
- Anchor regexes with `^` and `$` when validating
- Clean data once on write instead of regex on every read

QUICK SUMMARY

- `%` any run · `_` one char
- `ILIKE` / `~*` — case-insensitive (PG)
- Leading `%` → no index

Arithmetic & Concatenation Operators

+ - * / · % · integer division · || · CONCAT · operator precedence

MATH AND JOINING STRINGS

SQL has the usual **+ − ∗ /** plus **%** for the remainder. Dividing two integers gives an **integer** in PostgreSQL, SQL Server and SQLite (the fraction is dropped), but a decimal in MySQL. ∗, / and % bind before + and −. To join strings, the standard operator is **||**; MySQL uses **CONCAT()** and SQL Server **+**. Any NULL operand makes the result NULL, except in CONCAT, which treats NULL as an empty string.

OPERATOR FACTS

- Integer division truncates toward zero: `-7 / 2 = -3`
- Dividing by zero is an error (MySQL: NULL)
- `x / NULLIF(y, 0)` returns NULL instead (sheet 26)
- `||` turns numbers into text automatically (PG)

NOSQL COMPARISON

- MongoDB updates: `$inc` and `$mul` do arithmetic in place
- Pipelines: `$add`, `$multiply`, `$divide`, `$mod`
- `$concat` returns null if any input is null, like `||`

ARITHMETIC & CONCATENATION

```
SELECT order_id,
       unit_price * quantity AS line_total,
       round(unit_price * quantity * 1.08, 2) AS with_tax
FROM   order_items;

SELECT 7 / 2, 7 / 2.0, 7 % 3;       -- 3 | 3.5000… | 1
SELECT -7 / 2, -7 % 3;              -- -3 | -1
SELECT 2 + 3 * 4, (2 + 3) * 4;      -- 14 | 20
SELECT 7 / 0;       -- ERROR: division by zero
SELECT 7 / NULLIF(0, 0);            -- NULL
SELECT 2 ^ 10;                      -- 1024 (PG: power)

SELECT first_name || ' ' || last_name AS full_name
FROM   employees;
SELECT 'Order #' || order_id FROM orders;
SELECT 'a' || NULL;                 -- NULL
SELECT concat('a', NULL, 'b');      -- ab
SELECT concat_ws(', ', city, country) FROM customers;
```

CODE EXPLAINED

- → `round(…, 2)` keeps money at two decimals
- → `7 / 2.0` — one decimal operand makes it decimal division
- → `%` takes the sign of the left operand
- → `concat_ws` skips NULLs and puts the separator between

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`||` joins; int / int truncates; `^` is power
MySQL	`CONCAT()`; `||` means OR; / is decimal, `DIV` integer
SQL Server	`+` joins (`||` from 2025); `^` is XOR
SQLite	`||` joins; int / int truncates

IN AN ORM

```
await prisma.product.update({
  where: { id: 7 },
  data: {
    stock: { decrement: 1 },   // SET stock = stock - 1
    price: { multiply: 1.1 },  //     price = price * 1.1
  },
});
```

ARITHMETIC

OP	MEANS	EXAMPLE → RESULT
`+` · `-`	add · subtract	`10 - 3` → 7
`*`	multiply	`4 * 2.5` → 10.0
`/`	divide	`9 / 2` → 4 · `9 / 2.0` → 4.5
`%`	remainder	`9 % 4` → 1
`^`	power (PG); XOR elsewhere	`2 ^ 3` → 8
`- (unary)`	negate	`-price`

DIVISION BY ENGINE

EXPRESSION	PG	MYSQL	SQL SERVER
`7 / 2`	3	3.5000	3
`7 / 2.0`	3.5000…	3.5000	3.500000
`7 DIV 2`	—	3	—
`7 / 0`	error	NULL	error
`-7 % 3`	-1	-1	-1

⚠ COMMON MISTAKES

- `avg_qty = sum(qty) / count(*)` silently truncating
- MySQL `'a' || 'b'` returning 0 (it's logical OR)
- A NULL middle name blanking the whole `||` result

✓ TIPS & BEST PRACTICES

- Multiply by `1.0` or cast to get decimal division
- Use `concat_ws` for optional parts of names and addresses
- Round only at the end of a calculation

QUICK SUMMARY

- `+ - * / %` · `*` before `+`
- int / int → int (not MySQL)
- `||` · `CONCAT` · `+` (SQL Server)

CASE Expressions

simple CASE · searched CASE · ELSE · CASE in ORDER BY · CASE in aggregates · IIF

IF / ELSE INSIDE A QUERY

CASE is SQL's if/else. It's an **expression**, so it returns a value and can go anywhere a value can: SELECT, WHERE, ORDER BY, GROUP BY, inside an aggregate, or in UPDATE … SET. A **searched** CASE tests any condition; a **simple** CASE compares one value to a list. Branches are checked top to bottom and the **first match wins**; with no match and no ELSE, the result is NULL.

CASE FACTS

- All branches must return compatible types
- Missing `ELSE` → NULL
- Simple `CASE x WHEN NULL` never matches
- `IIF(c, a, b)` is shorthand (SQL Server, SQLite)

NOSQL COMPARISON

- MongoDB: `$cond: { if: …, then: …, else: … }`
- `$switch: { branches: [{ case, then }], default }` for many branches
- Both work only inside aggregation pipelines

CASE

```sql
-- Searched CASE: any conditions
SELECT name, price,
       CASE WHEN price >= 100 THEN 'premium'
            WHEN price >= 20  THEN 'standard'
            ELSE 'budget'
       END AS tier
FROM   products;

-- Simple CASE: compare one value
SELECT order_id,
       CASE status WHEN 'paid'    THEN 'Paid'
                   WHEN 'shipped' THEN 'On its way'
                   ELSE 'Other' END AS label
FROM   orders;

-- Inside aggregates: several counts in one pass
SELECT count(CASE WHEN status = 'paid' THEN 1 END) AS paid,
       sum(CASE WHEN status = 'cancelled'
                THEN total ELSE 0 END)   AS lost
FROM   orders;
-- In ORDER BY and UPDATE
SELECT name FROM products
ORDER  BY CASE WHEN stock = 0 THEN 1 ELSE 0 END, name;
UPDATE products
SET    price = CASE WHEN category = 'Books'
                    THEN price * 0.9 ELSE price END;
```

CODE EXPLAINED

→ Order matters: a 150 price stops at `premium`

→ `count(CASE … THEN 1 END)` counts only matches (no ELSE)

→ `ORDER BY CASE` pushes out-of-stock items to the end

→ `ELSE price` keeps other rows unchanged

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`CASE` only; `FILTER (WHERE …)` for aggregates
MySQL	`CASE` and `IF(cond, a, b)`
SQL Server	`CASE`, `IIF(c, a, b)`, `CHOOSE(i, a, b, …)`
SQLite	`CASE` and `iif(c, a, b)` (3.32+)

IN AN ORM

```sql
// Prisma has no CASE builder: use raw SQL
const rows = await prisma.$queryRaw`
  SELECT name,
         CASE WHEN price >= 100 THEN 'premium'
              ELSE 'standard' END AS tier
  FROM   products`;
```

TWO FORMS

FORM	SYNTAX
searched	`CASE WHEN cond THEN a … ELSE z END`
simple	`CASE x WHEN v1 THEN a … ELSE z END`
shorthand	`IIF(cond, a, b)` · `IF(cond, a, b)`
NULL default	`COALESCE(x, 'n/a')` (sheet 26)

WHERE CASE GOES

CLAUSE	EXAMPLE USE
SELECT	derive a label or bucket
WHERE	conditional filter (keep it simple)
ORDER BY	custom sort order
GROUP BY	group by a computed bucket
SUM / COUNT	conditional totals (sheet 29)
UPDATE SET	different change per row

⚠ COMMON MISTAKES

- No `ELSE` and an unexpected NULL
- Mixed branch types: `THEN 'none'` vs `THEN 0`
- Putting a broad `WHEN` first so later ones never fire

✓ TIPS & BEST PRACTICES

- Most specific `WHEN` first
- Long code → label mappings belong in a lookup table
- PG: `count(*) FILTER (WHERE …)` reads better than `CASE`

QUICK SUMMARY

- `CASE WHEN … THEN … ELSE … END`
- First match wins; no match → `ELSE` or NULL
- Works in any clause

String Functions — Transform

RESHAPING TEXT

These functions return a **changed copy** of a string; the stored value only changes if you UPDATE it. **`length`** counts characters, **`upper` / `lower` / `initcap`** change case, **`trim`** strips spaces (or any characters you name) from one or both ends, **`lpad` / `rpad`** pad to a fixed width, and **`repeat`** repeats. The most common use is **cleaning input**: `lower(trim(email))` before comparing or storing.

TRANSFORM FACTS

- Functions don't modify data unless used in `UPDATE`
- `lpad` **cuts** strings longer than the width
- MySQL `LENGTH()` counts **bytes**; use `CHAR_LENGTH()`
- NULL in → NULL out

NOSQL COMPARISON

- MongoDB: `$toUpper`, `$toLower`, `$trim`, `$ltrim`, `$rtrim`
- `$strLenCP` counts characters, `$strLenBytes` bytes
- Used inside `$project` / `$set` pipeline stages

TRANSFORM FUNCTIONS

```
SELECT name,
       length(name)   AS chars,
       upper(name)    AS shout,
       lower(email)   AS email_key,
       initcap(name)  AS proper       -- Ada Lovelace
FROM   customers;

SELECT trim('  hi  '),                 -- 'hi'
       ltrim('  hi'), rtrim('hi  '),
       trim(BOTH '-' FROM '--x--'),    -- 'x'
       trim(LEADING '0' FROM '00420'); -- '420'

SELECT lpad(order_id::text, 6, '0') AS ref  -- 000042
FROM   orders;
SELECT rpad('ab', 5, '.'),             -- 'ab...'
       lpad('abcdef', 3),              -- 'abc' (cut!)
       repeat('=', 10);

-- Clean data in place
UPDATE customers
SET    email = lower(trim(email)),
       name  = initcap(trim(name));
```

CODE EXPLAINED

- → `initcap` capitalizes each word and lowercases the rest
- → `trim(BOTH | LEADING | TRAILING 'x' FROM s)` — standard form
- → `lpad(…, 6, '0')` — zero-padded reference numbers
- → The `UPDATE` normalizes every row once

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`length`, `initcap`, `lpad`, `repeat`
MySQL	`CHAR_LENGTH()` chars, `LENGTH()` bytes; no `INITCAP`
SQL Server	`LEN()` ignores trailing spaces; `REPLICATE()`; no `LPAD`
SQLite	`upper` / `lower` ASCII only; `trim(s, chars)`; no `lpad`

IN AN ORM

```
// Normalize on write instead of on every read
await prisma.customer.create({
  data: { name, email: email.trim().toLowerCase() },
});
```

FUNCTIONS

CALL	RESULT
`length('héllo')`	`5`
`upper('abc') · lower('ABC')`	`ABC · abc`
`initcap('hello WORLD')`	`Hello World`
`trim('  x  ')`	`x`
`lpad('7', 3, '0')`	`007`
`rpad('ab', 4, '*')`	`ab**`
`repeat('ab', 3)`	`ababab`

LENGTH BY ENGINE

ENGINE	CHARACTERS	BYTES
PostgreSQL	`length · char_length`	`octet_length`
MySQL	`CHAR_LENGTH`	`LENGTH`
SQL Server	`LEN` (no trailing spaces)	`DATALENGTH`
SQLite	`length`	`octet_length` (3.43+)

⚠ COMMON MISTAKES

- Using MySQL `LENGTH()` to validate character limits
- `lpad` silently cutting values longer than the width
- Calling `lower()` in every query instead of storing clean data

✓ TIPS & BEST PRACTICES

- Normalize once on write: trim, case, collapse spaces
- Index `lower(email)` if you must compare case-insensitively
- `initcap` mangles names like `McDonald`; don't force it

QUICK SUMMARY

- `length` · `upper` · `lower` · `initcap`
- `trim` · `ltrim` · `rtrim`
- `lpad` · `rpad` · `repeat`

String Functions — Search & Extract

FINDING AND CUTTING TEXT

position(sub IN s) (or strpos) returns where a substring starts, counting from **1**, and **0** when it's absent. **substring**, **left** and **right** cut out pieces by position. **split_part** splits on a delimiter and returns the nth field, which is the quickest way to pull the domain out of an email. **replace** swaps every occurrence of one substring for another, and **reverse** flips the string.

SEARCH FACTS

- Positions start at 1, not 0
- Not found → `0`, not NULL
- `replace` is case-sensitive
- `translate` swaps or deletes single characters

NOSQL COMPARISON

- MongoDB: `$indexOfCP`, `$substrCP`, `$split`, `$replaceAll`
- `$split` returns an array; take a field with `$arrayElemAt`
- `$regexFind` extracts with a regex

SEARCH & EXTRACT

```sql
SELECT email,
       left(email, 1)              AS first_char,
       right(email, 4)             AS last_four,
       position('@' IN email)      AS at_pos,
       split_part(email, '@', 1)   AS local_part,
       split_part(email, '@', 2)   AS domain
FROM   customers;

SELECT substring('2026-10-18' FROM 1 FOR 4); -- 2026
SELECT substring('2026-10-18', 6, 2);        -- 10
SELECT strpos('banana', 'an');               -- 2
SELECT position('z' IN 'banana');            -- 0
SELECT replace('2026-10-18', '-', '/');      -- 2026/10/18
SELECT split_part('a,b,c', ',', -1);         -- c (PG 14+)
SELECT reverse('stressed');                  -- desserts
SELECT translate('(555) 123-4567', '() -', '');
-- 5551234567

SELECT name FROM products
WHERE  position('lamp' IN lower(name)) > 0;
```

CODE EXPLAINED

- → `split_part(email, '@', 2)` — everything after the `@`
- → `substring(s FROM i FOR n)` — standard; `(s, i, n)` also works
- → Negative `split_part` index counts from the end
- → `translate` with an empty target deletes those characters

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	strpos, split_part, translate, reverse
MySQL	LOCATE(sub, s), SUBSTRING_INDEX(s, '@', -1)
SQL Server	CHARINDEX(sub, s), SUBSTRING(s, i, n), STRING_SPLIT
SQLite	instr(s, sub), substr(s, i, n); no split_part

IN AN ORM

```tsx
await prisma.customer.findMany({
  where: { email: { endsWith: '@example.com' } },
});
// contains / startsWith / endsWith → LIKE
```

FUNCTIONS

CALL	RESULT
left('Postgres', 4)	Post
right('Postgres', 3)	res
substring('abcdef', 2, 3)	bcd
position('c' IN 'abc')	3
replace('a-b-c', '-', '')	abc
split_part('x.y.z', '.', 2)	y
reverse('abc')	cba

SAME TASK, FOUR ENGINES

TASK	PG	MYSQL	SQL SERVER	SQLITE
find	strpos	LOCATE	CHARINDEX	instr
slice	substring	SUBSTRING	SUBSTRING	substr
nth field	split_part	SUBSTRING_INDEX	STRING_SPLIT	—
first n	left	LEFT	LEFT	substr(s,1,n)

⚠ COMMON MISTAKES

- Treating a 0 result from `position` as "found at the start"
- Parsing structured data (JSON, CSV) with string functions
- Off-by-one slicing from 0-based habits

✓ TIPS & BEST PRACTICES

- Store parts separately if you query them often (`email_domain`)
- Use JSON functions for JSON (sheet 78)
- For complex patterns reach for regex (sheet 19)

QUICK SUMMARY

- `position` / `strpos` → index or 0
- `substring` · `left` · `right`
- `split_part` · `replace` · `reverse`

Numeric Functions

ROUNDING & MATH

`round(x, d)` rounds to d decimal places (a negative d rounds to tens, hundreds…). **`ceil`** goes up, **`floor`** goes down, and **`trunc`** just cuts digits off, which differs from `floor` for negative numbers. **`abs`**, **`mod`**, **`power`** and **`sqrt`** do what their names say. **`random()`** returns a value from 0 up to (not including) 1, handy for sampling and test data.

MATH FACTS

- `numeric` rounds half away from zero: `2.5` → 3
- `float8` rounds half to even: `2.5` → 2
- `floor(-4.1)` = -5, `trunc(-4.1)` = -4
- PG `log(x)` is base 10, `ln(x)` natural

NOSQL COMPARISON

- MongoDB pipelines: `$round`, `$ceil`, `$floor`, `$trunc`, `$abs`, `$mod`, `$pow`
- `$rand` gives 0 ≤ x < 1
- `$sample: { size: 3 }` picks random documents

NUMERIC FUNCTIONS

```
SELECT round(2.5), round(-2.5), round(3.14159, 2);
-- 3 | -3 | 3.14
SELECT round(1234.5, -2);                    -- 1200
SELECT ceil(4.1), floor(4.9), trunc(4.99); -- 5 | 4 | 4
SELECT ceil(-4.1), floor(-4.1), trunc(-4.9);
-- -4 | -5 | -4
SELECT abs(-7), mod(17, 5), 17 % 5;          -- 7 | 2 | 2
SELECT power(2, 10), sqrt(16), cbrt(27);     -- 1024 | 4 | 3
SELECT sign(-3), ln(10), log(100);           -- -1 | 2.30… | 2

-- Order totals to two places
SELECT order_id, round(sum(unit_price * quantity), 2)
FROM   order_items
GROUP  BY order_id;

SELECT ceil(95 / 20.0);      -- pages for 95 rows → 5

SELECT random();                           -- 0 ≤ x < 1
SELECT floor(random() * 6 + 1)::int;       -- dice 1-6
SELECT * FROM products ORDER BY random() LIMIT 3;
```

CODE EXPLAINED

- → `round(x, -2)` rounds to the nearest hundred
- → `ceil(95 / 20.0)` — the `.0` avoids integer division
- → `floor(random() * n + 1)` → whole number from 1 to n
- → `ORDER BY random()` sorts the whole table: fine for small ones

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`random()`, `trunc`, `log` base 10, `setseed()`
MySQL	`RAND()`, `TRUNCATE(x, d)`, `LOG(x)` is natural
SQL Server	`CEILING`; `RAND()` once per query → `NEWID()` per row
SQLite	`random()` is a 64-bit integer; math functions 3.35+

IN AN ORM

```
const p = await prisma.product.findUniqueOrThrow({
  where: { id: 7 },
});
p.price.mul(1.08).toDecimalPlaces(2); // exact Decimal math
```

ROUNDING COMPARED

X	ROUND	CEIL	FLOOR	TRUNC
2.4	2	3	2	2
2.5	3	3	2	2
-2.4	-2	-2	-3	-2
-2.5	-3	-2	-3	-2
0.5	1	1	0	0
7.85 (d = 1)	7.9	—	—	7.8

MORE FUNCTIONS

CALL	RESULT
`abs(-3.2)`	`3.2`
`mod(-7, 3)`	`-1` (sign of the left side)
`power(1.05, 10)`	`1.628…` compound growth
`sqrt(2)` · `cbrt(8)`	`1.414…` · `2`
`sign(x)`	`-1`, `0` or `1`
`exp(1)` · `ln(x)` · `log(x)`	`2.718…` · natural · base 10
`width_bucket(x, 0, 100, 4)`	bucket 1–4 for histograms

⚠ COMMON MISTAKES

- Rounding each line, then summing: totals drift by cents
- Expecting `trunc` and `floor` to agree on negatives
- SQL Server `ORDER BY RAND()` : same value for every row

✓ TIPS & BEST PRACTICES

- Round once, at the end, on `numeric` values
- Sample huge tables with `TABLESAMPLE SYSTEM (1)` (PG)
- Use `setseed()` for repeatable random test data

QUICK SUMMARY

- `round` · `ceil` · `floor` · `trunc`
- `abs` · `mod` · `power` · `sqrt`
- `random()` — 0 ≤ x < 1

Date & Time Functions

NOW() · CURRENT_DATE · EXTRACT · DATE_TRUNC · INTERVAL · DATEADD / DATEDIFF · TO_CHAR · AT TIME ZONE

WORKING WITH DATES

now() and **current_date** give the current moment and day (fixed for the whole transaction). **extract(part FROM d)** pulls out a year, month, weekday or hour. **date_trunc('month', d)** rounds down to the start of a unit, which is how you group by month. Add or subtract **INTERVALs** for date math, format with **to_char**, and convert zones with **AT TIME ZONE**. This is the area where dialects differ most.

DATE FACTS

- `now()` is the transaction start; `clock_timestamp()` is live
- `date - date` → whole days (PG)
- `DOW` : 0 = Sunday; `ISODOW` : 1 = Monday
- Group by `date_trunc`, not by `to_char` text

NOSQL COMPARISON

- MongoDB: `$dateTrunc`, `$dateAdd`, `$dateDiff`, `$dateToString`
- Each takes a `timezone` option, e.g. `'America/Chicago'`
- `$year`, `$month`, `$dayOfWeek` extract parts

DATE & TIME FUNCTIONS

```sql
SELECT now(), current_date, current_timestamp;

SELECT extract(YEAR FROM order_date) AS yr,
       extract(DOW  FROM order_date) AS dow
FROM   orders;

-- Revenue per month
SELECT date_trunc('month', order_date) AS month,
       sum(total)                      AS revenue
FROM   orders
GROUP  BY month
ORDER  BY month;

SELECT order_date + INTERVAL '30 days' AS due FROM orders;
SELECT DATE '2026-12-25' - CURRENT_DATE AS days_left;
SELECT to_char(DATE '2026-10-18', 'Dy DD Mon YYYY');
-- Sun 18 Oct 2026
SELECT now() AT TIME ZONE 'Asia/Tokyo';

-- The last 7 full days
SELECT * FROM orders
WHERE  order_date >= CURRENT_DATE - 7
  AND  order_date <  CURRENT_DATE;
```

CODE EXPLAINED

- → `date_trunc('month', …)` → the 1st of the month at 00:00
- → `GROUP BY month` — PG allows the output alias here
- → `date - date` returns an integer number of days
- → `AT TIME ZONE` on `TIMESTAMPTZ` → local wall-clock time

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`date_trunc`, `+ INTERVAL`, `age`, `to_char`
MySQL	`DATE_ADD(d, INTERVAL 30 DAY)`, `DATEDIFF`, `DATE_FORMAT`
SQL Server	`DATEADD(day, 30, d)`, `DATEDIFF`, `DATETRUNC` (2022)
SQLite	`date(d, '+30 days')`, `strftime('%Y-%m', d)`

IN AN ORM

```ts
const since = new Date(Date.now() - 7 * 86_400_000);
await prisma.order.findMany({
  where: { orderDate: { gte: since } },
});
// grouping by date_trunc → $queryRaw
```

EXTRACT FIELDS

FIELD	RETURNS
`YEAR MONTH DAY`	calendar parts
`QUARTER · WEEK`	1–4 · ISO 1–53
`DOW · ISODOW`	0 = Sun · 1 = Mon
`DOY`	day of year
`HOUR MINUTE`	time parts
`EPOCH`	Unix seconds

SAME TASK, FOUR ENGINES

TASK	PG	MYSQL	SQL SERVER	SQLITE
now	`now()`	`NOW()`	`SYSDATETIME()`	`datetime('now')`
+30 days	`d + 30`	`d + INTERVAL 30 DAY`	`DATEADD(day,30,d)`	`date(d,'+30 days')`
days a→b	`b - a`	`DATEDIFF(b,a)`	`DATEDIFF(day,a,b)`	`julianday` diff
YYYY-MM	`to_char(d,'YYYY-MM')`	`DATE_FORMAT(d,'%Y-%m')`	`FORMAT(d,'yyyy-MM')`	`strftime('%Y-%m',d)`
month start	`date_trunc('month',d)`	`DATE_FORMAT(d,'%Y-%m-01')`	`DATETRUNC(month,d)`	`date(d,'start of month')`

⚠ COMMON MISTAKES

- Grouping on `to_char(d, 'Mon')` : every year's October merges
- `WHERE date(created_at) = …` blocking the index
- Mixing up `DATEDIFF` argument order between engines

✓ TIPS & BEST PRACTICES

- Filter with half-open ranges on the raw column
- Keep zone conversion at the edges (display, reports)
- Generate missing days with `generate_series` (sheet 79)

QUICK SUMMARY

- now() · current_date
- extract · date_trunc
- + INTERVAL · age · to_char

Conditional & NULL Functions

COALESCE · NULLIF · GREATEST · LEAST · IFNULL · ISNULL · NVL

DEFAULTS AND GUARDS

`coalesce(a, b, …)` returns its first non-NULL argument, so it's the standard way to supply a default. **`nullif(a, b)`** does the reverse: it returns NULL when `a = b`, which turns zeros into NULLs to prevent division errors, or empty strings into real NULLs. **`greatest`** and **`least`** pick the largest or smallest of their arguments. `IFNULL`, `ISNULL` and `NVL` are two-argument, engine-specific versions of `coalesce`.

FUNCTION FACTS

- `coalesce` stops at the first non-NULL argument
- `x / nullif(y, 0)` → NULL instead of an error
- `sum()` of no rows is NULL; wrap it in `coalesce`
- PG `greatest` ignores NULLs; MySQL's returns NULL

NOSQL COMPARISON

- MongoDB: `$ifNull: ['$city', 'Unknown']` (several args from 5.0)
- `$max` / `$min` over an array act as greatest / least
- `$cond` with `$eq` mimics `nullif`

COALESCE, NULLIF & FRIENDS

```sql
SELECT name, coalesce(category, 'Uncategorized') AS cat
FROM   products;
SELECT coalesce(city, country, 'n/a') FROM customers;

-- Avoid divide-by-zero
SELECT order_id,
       sum(unit_price * quantity)
         / nullif(sum(quantity), 0) AS avg_unit_price
FROM   order_items
GROUP  BY order_id;

-- Treat '' as missing, then default
SELECT coalesce(nullif(trim(city), ''), 'Unknown')
FROM   customers;

-- Aggregate over zero rows
SELECT coalesce(sum(total), 0) AS refunded
FROM   orders
WHERE  status = 'refunded';

SELECT greatest(10, 20, 5), least(10, 20, 5);  -- 20 | 5
SELECT name, greatest(price * 0.9, 5.00) AS sale
FROM   products;                    -- never below 5.00
SELECT greatest(1, NULL, 3);        -- 3 in PG
```

CODE EXPLAINED

- → `coalesce(city, country, …)` tries each in turn
- → `nullif(…, 0)` makes the division return NULL, not fail
- → `nullif(trim(city), '')` turns blanks into NULL first
- → `greatest(…, 5.00)` puts a floor under a price

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`coalesce`, `nullif`, `greatest` / `least` skip NULLs
MySQL	`IFNULL(a, b)`; `GREATEST` returns NULL if any arg is
SQL Server	`ISNULL(a, b)` uses `a`'s type; `GREATEST` 2022+
SQLite	`ifnull(a, b)`; `max(a, b)` / `min(a, b)` multi-arg

IN AN ORM

```javascript
const { _sum } = await prisma.order.aggregate({
  _sum: { total: true },
  where: { status: 'refunded' },
});
const refunded = _sum.total ?? 0;   // COALESCE in JS
```

FUNCTIONS

CALL	RESULT
`coalesce(NULL, NULL, 'x')`	`x`
`nullif(5, 5)`	`NULL`
`nullif(5, 3)`	`5`
`greatest(3, 9, 4)`	`9`
`least('b', 'a')`	`a`
`coalesce(nullif(s, ''), 'n/a')`	default for blank or NULL

TWO-ARGUMENT VERSIONS

ENGINE	FUNCTION	WATCH OUT
MySQL / SQLite	`IFNULL(a, b)`	—
SQL Server	`ISNULL(a, b)`	result typed like `a`: may truncate
Oracle	`NVL(a, b)`	evaluates both args
everywhere	`COALESCE(a, b)`	standard; prefer it

⚠ COMMON MISTAKES

- SQL Server `ISNULL(code, 'unknown')` cut to `code`'s length
- Showing a NULL `sum` as blank instead of 0
- Using `coalesce` to hide bad joins instead of fixing them

✓ TIPS & BEST PRACTICES

- Prefer `coalesce` everywhere: it's standard and multi-arg
- Turn sentinel values (`''`, `0`, `-1`) into NULL with `nullif`
- Default at display time; keep NULL in storage when it's true

QUICK SUMMARY

- `coalesce` — first non-NULL
- `nullif` — NULL when equal
- `greatest` / `least` — max / min of args

Aggregate Functions

COUNT(*) · COUNT(col) · SUM · AVG · MIN · MAX · NULL handling

MANY ROWS → ONE VALUE

An **aggregate** reduces a set of rows to one value. With no `GROUP BY`, the whole (filtered) table is one group and you get one row back. **`count(*)`** counts rows; **`count(col)`** counts non-NULL values. **`sum`**, **`avg`**, **`min`** and **`max` ignore NULLs**. Over zero rows, `count` returns 0 and the others return NULL. `min` and `max` work on text and dates as well as numbers.

AGGREGATE FACTS

- `avg` divides by the non-NULL count only
- Plain columns + aggregates need `GROUP BY` (sheet 28)
- Aggregates can't go in `WHERE`; use `HAVING` (sheet 29)
- `count(DISTINCT col)` counts unique values

NOSQL COMPARISON

- MongoDB: `{ $group: { _id: null, n: { $sum: 1 }, avg: { $avg: '$total' } } }`
- `countDocuments(filter)` for a plain count
- `$sum` / `$avg` also skip missing and null values

AGGREGATES

```sql
SELECT count(*)         AS orders,
       count(total)     AS with_total,   -- skips NULL
       sum(total)       AS revenue,
       avg(total)       AS avg_order,
       min(order_date)  AS first_order,
       max(order_date)  AS last_order
FROM   orders
WHERE  status <> 'cancelled';

SELECT count(DISTINCT customer_id) AS buyers
FROM   orders;

-- NULLs and empty input
SELECT avg(x), sum(x), count(x), count(*)
FROM   (VALUES (10), (NULL), (20)) AS t(x);
-- 15 | 30 | 2 | 3
SELECT sum(total), count(*) FROM orders WHERE false;
-- NULL | 0

SELECT avg(coalesce(total, 0)) FROM orders; -- NULL = 0
SELECT min(name), max(name) FROM products;  -- text
SELECT customer_id, count(*) FROM orders;
-- ERROR: must appear in GROUP BY
```

CODE EXPLAINED

- → `avg` of 10, NULL, 20 is 15, not 10
- → `sum` over no rows is NULL; wrap in `coalesce(…, 0)`
- → `coalesce` inside `avg` counts NULLs as zero
- → Adding a plain column requires `GROUP BY`

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`avg(int)` → numeric; `bool_and`, `bool_or`
MySQL	`AVG(int)` → decimal; `BIT_AND`, `BIT_OR`
SQL Server	`AVG(int)` → **int**; `COUNT_BIG` for huge tables
SQLite	`total()` returns 0.0 instead of NULL

IN AN ORM

```ts
const s = await prisma.order.aggregate({
  where: { status: { not: 'cancelled' } },
  _count: { _all: true },
  _sum: { total: true },
  _avg: { total: true },
});
```

AGGREGATES & NULLS

FUNCTION	NULLS	NO ROWS →
`count(*)`	counted	0
`count(col)`	skipped	0
`sum(col)`	skipped	NULL
`avg(col)`	skipped	NULL
`min · max`	skipped	NULL
`string_agg`	skipped	NULL

STATISTICS (POSTGRESQL)

CALL	GIVES
`stddev(x) · variance(x)`	spread (sample)
`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	median
`percentile_disc(0.9) WITHIN GROUP (ORDER BY x)`	90th percentile value
`mode() WITHIN GROUP (ORDER BY x)`	most frequent value
`corr(x, y)`	correlation -1…1
`bool_and(x) · bool_or(x)`	all / any true

⚠ COMMON MISTAKES

- Using `count(col)` when you meant rows, or the reverse
- Letting `avg` ignore NULLs that really mean 0
- SQL Server `AVG(qty)` on an int column returning a whole number

✓ TIPS & BEST PRACTICES

- Filter with `WHERE` before aggregating; it's cheaper
- Use `FILTER` for several conditional counts (sheet 29)
- Cast before averaging ints: `avg(qty * 1.0)`

QUICK SUMMARY

- `count` · `sum` · `avg` · `min` · `max`
- NULLs skipped (except `count(*)`)
- No rows → `count` 0, others NULL

GROUP BY

GROUP BY · multiple columns · grouping expressions · functional dependency · ONLY_FULL_GROUP_BY

ONE ROW PER GROUP

GROUP BY sorts rows into **groups**, one per distinct value (or combination of values) of the listed columns or expressions, and each aggregate is computed **per group**. Every column in SELECT must either appear in GROUP BY or sit inside an aggregate. The exception is **functional dependency**: group by a table's primary key and PostgreSQL and MySQL let you select its other columns too. All NULLs fall into one group.

GROUPING RULES

- Selectable: grouped columns, aggregates, constants
- Not selectable: other plain columns (with exceptions)
- Group order isn't output order; add `ORDER BY`
- `GROUP BY` runs after `WHERE`, before `HAVING`

NOSQL COMPARISON

- MongoDB: `{ $group: { _id: '$status', n: { $sum: 1 } } }`
- Several keys: `_id: { country: '$country', city: '$city' }`
- Group key comes back in `_id`; rename it with `$project`

GROUP BY

```sql
SELECT status, count(*) AS n, sum(total) AS revenue
FROM   orders
GROUP  BY status
ORDER  BY revenue DESC;

-- Several columns: one group per combination
SELECT country, city, count(*) AS customers
FROM   customers
GROUP  BY country, city;

-- Group by an expression
SELECT date_trunc('month', order_date) AS month,
       count(*)                        AS orders
FROM   orders
GROUP  BY date_trunc('month', order_date);

-- Join, then group (name depends on the PK)
SELECT c.customer_id, c.name, count(o.order_id) AS n
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id;

SELECT category, name, count(*)
FROM   products GROUP BY category;
-- ERROR: column "products.name" must appear in GROUP BY
```

CODE EXPLAINED

→ One output row per `status`, sorted by revenue

→ `country, city` → a group per unique pair

→ `count(o.order_id)` counts 0 for customers with no orders

→ `name` is legal because `customer_id` is the primary key

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	columns dependent on a grouped PK are allowed
MySQL	`ONLY_FULL_GROUP_BY` on by default; `ANY_VALUE(col)`
SQL Server	list every non-aggregated column
SQLite	allows ungrouped columns (value from any row)

IN AN ORM

```ts
await prisma.order.groupBy({
  by: ['status'],
  _count: { _all: true },
  _sum: { total: true },
  orderBy: { _sum: { total: 'desc' } },
});
```

INPUT: ORDERS

ORDER_ID	STATUS	TOTAL
1	paid	20.00
2	paid	30.00
3	shipped	15.00
4	NULL	5.00
5	shipped	NULL

OUTPUT: GROUP BY STATUS

STATUS	COUNT(*)	SUM(TOTAL)
paid	2	50.00
shipped	2	15.00
NULL	1	5.00

⚠ COMMON MISTAKES

- Selecting a column that isn't grouped or aggregated
- Turning off `ONLY_FULL_GROUP_BY` and getting random values
- Grouping by a display string (`to_char`) instead of a date

✓ TIPS & BEST PRACTICES

- Group by the key, select the details (functional dependency)
- Aggregate in a CTE, then join for names (sheet 41)
- Index the grouping columns for big tables

QUICK SUMMARY

- `GROUP BY` → one row per group
- SELECT = grouped columns + aggregates
- NULLs share one group

HAVING & Conditional Aggregation

FILTERING GROUPS

HAVING filters **groups** after aggregation, so it can test aggregates like `count(*) >= 3`. **WHERE** filters **rows** before grouping and can't. **Conditional aggregation** computes several filtered totals in one pass: the standard **`FILTER (WHERE …)`** clause attaches a condition to one aggregate, and **`SUM(CASE …)`** does the same on engines without `FILTER`. It's how you build pivot-style reports.

FILTER FACTS

- Use `WHERE` for anything that doesn't need an aggregate
- `HAVING` without `GROUP BY` treats all rows as one group
- `FILTER` reads better than `CASE`
- `count(CASE … THEN 1 END)` — no `ELSE`

NOSQL COMPARISON

- MongoDB: HAVING is a `$match` stage **after** `$group`
- `[{ $group: { _id: '$customerId', n: { $sum: 1 } } }, { $match: { n: { $gte: 3 } } }]`
- Conditional sums: `$sum: { $cond: [cond, 1, 0] }`

HAVING & FILTER

```sql
-- Customers with 3+ orders
SELECT customer_id, count(*) AS n
FROM   orders
GROUP  BY customer_id
HAVING count(*) >= 3;

-- WHERE first, then HAVING
SELECT category, avg(price) AS avg_price
FROM   products
WHERE  stock > 0              -- rows
GROUP  BY category
HAVING avg(price) > 50;       -- groups

-- One pass, several filtered aggregates
SELECT customer_id,
       count(*)                                  AS orders,
       count(*) FILTER (WHERE status = 'paid') AS paid,
       sum(total) FILTER (
         WHERE status = 'cancelled')             AS lost
FROM   orders
GROUP  BY customer_id;

-- Portable CASE form
SELECT customer_id,
       sum(CASE WHEN status = 'paid' THEN 1 ELSE 0 END)
         AS paid
FROM   orders
GROUP  BY customer_id;
```

CODE EXPLAINED

→ `HAVING count(*) >= 3` drops small groups

→ `stock > 0` belongs in `WHERE` : it's per row

→ `FILTER` counts only the rows matching its condition

→ `CASE` returns 1 or 0 per row; `sum` adds them up

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`FILTER (WHERE …)` on any aggregate
MySQL	no `FILTER` → `SUM(status = 'paid')` (true = 1)
SQL Server	no `FILTER` → `SUM(CASE … END)`
SQLite	`FILTER` supported since 3.30

IN AN ORM

```ts
await prisma.order.groupBy({
  by: ['customerId'],
  _count: { _all: true },
  having: { customerId: { _count: { gte: 3 } } },
});
```

WHERE VS HAVING

	WHERE	HAVING
filters	rows	groups
runs	before `GROUP BY`	after `GROUP BY`
aggregates allowed	no	yes
uses indexes	yes	rarely
example	`status = 'paid'`	`sum(total) > 500`

PIVOT REPORT

CUSTOMER	ORDERS	PAID	CANCELLED	LOST
7	5	3	1	42.00
9	2	2	0	NULL
12	4	1	2	88.50
→	`count(*)`	`FILTER`	`CASE`	`sum … FILTER`

⚠ COMMON MISTAKES

- Putting row conditions in `HAVING` : same result, slower
- Using an aggregate in `WHERE` (error)
- `SUM(CASE … END)` without `ELSE 0` and getting NULL

✓ TIPS & BEST PRACTICES

- Read it as: filter rows, group, filter groups
- Build dashboards with one `FILTER` -based query
- SQL Server: repeat the expression in `HAVING` , not the alias

QUICK SUMMARY

- `WHERE` → rows · `HAVING` → groups
- `FILTER (WHERE …)` per aggregate
- `SUM(CASE …)` everywhere

ROLLUP, CUBE & GROUPING SETS

SUBTOTALS IN ONE QUERY

These extensions run **several groupings at once** and stack the results. **`GROUPING SETS`** lists exactly which groupings you want; **`ROLLUP (a, b)`** is the hierarchy `(a, b)`, `(a)`, `()` — detail, subtotal, grand total; **`CUBE (a, b)`** is every combination. In subtotal rows the rolled-up columns are **NULL**, and **`GROUPING(col)`** returns 1 there, so you can tell a subtotal from a real NULL.

FACTS

- `()` is the empty grouping: the grand total
- `ROLLUP (a, b, c)` → 4 levels; order matters
- `CUBE` with n columns → 2^n groupings
- Sort with `GROUPING()` to put totals last

NOSQL COMPARISON

- MongoDB has no `ROLLUP`
- `$facet` runs several `$group` pipelines over the same input in one query
- Or compute the total in the app from the detail rows

ROLLUP, CUBE & GROUPING SETS

```sql
-- Revenue by country and city, with subtotals
SELECT c.country,
       CASE WHEN GROUPING(c.city) = 1
            THEN '(subtotal)' ELSE c.city END AS city,
       sum(o.total) AS revenue
FROM   orders o
JOIN   customers c USING (customer_id)
GROUP  BY ROLLUP (c.country, c.city)
ORDER  BY c.country, GROUPING(c.city), city;

-- Every combination of department and hire year
SELECT dept_id,
       extract(YEAR FROM hire_date) AS yr,
       count(*)                     AS hires
FROM   employees
GROUP  BY CUBE (dept_id, extract(YEAR FROM hire_date));

-- Just the groupings you name
SELECT status, customer_id, count(*)
FROM   orders
GROUP  BY GROUPING SETS ((status), (customer_id), ());
```

CODE EXPLAINED

→ `ROLLUP` adds a subtotal per country and one grand total

→ `GROUPING(c.city) = 1` marks rows where city was rolled up

→ The grand-total row has NULL country and NULL city

→ `GROUPING SETS` here = 3 separate `GROUP BY` s, one scan

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`ROLLUP`, `CUBE`, `GROUPING SETS`, `GROUPING()`
MySQL	`GROUP BY a, b WITH ROLLUP` only; no `CUBE`
SQL Server	all three, plus legacy `WITH ROLLUP / CUBE`
SQLite	none → `UNION ALL` of separate `GROUP BY`s

IN AN ORM

```sql
const rows = await prisma.$queryRaw`
  SELECT status, count(*)::int AS n
  FROM   orders
  GROUP  BY ROLLUP (status)`;
// ::int avoids count's bigint → JS BigInt
```

WHAT EACH PRODUCES

CLAUSE	GROUPINGS
`ROLLUP (a, b)`	`(a, b)` `(a)` `()`
`ROLLUP (a, b, c)`	`(a, b, c)` `(a, b)` `(a)` `()`
`CUBE (a, b)`	`(a, b)` `(a)` `(b)` `()`
`CUBE (a, b, c)`	all 8 combinations
`GROUPING SETS ((a), (b))`	`(a)` `(b)` — no total
`GROUPING SETS ((a), ())`	`(a)` + grand total

ROLLUP OUTPUT

COUNTRY	CITY	REVENUE
CA	Toronto	400.00
CA	(subtotal)	400.00
US	Austin	900.00
US	Boston	600.00
US	(subtotal)	1,500.00
NULL	(subtotal)	1,900.00 ← grand total

⚠ COMMON MISTAKES

- Treating subtotal NULLs as missing data
- Getting `ROLLUP` column order backwards (city before country)
- `CUBE` on many columns: rows explode

✓ TIPS & BEST PRACTICES

- Label totals with `GROUPING()`, not `coalesce`
- Keep `ROLLUP` to 2–3 levels for readable reports
- Format totals in the report tool, not in SQL

QUICK SUMMARY

- `ROLLUP` — hierarchy + total
- `CUBE` — all combinations
- `GROUPING SETS` — exactly what you list

String & Array Aggregation

STRING_AGG · GROUP_CONCAT · ARRAY_AGG · LISTAGG · JSON_AGG · WITHIN GROUP

COLLAPSING ROWS INTO LISTS

These aggregates join a group's values into **one list** instead of one number. **`string_agg(x, ', ')`** builds delimited text; **`array_agg`** builds a PostgreSQL array; **`json_agg`** builds a JSON array, often of objects from `json_build_object`, so one query can return API-ready nested data. Add **`ORDER BY`** inside the call to control list order, and `DISTINCT` to drop repeats.

LIST FACTS

- Without `ORDER BY` inside, list order is arbitrary
- NULL inputs are skipped by `string_agg`
- MySQL `GROUP_CONCAT` cuts at 1,024 bytes by default
- `json_agg` keeps NULLs as `null`

NOSQL COMPARISON

- MongoDB: `$push` collects values into an array inside `$group`
- `$addToSet` collects distinct values
- With embedded documents the list is often already stored

STRING & ARRAY AGGREGATION

```sql
-- Comma-separated product names per category
SELECT category,
       string_agg(name, ', ' ORDER BY name) AS products
FROM   products
GROUP  BY category;

SELECT string_agg(DISTINCT city, ' / ') FROM customers;

-- Arrays (PostgreSQL)
SELECT customer_id,
       array_agg(order_id ORDER BY order_date) AS orders
FROM   orders
GROUP  BY customer_id;

-- Nested JSON: one row per order with its items
SELECT o.order_id,
       json_agg(json_build_object(
         'product', p.name,
         'qty',     i.quantity)) AS items
FROM   orders o
JOIN   order_items i USING (order_id)
JOIN   products p    USING (product_id)
GROUP  BY o.order_id;
```

CODE EXPLAINED

- → `ORDER BY name` inside the call sorts each list
- → `DISTINCT` inside drops repeated cities
- → `json_build_object` makes one JSON object per row
- → `json_agg` wraps them into an array per order

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`string_agg(x, ',' ORDER BY y)`, `array_agg`, `json_agg`
MySQL	`GROUP_CONCAT(x ORDER BY y SEPARATOR ',')`, `JSON_ARRAYAGG`
SQL Server	`STRING_AGG(x, ',') WITHIN GROUP (ORDER BY y)`
SQLite	`group_concat(x, ',')`; `string_agg` + `ORDER BY` 3.44+

IN AN ORM

```sql
// The ORM way: load children as nested arrays
await prisma.order.findMany({
  include: { items: { include: { product: true } } },
});
```

OUTPUT SHAPES

CALL	ONE GROUP'S RESULT
`string_agg(name, ', ')`	`Desk, Lamp, Shelf`
`array_agg(order_id)`	`{12,15,31}`
`json_agg(name)`	`["Desk", "Lamp"]`
`json_object_agg(k, v)`	`{"a": 1, "b": 2}`
`LISTAGG(x, ', ')`	standard form + `WITHIN GROUP` (Oracle, Db2)

GOOD USES

NEED	USE
tag list next to each post	`string_agg(tag, ', ')`
API response with children	`json_agg(json_build_object(…))`
IDs to pass to another query	`array_agg(id)`
CSV-style export line	`string_agg(col, ',')`

⚠ COMMON MISTAKES

- Forgetting `ORDER BY` inside and getting shuffled lists
- MySQL lists silently cut at `group_concat_max_len`
- Splitting the string back apart later: keep arrays or JSON

✓ TIPS & BEST PRACTICES

- Raise `group_concat_max_len` for long MySQL lists
- Use `jsonb_agg` when you'll query the result further
- `array_agg(DISTINCT x ORDER BY x)` for tidy, unique lists

QUICK SUMMARY

- `string_agg` / `GROUP_CONCAT` → text
- `array_agg` → array · `json_agg` → JSON
- `ORDER BY` inside controls order

INNER JOIN

MATCHING ROWS ACROSS TABLES

A **join** pairs rows from two tables wherever the **ON** condition is true. **`INNER JOIN`** (plain `JOIN`) keeps only the pairs that match: an order with no customer, or a customer with no orders, disappears. A row that matches three rows appears **three times**. **`USING (col)`** is shorthand when both tables share a column name. **Table aliases** (`orders o`) keep long joins readable.

JOIN FACTS

- `JOIN` = `INNER JOIN`
- Join on keys: FK = PK is the normal case
- `USING` merges the shared column into one
- `NATURAL JOIN` matches every same-named column

NOSQL COMPARISON

- MongoDB: `$lookup: { from: 'customers', localField: 'customer_id', foreignField: '_id', as: 'customer' }`
- `$unwind` turns the array into one document per match
- Often the name is simply **embedded** in the order instead

INNER JOIN

```sql
SELECT o.order_id, o.order_date, c.name
FROM   orders o
JOIN   customers c
  ON   c.customer_id = o.customer_id;

SELECT order_id, name
FROM   orders
JOIN   customers USING (customer_id);

-- Extra conditions belong in ON too
SELECT p.name, i.quantity
FROM   order_items i
JOIN   products p
  ON   p.product_id = i.product_id
 AND   p.category   = 'Books';

SELECT e.first_name, d.name AS department
FROM   employees e
INNER  JOIN departments d ON d.dept_id = e.dept_id;

-- Old comma style: same result, easy to break
SELECT o.order_id, c.name
FROM   orders o, customers c
WHERE  c.customer_id = o.customer_id;
```

CODE EXPLAINED

- → `c.customer_id = o.customer_id` — FK in orders = PK in customers
- → `USING` outputs a single `customer_id` column
- → Employees with a NULL `dept_id` drop out of an inner join
- → Comma joins without a `WHERE` become cross joins (sheet 34)

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`ON`, `USING`, `NATURAL`; `USING (…) AS j` (PG 14+)
MySQL	`ON`, `USING`, `NATURAL`; `STRAIGHT_JOIN` fixes order
SQL Server	`ON` only: no `USING`, no `NATURAL JOIN`
SQLite	`ON`, `USING`, `NATURAL`

IN AN ORM

```ts
await prisma.order.findMany({
  select: {
    id: true,
    orderDate: true,
    customer: { select: { name: true } },  // relation
  },
});
```

JOIN SYNTAX

FORM	EXAMPLE
`ON`	`JOIN customers c ON c.customer_id = o.customer_id`
`USING`	`JOIN customers USING (customer_id)`
`NATURAL`	`NATURAL JOIN customers` (avoid)
comma	`FROM a, b WHERE a.id = b.a_id` (avoid)
composite key	`ON a.x = b.x AND a.y = b.y`

WHAT INNER KEEPS

ORDER	CUSTOMER_ID	CUSTOMERS ROW?	IN RESULT?
101	7	Ada (7)	yes
102	7	Ada (7)	yes — Ada twice
103	9	Bo (9)	yes
104	NULL	none	no
—	—	Cy (11), no orders	no

⚠ COMMON MISTAKES

- `NATURAL JOIN` breaking when both tables gain a `created_at`
- Forgetting a join condition and multiplying rows
- Using `INNER` when rows without a match should stay (sheet 33)

✓ TIPS & BEST PRACTICES

- Use short, meaningful aliases: `o`, `c`, `oi`
- Always write explicit `JOIN … ON`
- Index foreign key columns used in joins (sheet 71)

QUICK SUMMARY

- `JOIN … ON` — matched pairs only
- `USING (col)` — same-named key
- One row per matching pair

Outer Joins

LEFT JOIN · RIGHT JOIN · FULL OUTER JOIN · NULL-extended rows · ON vs WHERE filter

KEEPING UNMATCHED ROWS

An **outer join** keeps rows that have **no match** and fills the other side's columns with **NULL**. **`LEFT JOIN`** keeps every row from the left table; **`RIGHT JOIN`** every row from the right; **`FULL OUTER JOIN`** both. The trap is where you filter: a condition on the right table belongs in **`ON`**. Put it in `WHERE` and the NULL rows fail it, which silently turns the outer join back into an inner join.

OUTER JOIN FACTS

- `LEFT JOIN` = `LEFT OUTER JOIN`
- `RIGHT JOIN` is a `LEFT JOIN` with tables swapped
- Count the right side's key, not `*`, to get zeros
- `WHERE right.id IS NULL` → rows with no match (sheet 36)

NOSQL COMPARISON

- `$lookup` already behaves like a **left** outer join
- No match → an empty array, not a dropped document
- `$unwind` with `preserveNullAndEmptyArrays: true` keeps them

OUTER JOINS

```sql
-- Every customer, with orders if any
SELECT c.name, o.order_id, o.total
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id;

-- Order count per customer, including zero
SELECT c.name, count(o.order_id) AS orders
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.name;

-- Filter the right table in ON
SELECT c.name, o.order_id
FROM   customers c
LEFT   JOIN orders o
  ON   o.customer_id = c.customer_id
 AND   o.status = 'paid';     -- all customers kept
-- WHERE o.status = 'paid'   -- would drop them

-- Every department, even with no staff
SELECT d.name, e.first_name
FROM   employees e
RIGHT  JOIN departments d ON d.dept_id = e.dept_id;

SELECT c.customer_id, o.order_id
FROM   customers c
FULL   OUTER JOIN orders o
  ON   o.customer_id = c.customer_id;
```

CODE EXPLAINED

- → Customers with no orders get NULL `order_id` and `total`
- → `count(o.order_id)` → 0 for them; `count(*)` would say 1
- → `o.status` in `ON` limits which orders attach
- → `FULL OUTER` also keeps orders with no customer

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`LEFT`, `RIGHT`, `FULL` all supported
MySQL	no `FULL OUTER JOIN` → `LEFT … UNION … RIGHT`
SQL Server	all three; old `*=` syntax is gone
SQLite	`RIGHT` and `FULL` since 3.39

IN AN ORM

```prisma
await prisma.customer.findMany({
  include: {
    orders: { where: { status: 'paid' } },
  },
});  // every customer; orders = [] when none match
```

WHICH ROWS SURVIVE

JOIN	UNMATCHED LEFT	UNMATCHED RIGHT
`INNER`	dropped	dropped
`LEFT`	kept, right side NULL	dropped
`RIGHT`	dropped	kept, left side NULL
`FULL OUTER`	kept	kept

ON VS WHERE IN A LEFT JOIN

CONDITION ON THE RIGHT TABLE	EFFECT
`in ON`	limits matches; every left row stays
`in WHERE`	removes NULL rows → acts as INNER
`WHERE right.key IS NULL`	only left rows with no match
`WHERE right.col IS NULL OR …`	keeps them; usually means ON

⚠ COMMON MISTAKES

- Filtering the right table in `WHERE` after a `LEFT JOIN`
- `count(*)` showing 1 order for customers with none
- Mixing `LEFT` and `RIGHT` in one query; rewrite as all `LEFT`

✓ TIPS & BEST PRACTICES

- Default to `LEFT JOIN` and put the "keep all" table first
- Wrap nullable outputs in `coalesce` for display
- Check row counts before and after adding a join

QUICK SUMMARY

- `LEFT` / `RIGHT` / `FULL` keep unmatched rows
- Missing side → NULLs
- Right-table filters go in `ON`

CROSS JOIN & Self Joins

CROSS JOIN · Cartesian product · self join · employee/manager · pairs · date spine

EVERY COMBINATION, AND A TABLE WITH ITSELF

`CROSS JOIN` pairs **every** row of one table with every row of another (the Cartesian product): 3 sizes × 2 colors = 6 rows. It's useful for building grids and calendars, and dangerous by accident. A **self join** joins a table to itself under two aliases, which is how you follow `employees.manager_id` back to `employees`, or compare rows within one table. A **date spine** (one row per day) left-joined to your data shows days with zero activity.

FACTS

- Row count of a cross join = m × n
- A self join needs two different aliases
- `a.id < b.id` lists each pair once
- For the previous row, use `LAG` (sheet 46)

NOSQL COMPARISON

- MongoDB has no cross join
- Self references use `$lookup` on the same collection
- Whole hierarchies: `$graphLookup` (sheet 42)

CROSS & SELF JOINS

```sql
-- Every size with every color: 3 × 2 = 6 rows
SELECT s.size, c.color
FROM   (VALUES ('S'), ('M'), ('L')) AS s(size)
CROSS  JOIN (VALUES ('red'), ('blue')) AS c(color);

-- Self join: employee and manager
SELECT e.first_name AS employee,
       m.first_name AS manager
FROM   employees e
LEFT   JOIN employees m ON m.emp_id = e.manager_id;

-- Same-category product pairs, each pair once
SELECT a.name, b.name
FROM   products a
JOIN   products b
  ON   a.category   = b.category
 AND   a.product_id < b.product_id;

-- Date spine: every day, even with no orders
SELECT d::date AS day, count(o.order_id) AS orders
FROM   generate_series(DATE '2026-10-01',
                       DATE '2026-10-31',
                       INTERVAL '1 day') AS d
LEFT   JOIN orders o ON o.order_date = d::date
GROUP  BY d
ORDER  BY d;
```

CODE EXPLAINED

- → `VALUES … AS s(size)` — an inline table with a column name
- → `LEFT JOIN` keeps the top boss, whose `manager_id` is NULL
- → `<` instead of `<>` avoids (A, B) and (B, A) duplicates
- → `generate_series` builds the 31 days to join against

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`CROSS JOIN`; `generate_series` for spines
MySQL	`CROSS JOIN`; recursive CTE for spines (sheet 42)
SQL Server	`CROSS JOIN`; `GENERATE_SERIES` (2022+)
SQLite	`CROSS JOIN` also forces join order

IN AN ORM

```ts
// schema: a self-relation named "Manager"
await prisma.employee.findMany({
  include: {
    manager: { select: { firstName: true } },
  },
});
```

ROW COUNTS

JOIN	ROWS (M LEFT, N RIGHT)
`CROSS JOIN`	m × n
`INNER JOIN on a key`	≤ number of matches
`self join, all pairs a ≠ b`	n × (n − 1)
`self join, a < b`	n × (n − 1) / 2
`date spine LEFT JOIN`	≥ number of days

USES

JOIN	GOOD FOR
`CROSS JOIN`	size × color grids, store × day calendars
`CROSS JOIN 1-row table`	attach a constant or setting to every row
`self join`	manager chains, comparing rows in one table
`self join on dates`	consecutive events (prefer `LAG`)
`spine + LEFT JOIN`	charts with no missing days

⚠ COMMON MISTAKES

- Accidental cross join from a missing `ON`: millions of rows
- Self join with `<>` returning every pair twice
- Using `INNER` for the manager join and losing the CEO

✓ TIPS & BEST PRACTICES

- Check `count(*)` on any join that looks slow
- Name self-join aliases by role: `e` mployee, `m` anager
- Keep a calendar table for reporting spines

QUICK SUMMARY

- `CROSS JOIN` — all combinations
- Self join — two aliases, one table
- Spine + `LEFT JOIN` — no gaps

Multi-Table Joins

3+ table joins · junction table · join fan-out · duplicate rows · join order

CHAINING JOINS

Joins chain left to right: each `JOIN` adds a table to the rows built so far. A **junction table** (`order_items`) links two tables in a many-to-many relationship, so going from orders to products takes two joins. The big risk is **fan-out**: joining a parent to a child repeats the parent once per child row, so summing a parent column after the join counts it many times. The fix is to **aggregate the child first**, then join.

MULTI-JOIN FACTS

- The planner picks join order; write it for readers
- Each join should follow a key relationship
- Two child tables at once multiply each other
- `DISTINCT` hides fan-out; it doesn't fix it

NOSQL COMPARISON

- Chained `$lookup` stages work but get slow
- Typical design embeds order items, copying name and price
- Copied data is a snapshot: it doesn't follow later changes

MULTI-TABLE JOINS

```sql
-- Order lines with customer and product names
SELECT o.order_id, c.name AS customer,
       p.name AS product, i.quantity, i.unit_price
FROM   orders o
JOIN   customers   c ON c.customer_id = o.customer_id
JOIN   order_items i ON i.order_id    = o.order_id
JOIN   products    p ON p.product_id  = i.product_id
WHERE  o.order_date >= DATE '2026-10-01';

-- Fan-out: o.total repeats once per order item
SELECT c.name, sum(o.total) AS spent        -- WRONG
FROM   customers c
JOIN   orders o      USING (customer_id)
JOIN   order_items i USING (order_id)
GROUP  BY c.name;

-- Fix: aggregate the child first, then join
SELECT c.name, sum(o.total) AS spent,
       sum(q.items)          AS items
FROM   customers c
JOIN   orders o USING (customer_id)
JOIN   (SELECT order_id, sum(quantity) AS items
        FROM order_items GROUP BY order_id) q
       USING (order_id)
GROUP  BY c.name;
```

CODE EXPLAINED

- → Each `JOIN … ON` follows one foreign key
- → An order with 3 items appears 3 times after the item join
- → `sum(o.total)` then counts that order 3 times
- → The derived table returns one row per order (sheet 40)

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	past `join_collapse_limit` (8) it keeps your order
MySQL	`STRAIGHT_JOIN` forces the written order
SQL Server	`OPTION (FORCE ORDER)` hint
SQLite	`CROSS JOIN` forces the order

IN AN ORM

```ts
await prisma.order.findMany({
  where: { orderDate: { gte: new Date('2026-10-01') } },
  include: {
    customer: true,
    items: { include: { product: true } },
  },
});
```

JOIN PATHS IN THE SHOP SCHEMA

FROM → TO	ON
`orders → customers`	`o.customer_id = c.customer_id`
`orders → order_items`	`i.order_id = o.order_id`
`order_items → products`	`p.product_id = i.product_id`
`employees → departments`	`d.dept_id = e.dept_id`
`employees → manager`	`m.emp_id = e.manager_id`

FAN-OUT CHECKLIST

SYMPTOM	LIKELY CAUSE	FIX
`totals too high`	parent summed after child join	aggregate child first
`duplicate rows`	joined two children at once	separate subqueries
`count(*) too big`	counting joined rows	`count(DISTINCT key)`
`DISTINCT "fixes" it`	real fan-out underneath	find the 1-to-many join

⚠ COMMON MISTAKES

- Summing a parent column after joining its children
- Joining two unrelated child tables in one query
- Missing one join condition among many

✓ TIPS & BEST PRACTICES

- Line up `JOIN` and `ON` clauses so missing keys stand out
- Check row counts after each added join
- Pre-aggregate in CTEs, then join the summaries (sheet 41)

QUICK SUMMARY

- Joins chain table by table
- Junction table = two joins
- Fan-out → aggregate first

Semi & Anti Joins

"HAS ANY" AND "HAS NONE"

A **semi-join** returns rows from one table that **have at least one** match in another, without duplicating them. Write it with **`EXISTS`** or **`IN`**. An **anti-join** returns rows with **no** match: **`NOT EXISTS`** or **`LEFT JOIN … WHERE right.key IS NULL`**. Avoid **`NOT IN`** with a subquery: if the subquery returns a single NULL, `NOT IN` is never true and you get **no rows** at all.

FACTS

- `EXISTS` stops at the first match
- `SELECT 1` inside `EXISTS` is convention; the list is ignored
- Semi-joins never duplicate the outer row
- `NOT EXISTS` is NULL-safe; `NOT IN` isn't

NOSQL COMPARISON

- MongoDB: `$lookup`, then `$match: { orders: { $ne: [] } }` (has any)
- `{ orders: { $size: 0 } }` for has none
- Or keep an `orderCount` field on the customer

EXISTS, IN & ANTI-JOINS

```sql
-- Semi-join: customers who have ordered
SELECT c.name
FROM   customers c
WHERE  EXISTS (SELECT 1 FROM orders o
               WHERE o.customer_id = c.customer_id);

SELECT name FROM customers
WHERE  customer_id IN (SELECT customer_id FROM orders);

-- Anti-join: customers who never ordered
SELECT c.name
FROM   customers c
WHERE  NOT EXISTS (SELECT 1 FROM orders o
                   WHERE o.customer_id = c.customer_id);

SELECT c.name
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.order_id IS NULL;

-- NOT IN trap: employees who manage nobody
SELECT first_name FROM employees
WHERE  emp_id NOT IN (SELECT manager_id FROM employees);
-- manager_id has a NULL (the CEO) → 0 rows!
SELECT first_name FROM employees e
WHERE  NOT EXISTS (SELECT 1 FROM employees r
                   WHERE r.manager_id = e.emp_id);
```

CODE EXPLAINED

- → `EXISTS` and `IN` return each customer once, however many orders
- → `o.order_id IS NULL` keeps only unmatched left rows
- → `x NOT IN (1, NULL)` is UNKNOWN for every `x`
- → `NOT EXISTS` ignores the NULL and works

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	plans `EXISTS` / `NOT EXISTS` as semi / anti joins
MySQL	semi- and anti-join optimizations (8.0.17+)
SQL Server	same results; `NOT EXISTS` plans cleanly
SQLite	runs `EXISTS` per row: index the inner column

IN AN ORM

```ts
await prisma.customer.findMany({
  where: { orders: { some: {} } },   // EXISTS
});
await prisma.customer.findMany({
  where: { orders: { none: {} } },   // NOT EXISTS
});
```

PATTERNS

PATTERN	RETURNS	NULL-SAFE?
`EXISTS (…)`	rows with a match	yes
`IN (subquery)`	rows with a match	yes
`NOT EXISTS (…)`	rows with no match	yes
`LEFT JOIN … IS NULL`	rows with no match	yes
`NOT IN (subquery)`	rows with no match	**no**
`JOIN + DISTINCT`	rows with a match	yes, but slower

PICK ONE

NEED	WRITE
`has at least one`	`EXISTS` or `IN`
`has none`	`NOT EXISTS`
`has none + right-side columns`	`LEFT JOIN … IS NULL`
`literal list without NULLs`	`NOT IN (1, 2, 3)` is fine

⚠ COMMON MISTAKES

- `NOT IN (SELECT nullable_col …)` returning nothing
- Using `JOIN` for "has any" and getting duplicates
- Testing a column that is never NULL in the anti-join `IS NULL`

✓ TIPS & BEST PRACTICES

- Default to `EXISTS` / `NOT EXISTS` for subquery tests
- Index the foreign key the subquery looks up
- In anti-joins, test the right table's primary key `IS NULL`

QUICK SUMMARY

- Semi-join — `EXISTS` / `IN`
- Anti-join — `NOT EXISTS` / `LEFT JOIN … IS NULL`
- Avoid `NOT IN` with subqueries

LATERAL & APPLY

LATERAL · CROSS APPLY · OUTER APPLY · top-N per row · set-returning functions

A SUBQUERY PER ROW

A normal subquery in FROM can't see the other tables in the same FROM. **LATERAL** lifts that rule: the subquery runs **once per outer row** and may reference its columns, like a `for each` loop. That makes **top-N per group** easy: the 3 latest orders for each customer. SQL Server calls the same thing **`CROSS APPLY`** (inner) and **`OUTER APPLY`** (keeps rows with no result).

LATERAL FACTS

- `CROSS JOIN LATERAL` drops rows with no result
- `LEFT JOIN LATERAL … ON true` keeps them
- Index the inner lookup or each row scans
- Window functions also do top-N (sheet 49)

NOSQL COMPARISON

- MongoDB `$lookup` with a `pipeline` runs per document:
- `pipeline: [{ $match: { $expr: … } }, { $sort: { date: -1 } }, { $limit: 3 }]`
- That's the same top-N-per-parent idea

LATERAL & APPLY

```sql
-- Latest 3 orders per customer
SELECT c.name, r.order_id, r.order_date
FROM   customers c
CROSS  JOIN LATERAL (
         SELECT o.order_id, o.order_date
         FROM   orders o
         WHERE  o.customer_id = c.customer_id
         ORDER  BY o.order_date DESC
         LIMIT  3) AS r;

-- Keep customers with no orders
SELECT c.name, r.order_id, r.total
FROM   customers c
LEFT   JOIN LATERAL (
         SELECT order_id, total FROM orders o
         WHERE  o.customer_id = c.customer_id
         ORDER  BY order_date DESC
         LIMIT  1) AS r ON true;

-- SQL Server equivalent
SELECT c.name, r.order_id
FROM   customers c
CROSS  APPLY (SELECT TOP (3) order_id FROM orders o
              WHERE  o.customer_id = c.customer_id
              ORDER  BY order_date DESC) AS r;
```

CODE EXPLAINED

- → `c.customer_id` inside the subquery is the lateral reference
- → `LIMIT 3` applies per customer, not to the whole result
- → `ON true` — the subquery already did the matching
- → `CROSS APPLY` ≈ `CROSS JOIN LATERAL`

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`JOIN LATERAL`; `FROM` functions are implicitly lateral
MySQL	`LATERAL` derived tables since 8.0.14
SQL Server	`CROSS APPLY` / `OUTER APPLY`
SQLite	no `LATERAL` → window functions or a correlated subquery

IN AN ORM

```ts
await prisma.customer.findMany({
  include: {
    orders: { orderBy: { orderDate: 'desc' }, take: 3 },
  },
});  // top-N per parent without raw SQL
```

EQUIVALENTS

POSTGRESQL / MYSQL	SQL SERVER
`CROSS JOIN LATERAL (…) r`	CROSS APPLY (…) r
`LEFT JOIN LATERAL (…) r ON true`	OUTER APPLY (…) r
`FROM t, LATERAL f(t.col)`	CROSS APPLY f(t.col)
`unnest(t.arr)`	CROSS APPLY OPENJSON(t.js)

WHEN TO USE

NEED	TOOL
top N rows per parent	`LATERAL … LIMIT n`
several columns from one latest row	`LATERAL … LIMIT 1`
expand an array / JSON per row	`LATERAL unnest()` / `jsonb_array_elements`
rank across the whole set	window function (sheet 45)
single value per row	scalar subquery (sheet 38)

⚠ COMMON MISTAKES

- `CROSS JOIN LATERAL` silently dropping parents with no rows
- No index on the inner `WHERE` column: one scan per outer row
- Using `LATERAL` where a simple join would do

✓ TIPS & BEST PRACTICES

- Index `(customer_id, order_date DESC)` for top-N lookups
- Use `LIMIT 1` lateral to fetch several columns of the latest row
- Compare plans with the `ROW_NUMBER()` version (sheet 49)

QUICK SUMMARY

- `LATERAL` — subquery sees outer row
- `CROSS / OUTER APPLY` in SQL Server
- Top-N per group made simple

Subqueries — Scalar & IN

QUERIES INSIDE QUERIES

A **subquery** is a SELECT in parentheses used inside another statement. A **scalar subquery** returns one row and one column, so it can stand in for a value anywhere; zero rows give NULL and two or more rows are an error. A **column subquery** returns a list for **IN**, **ANY** or **ALL**. A subquery that doesn't reference the outer query runs **once**.

SUBQUERY FACTS

- Always wrap subqueries in parentheses
- `= ANY (…)` is the same as `IN (…)`
- `> ALL` of an **empty** set is true for every row
- Referencing the outer query makes it correlated (sheet 39)

NOSQL COMPARISON

- MongoDB has no inline subqueries: run two queries
- Or compute the comparison value in the pipeline:
- `$setWindowFields` / `$group`, then `$match` with `$expr`

SCALAR & IN SUBQUERIES

```sql
-- Products priced above the average
SELECT name, price
FROM   products
WHERE  price > (SELECT avg(price) FROM products);

-- Scalar subquery as a column
SELECT name, price,
       price - (SELECT avg(price) FROM products) AS vs_avg
FROM   products;

-- IN: orders from Canadian customers
SELECT order_id, total
FROM   orders
WHERE  customer_id IN (SELECT customer_id
                       FROM   customers
                       WHERE  country = 'CA');

-- ALL / ANY
SELECT name FROM products
WHERE  price > ALL (SELECT price FROM products
                    WHERE  category = 'Books');
SELECT name FROM products
WHERE  price = ANY (SELECT unit_price FROM order_items);

SELECT (SELECT order_id FROM orders);
-- ERROR: more than one row returned by a subquery
```

CODE EXPLAINED

- → `(SELECT avg(price) …)` runs once; its value is reused
- → `IN (SELECT …)` builds the list of Canadian customer IDs
- → `> ALL (…)` — pricier than every book
- → A scalar subquery must return at most one row

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`ANY` / `ALL` also accept arrays: `= ANY(ARRAY[1,2])`
MySQL	`ANY`, `SOME`, `ALL`; no `LIMIT` inside `IN (…)`
SQL Server	`ANY`, `SOME`, `ALL`; `TOP (1)` keeps scalars safe
SQLite	no `ANY` / `ALL` → `IN`, `max()`, `min()`

IN AN ORM

```ts
const { _avg } = await prisma.product.aggregate({
  _avg: { price: true },
});
await prisma.product.findMany({
  where: { price: { gt: _avg.price ?? 0 } },
});
```

SUBQUERY SHAPES

SHAPE	USE IT AS	SHEET
`scalar (1 × 1)`	a value in `SELECT`, `WHERE`, `SET`	38
`column (n × 1)`	`IN`, `ANY`, `ALL`	38
`table (n × m)`	`FROM (…) AS t`	40
`correlated`	per-row comparison	39
`existence`	`EXISTS (…)`	36

ANY / ALL

EXPRESSION	TRUE WHEN
`x > ANY (s)`	x beats at least one value
`x > ALL (s)`	x beats every value
`x = ANY (s)`	same as `x IN (s)`
`x <> ALL (s)`	same as `x NOT IN (s)` (NULL trap)
`x > ALL (empty)`	always true

⚠ COMMON MISTAKES

- A "scalar" subquery that sometimes returns two rows
- `> ALL` against an empty set matching everything
- Repeating the same scalar subquery in five places

✓ TIPS & BEST PRACTICES

- Name a repeated subquery once with a CTE (sheet 41)
- Prefer `EXISTS` for big correlated lists (sheet 36)
- Add `LIMIT 1` / `max()` when one row must win

QUICK SUMMARY

- Scalar → one value
- Column → `IN` / `ANY` / `ALL`
- Uncorrelated → runs once

Correlated Subqueries

SUBQUERIES THAT LOOK OUTWARD

A **correlated subquery** refers to a column of the **outer** query (`p.category` below), so logically it runs **once per outer row**. That's how you compare each row with its own group: products priced above **their category's** average, or each customer's latest order date. Planners often rewrite these into joins, but not always. When one is slow, rewrite it as a **join to a grouped derived table**, a **window function** or a LATERAL join (sheet 37).

FACTS

- You can't run a correlated subquery on its own
- `EXISTS` subqueries are almost always correlated
- Outer aliases must be unique to avoid confusion
- Slow? Check the plan for a per-row SubPlan (sheet 73)

NOSQL COMPARISON

- MongoDB `$lookup` with `let` + a `pipeline` using `$expr` is correlated
- Each document's values feed the inner pipeline
- Same per-document cost: index the inner match field

CORRELATED SUBQUERIES

```sql
-- Products above their own category's average
SELECT p.name, p.category, p.price
FROM   products p
WHERE  p.price > (SELECT avg(x.price)
                  FROM   products x
                  WHERE  x.category = p.category);

-- Rewrite 1: join a grouped derived table
SELECT p.name, p.category, p.price
FROM   products p
JOIN  (SELECT category, avg(price) AS avg_price
       FROM   products GROUP BY category) a
  ON   a.category = p.category
WHERE  p.price > a.avg_price;

-- Rewrite 2: window function, one pass
SELECT name, category, price
FROM  (SELECT *, avg(price) OVER w AS cat_avg
       FROM products WINDOW w AS (PARTITION BY category)) t
WHERE  price > cat_avg;
```

CODE EXPLAINED

- → `x.category = p.category` — the outer reference
- → Logically the subquery reruns for every product row
- → Rewrite 1 computes each average once, then joins
- → Rewrite 2 attaches the average to every row (sheet 44)

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	flattens `EXISTS` / `IN`; scalar ones run per row
MySQL	`subquery_to_derived` switch can rewrite (off by default)
SQL Server	usually rewrites them into joins itself
SQLite	runs per row; index the inner `WHERE` column

IN AN ORM

```
// Relation filters compile to correlated EXISTS
await prisma.customer.findMany({
  where: { orders: { some: { total: { gt: 500 } } } },
});
```

CORRELATED VS NOT

	UNCORRELATED	CORRELATED
references outer query	no	yes
runs	once	per outer row (logically)
can run alone	yes	no
typical use	constant, `IN` list	per-row compare, `EXISTS`

REWRITE OPTIONS

PATTERN	REWRITE AS	SHEET
compare to group aggregate	join grouped derived table	40
compare to group aggregate	window function	44
latest row per parent	`LATERAL` / `DISTINCT ON`	37
has any / has none	`EXISTS` / `NOT EXISTS`	36
count per parent	`LEFT JOIN … GROUP BY`	33

⚠ COMMON MISTAKES

- Reusing the same alias inside and outside (`p` and `p`)
- A per-row scalar subquery over a big table with no index
- Assuming every engine flattens it into a join

✓ TIPS & BEST PRACTICES

- Index the inner lookup column (`orders.customer_id`)
- Compare plans of the subquery and rewritten forms
- Keep correlated subqueries for clarity on small sets

QUICK SUMMARY

- Inner query references outer row
- Logically runs per row
- Rewrite with joins or windows when slow

Derived Tables

FROM (subquery) · required alias · inline view · nested aggregation · aggregate of aggregate

A QUERY AS A TABLE

A **derived table** is a subquery in FROM; the outer query treats its result like a table (it's also called an **inline view**). It's the classic way to **aggregate twice** (average of per-customer counts), to filter on a column you computed, or to pre-aggregate a child table before joining it. Most engines require an **alias** after the parentheses, and every column needs a name.

FACTS

- Alias required in MySQL and SQL Server
- Name computed columns, or alias them `AS t(a, b)`
- The planner usually merges it into the outer query
- Same idea, named up front: a CTE (sheet 41)

NOSQL COMPARISON

- MongoDB pipelines nest naturally: `$group`, then `$group` again
- `[{ $group: { _id: '$customerId', n: { $sum: 1 } } }, { $group: { _id: null, avg: { $avg: '$n' } } }]`

DERIVED TABLES

```sql
-- Average number of orders per customer
SELECT avg(n) AS avg_orders
FROM  (SELECT customer_id, count(*) AS n
       FROM   orders
       GROUP  BY customer_id) AS per_customer;

-- Filter on a computed column
SELECT *
FROM  (SELECT name, price * stock AS stock_value
       FROM   products) AS t
WHERE  stock_value > 1000;

-- Largest order per customer, with its date
SELECT o.customer_id, o.order_id, o.total, o.order_date
FROM   orders o
JOIN  (SELECT customer_id, max(total) AS top
       FROM   orders
       GROUP  BY customer_id) m
  ON   m.customer_id = o.customer_id
 AND   m.top = o.total;

-- Name the columns on the alias
SELECT t.status, t.n
FROM  (SELECT status, count(*) FROM orders GROUP BY status)
       AS t(status, n);
```

CODE EXPLAINED

- → `avg(count(*))` isn't allowed; the derived table splits it in two
- → `stock_value` exists for the outer `WHERE`
- → Joining back on `max(total)` returns ties too
- → `AS t(status, n)` names unnamed columns

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	alias optional since PG 16
MySQL	"Every derived table must have its own alias"
SQL Server	alias required; every column must be named
SQLite	alias optional

IN AN ORM

```ts
const per = await prisma.order.groupBy({
  by: ['customerId'],
  _count: { _all: true },
});
const avg = per.reduce((s, r) => s + r._count._all, 0)
  / per.length;   // second aggregation in JS
```

USES

NEED	PATTERN
aggregate of an aggregate	`SELECT avg(n) FROM (… GROUP BY …) t`
filter on an alias	`SELECT * FROM (…) t WHERE alias > …`
pre-aggregate before a join	`JOIN (… GROUP BY key) s USING (key)`
top row per group	`JOIN (… max(x) …)` or `ROW_NUMBER()` (sheet 49)
literal rows	`FROM (VALUES (1,'a'), (2,'b')) AS v(id, s)`

DERIVED TABLE VS CTE VS VIEW

	DERIVED	CTE	VIEW
defined	inline in `FROM`	`WITH` at the top	`CREATE VIEW`
reusable	no	in one statement	everywhere
readability	nests inward	top to bottom	hidden
sheet	40	41	64

⚠ COMMON MISTAKES

- Forgetting the alias (MySQL, SQL Server error)
- Unnamed expression columns in the derived table
- Nesting three levels deep instead of using a CTE

✓ TIPS & BEST PRACTICES

- Switch to a CTE once nesting passes two levels
- Pre-aggregate children to avoid join fan-out (sheet 35)
- Push filters into the inner query when you can

QUICK SUMMARY

- `FROM (SELECT …) AS t`
- Aggregate twice, filter on aliases
- Alias it (required in many engines)

Common Table Expressions

WITH · multiple CTEs · chained CTEs · MATERIALIZED · NOT MATERIALIZED · CTE vs subquery

NAMED STEPS WITH WITH

A **CTE** names a subquery at the top of a statement with **`WITH name AS (…)`**; the main query then uses it like a table. Several CTEs, separated by commas, can each build on the ones before, so a long report reads **top to bottom** as named steps instead of nesting inward. A CTE lives only for its one statement. PostgreSQL inlines simple CTEs; **`MATERIALIZED`** forces it to compute the CTE once and keep the result.

CTE FACTS

- One `WITH`, many CTEs: `WITH a AS (…), b AS (…)`
- Later CTEs can reference earlier ones
- A CTE can be used several times in the main query
- PG: `INSERT … RETURNING` can feed a CTE

NOSQL COMPARISON

- A MongoDB pipeline is already a list of steps
- No named, reusable intermediate results inside one query
- `$facet` runs several sub-pipelines on the same input

WITH

```
WITH paid AS (
  SELECT customer_id, sum(total) AS spent
  FROM   orders
  WHERE  status = 'paid'
  GROUP  BY customer_id
)
SELECT c.name, p.spent
FROM   customers c
JOIN   paid p USING (customer_id)
ORDER  BY p.spent DESC;

-- Chained CTEs: each builds on the last
WITH monthly AS (
  SELECT date_trunc('month', order_date) AS month,
         sum(total) AS revenue
  FROM   orders GROUP BY month
), changes AS (
  SELECT month, revenue,
         revenue - lag(revenue) OVER (ORDER BY month)
           AS change
  FROM   monthly
)
SELECT * FROM changes WHERE change < 0;
-- PostgreSQL 12+: compute once, keep the result
WITH big AS MATERIALIZED (
  SELECT * FROM orders WHERE total > 1000)
SELECT count(*) FROM big;
```

CODE EXPLAINED

- → `paid` is a named result the main query joins
- → `changes` reads from `monthly`, defined just above
- → `lag(…)` — previous month's revenue (sheet 46)
- → `MATERIALIZED` stops PG from inlining the CTE

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	inlined unless used twice; `[NOT] MATERIALIZED`
MySQL	CTEs since 8.0; no `MATERIALIZED` keyword
SQL Server	always inlined; end the previous statement with `;`
SQLite	CTEs with `MATERIALIZED` hints (3.35+)

IN AN ORM

```
// prisma/sql/topSpenders.sql holds the WITH query
import { topSpenders } from '../generated/prisma/sql';
const rows = await prisma.$queryRawTyped(topSpenders());
```

CTE VS SUBQUERY VS TEMP TABLE

	CTE	SUBQUERY	TEMP TABLE
scope	one statement	one spot	the session
reuse	by name	copy-paste	any query
can index	no	no	yes
reads	top → bottom	inside → out	separate steps

FORMS

WRITE	MEANS
`WITH a AS (…) SELECT …`	one CTE
`WITH a AS (…), b AS (…)`	several; b may use a
`WITH a(x, y) AS (…)`	name the columns
`WITH RECURSIVE …`	self-referencing (sheet 42)
`WITH d AS (DELETE … RETURNING *)`	data-modifying (PG)

⚠ COMMON MISTAKES

- SQL Server: missing `;` before `WITH` → syntax error
- Expecting a CTE to persist for the next statement
- Assuming every engine computes a CTE only once

✓ TIPS & BEST PRACTICES

- One CTE per logical step, named for what it holds
- Debug by running the main query as `SELECT * FROM step`
- Use a temp table when you need an index on the step

QUICK SUMMARY

- `WITH name AS (…)`
- Chain steps; read top to bottom
- Scope: one statement

Recursive CTEs

WITH RECURSIVE · anchor member · recursive member · UNION ALL · hierarchy · CYCLE

QUERIES THAT REPEAT

A **recursive CTE** refers to itself. It has two parts joined by **`UNION ALL`**: the **anchor** runs once and produces the starting rows; the **recursive member** joins the table to the rows produced in the previous round, and repeats until it adds nothing. That's how SQL walks **trees and graphs** of unknown depth (org charts, category trees, bill of materials) and generates series.

RECURSION FACTS

- Anchor first, then `UNION ALL`, then the recursive part
- Recursion stops when a round returns no rows
- Column types must match in both parts
- Always include a stop condition or cycle check

NOSQL COMPARISON

- MongoDB: `$graphLookup` walks a tree in one stage
- `{ from: 'employees', startWith: '$_id', connectFromField: '_id', connectToField: 'managerId', as: 'reports', maxDepth: 5 }`

WITH RECURSIVE

```sql
-- Everyone under employee 1, with depth
WITH RECURSIVE chain AS (
  SELECT emp_id, first_name, manager_id, 1 AS depth
  FROM   employees
  WHERE  emp_id = 1                          -- anchor
  UNION ALL
  SELECT e.emp_id, e.first_name, e.manager_id, c.depth+1
  FROM   employees e
  JOIN   chain c ON e.manager_id = c.emp_id -- recursive
)
SELECT * FROM chain ORDER BY depth;
-- Breadcrumb path from the top
WITH RECURSIVE path AS (
  SELECT emp_id, first_name::text AS trail
  FROM   employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, p.trail || ' > ' || e.first_name
  FROM   employees e
  JOIN   path p ON e.manager_id = p.emp_id
)
SELECT * FROM path;

-- 1 to 10 without generate_series
WITH RECURSIVE n(i) AS (
  SELECT 1 UNION ALL SELECT i + 1 FROM n WHERE i < 10)
SELECT i FROM n;
```

CODE EXPLAINED

- → Round 1: the anchor returns employee 1
- → Each round adds the direct reports of the last round
- → `::text` makes both parts the same type for `||`
- → `WHERE i < 10` is the stop condition

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`WITH RECURSIVE`; `SEARCH` and `CYCLE` clauses (14+)
MySQL	`WITH RECURSIVE`; `cte_max_recursion_depth` = 1000
SQL Server	plain `WITH`; `OPTION (MAXRECURSION n)`, default 100
SQLite	`WITH RECURSIVE`

IN AN ORM

```sql
const tree = await prisma.$queryRaw`
  WITH RECURSIVE chain AS (…)
  SELECT * FROM chain`;
// nested include only reaches a fixed depth:
// include: { reports: { include: { reports: true } } }
```

PARTS

PART	ROLE
`WITH RECURSIVE name`	declares the self-reference
`anchor SELECT`	starting rows, runs once
`UNION ALL`	append each round's rows
`recursive SELECT`	joins to `name` = last round
`stop`	no new rows, or a `WHERE` limit

GUARDS AGAINST LOOPS

GUARD	HOW
`depth limit`	`WHERE depth < 20` in the recursive part
`CYCLE (PG 14+)`	`) CYCLE emp_id SET is_cycle USING route`
`path check`	`WHERE NOT e.emp_id = ANY (path_ids)`
`UNION (not ALL)`	drops repeated rows; stops simple cycles
`engine limit`	`MAXRECURSION` / `cte_max_recursion_depth`

⚠ COMMON MISTAKES

- No stop condition on cyclic data: runs until a limit hits
- Type mismatch between anchor and recursive part
- SQL Server: writing `WITH RECURSIVE` (syntax error)

✓ TIPS & BEST PRACTICES

- Track `depth` and a path array for debugging
- Index the parent column (`manager_id`)
- For deep, frequent tree reads consider `ltree` or closure tables

QUICK SUMMARY

- Anchor + `UNION ALL` + recursive part
- Walks trees of any depth
- Always guard against cycles

Set Operations

UNION · UNION ALL · INTERSECT · EXCEPT · MINUS · column & type compatibility · ORDER BY placement

COMBINING RESULT SETS

Set operators combine the rows of two queries **vertically**. **`UNION ALL`** stacks them as they are; **`UNION`** stacks and removes duplicates; **`INTERSECT`** keeps rows present in both; **`EXCEPT`** (Oracle: `MINUS`) keeps rows in the first but not the second. Both queries must return the **same number of columns** with compatible types, matched by position. Column names come from the first query, and one `ORDER BY` at the end sorts the whole result.

SET FACTS

- `UNION ALL` is cheapest: no duplicate check
- `INTERSECT` binds tighter than `UNION` / `EXCEPT`
- Set operators treat NULLs as equal
- Joins combine sideways; set ops combine downward

NOSQL COMPARISON

- MongoDB: `$unionWith` appends another collection (UNION ALL)
- `$setUnion`, `$setIntersection`, `$setDifference` work on arrays
- No `INTERSECT` between collections; use `$lookup`

SET OPERATIONS

```sql
-- Everyone we can email
SELECT email, 'customer' AS kind FROM customers
UNION ALL
SELECT email, 'employee'         FROM employees
ORDER  BY email;                -- sorts everything

SELECT city FROM customers WHERE country = 'US'
UNION                           -- no duplicates
SELECT city FROM customers WHERE country = 'CA';

-- Customers who ordered in both September and October
SELECT customer_id FROM orders
WHERE  date_trunc('month', order_date) = '2026-09-01'
INTERSECT
SELECT customer_id FROM orders
WHERE  date_trunc('month', order_date) = '2026-10-01';

-- Products never ordered
SELECT product_id FROM products
EXCEPT
SELECT product_id FROM order_items;

-- Parentheses control grouping
(SELECT 1 UNION SELECT 2) EXCEPT SELECT 2;      -- 1
```

CODE EXPLAINED

- → `kind` labels which query each row came from
- → `ORDER BY` after the last query sorts the combined rows
- → `INTERSECT` returns each matching customer once
- → `EXCEPT` = rows in the first result missing from the second

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	all four, plus `INTERSECT ALL` / `EXCEPT ALL`
MySQL	`INTERSECT` / `EXCEPT` since 8.0.31
SQL Server	`UNION [ALL]`, `INTERSECT`, `EXCEPT` (no `ALL`)
SQLite	all four; no parentheses around the parts

IN AN ORM

```ts
const [cs, es] = await Promise.all([
  prisma.customer.findMany({ select: { email: true } }),
  prisma.employee.findMany({ select: { email: true } }),
]);
const everyone = [...cs, ...es];   // UNION ALL in JS
```

OPERATORS

OPERATOR	RETURNS	DUPLICATES
`UNION ALL`	all rows of both	kept
`UNION`	all rows of both	removed
`INTERSECT`	rows in both	removed
`EXCEPT · MINUS`	rows in first, not second	removed
`INTERSECT / EXCEPT ALL`	multiset versions (PG)	counted

RULES

RULE	DETAIL
`column count`	must match
`types`	compatible by position
`names`	taken from the first query
`ORDER BY / LIMIT`	once, at the very end
`grouping`	parentheses (not in SQLite)

⚠ COMMON MISTAKES

- `UNION` when `UNION ALL` was meant: slower, rows vanish
- Columns in a different order in the second query
- `ORDER BY` inside the first query (error or ignored)

✓ TIPS & BEST PRACTICES

- Default to `UNION ALL`; add dedupe only if needed
- Add a literal `kind` column to tell sources apart
- `NOT EXISTS` is often faster than `EXCEPT` on big tables

QUICK SUMMARY

- `UNION [ALL]` · `INTERSECT` · `EXCEPT`
- Same column count, compatible types
- One `ORDER BY` at the end

Window Function Basics

OVER () · PARTITION BY · ORDER BY · WINDOW clause · window vs GROUP BY

CALCULATIONS ACROSS ROWS

A **window function** computes a value over a set of rows related to the current row, but unlike `GROUP BY` it **keeps every row**. **`OVER (…)`** defines the window: **`PARTITION BY`** splits rows into groups, **`ORDER BY`** orders rows inside each group, and an empty `OVER ()` means the whole result. Any aggregate (`sum`, `avg`, `count`) becomes a window function when you add `OVER`. They run after `WHERE`, `GROUP BY` and `HAVING`, so you can't filter on them directly.

WINDOW FACTS

- Detail and group values side by side, no join
- Not allowed in `WHERE` : wrap in a CTE (sheet 49)
- `ORDER BY` inside `OVER` changes the frame (sheet 48)
- `WINDOW w AS (…)` names a window for reuse

NOSQL COMPARISON

- MongoDB 5.0+: `$setWindowFields`
- `{ partitionBy: '$category', sortBy: { price: 1 }, output: { catAvg: { $avg: '$price' } } }`
- Same idea: every document keeps its fields

OVER & PARTITION BY

```sql
-- Each product next to its category average
SELECT name, category, price,
       avg(price) OVER (PARTITION BY category) AS cat_avg
FROM   products;

-- Share of the grand total, every row kept
SELECT order_id, total,
       sum(total) OVER ()                      AS grand,
       round(100 * total / sum(total) OVER (), 1) AS pct
FROM   orders;

-- Number each customer's orders in date order
SELECT customer_id, order_id, order_date,
       row_number() OVER (PARTITION BY customer_id
                          ORDER BY order_date) AS nth
FROM   orders;

-- Named window, used twice
SELECT first_name, dept_id, salary,
       min(salary) OVER w AS dept_min,
       max(salary) OVER w AS dept_max
FROM   employees
WINDOW w AS (PARTITION BY dept_id);
```

CODE EXPLAINED

- → `PARTITION BY category` — one average per category, on every row
- → `OVER ()` — the window is the whole result
- → `row_number()` restarts at 1 for each customer
- → `WINDOW w` avoids repeating the same `OVER (…)`

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	full support, including the `WINDOW` clause
MySQL	since 8.0; `WINDOW` clause supported
SQL Server	since 2012; `WINDOW` clause from 2022
SQLite	since 3.25; `WINDOW` clause supported

IN AN ORM

```ts
const rows = await prisma.$queryRaw`
  SELECT name, category, price,
         avg(price) OVER (PARTITION BY category) AS cat_avg
  FROM   products`;
```

- Prisma has no window-function API: use `$queryRaw` or a TypedSQL file

GROUP BY VS WINDOW

	GROUP BY	WINDOW
rows out	one per group	one per input row
detail columns	only grouped ones	any
runs	before `SELECT`	during `SELECT`
filter on result	`HAVING`	outer query / CTE
example	`avg(price) … GROUP BY category`	`avg(price) OVER (PARTITION BY category)`

OVER (…) PARTS

PART	DOES
`PARTITION BY a`	separate window per value of `a`
`ORDER BY b`	row order in the window; enables ranks
`frame clause`	which rows feed the function (sheet 48)
`OVER ()`	one window: the whole result
`OVER w`	use a named `WINDOW w AS (…)`

⚠ COMMON MISTAKES

- Filtering with `WHERE rank_col = 1` in the same query
- Adding `ORDER BY` to an average window and getting a running average
- Using `GROUP BY` + self-join where a window would do

✓ TIPS & BEST PRACTICES

- Think of `PARTITION BY` as `GROUP BY` that keeps rows
- Name windows with `WINDOW` when used more than once
- Index `(partition cols, order cols)` for big windows

QUICK SUMMARY

- `f() OVER (PARTITION BY … ORDER BY …)`
- Keeps every row
- Runs after `GROUP BY` / `HAVING`

Ranking Functions

NUMBERING AND RANKING ROWS

Ranking functions number rows by the window's **ORDER BY**. They differ only in how they treat **ties**. **row_number** gives every row a unique number (ties broken arbitrarily); **rank** gives ties the same number and then **skips** (1, 2, 2, 4); **dense_rank** doesn't skip (1, 2, 2, 3). **ntile(n)** splits rows into n nearly equal buckets; **percent_rank** and **cume_dist** express position as a fraction from 0 to 1.

RANKING FACTS

- Ranking needs `ORDER BY` inside `OVER`
- `row_number` with ties is non-deterministic
- Add a unique tie-breaker for stable numbering
- `ntile` puts extra rows in the first buckets

NOSQL COMPARISON

- MongoDB `$setWindowFields` outputs: `$documentNumber`, `$rank`, `$denseRank`
- Same tie behavior as SQL's three functions

RANKING

```
SELECT name, price,
       row_number()   OVER w AS rn,
       rank()         OVER w AS rnk,
       dense_rank()   OVER w AS drnk,
       ntile(4)       OVER w AS quartile,
       percent_rank() OVER w AS pct,
       cume_dist()    OVER w AS cume
FROM   products
WINDOW w AS (ORDER BY price DESC);

-- Rank salaries within each department
SELECT dept_id, first_name, salary,
       dense_rank() OVER (PARTITION BY dept_id
                          ORDER BY salary DESC) AS pos
FROM   employees;

-- Stable numbering: break ties on the key
SELECT order_id, order_date,
       row_number() OVER (ORDER BY order_date, order_id)
         AS seq
FROM   orders;
```

CODE EXPLAINED

- → All six share one named window `w`
- → `PARTITION BY dept_id` restarts the ranking per department
- → `dense_rank` — "2nd highest salary" stays 2 with ties at 1st
- → `order_id` makes `seq` the same on every run

DIALECT NOTES

Same in all four (MySQL 8.0+, SQLite 3.25+). `QUALIFY` for filtering on a rank exists only in DuckDB, Snowflake and BigQuery (sheet 49).

IN AN ORM

```
const ranked = (await prisma.product.findMany({
  orderBy: { price: 'desc' },
})).map((p, i) => ({ ...p, rn: i + 1 }));
// row_number in JS; rank / dense_rank → $queryRaw
```

SAME DATA, EACH FUNCTION

PRICE	ROW_NUM	RANK	DENSE	PCT_RANK	CUME
50	1	1	1	0	0.25
40	2	2	2	0.33	0.75
40	3	2	2	0.33	0.75
30	4	4	3	1	1

PICK ONE

NEED	USE
unique sequence, paging	`row_number()`
competition ranking (1, 2, 2, 4)	`rank()`
Nth highest value	`dense_rank()`
quartiles, deciles	`ntile(4)` · `ntile(10)`
percentile position	`percent_rank()` · `cume_dist()`

⚠ COMMON MISTAKES

- Expecting `row_number` ties to come out the same each run
- Using `rank` for "Nth highest" and skipping N after ties
- Forgetting `ORDER BY` inside `OVER` (error or meaningless)

✓ TIPS & BEST PRACTICES

- Always end the window's `ORDER BY` with a unique column
- Filter on rank in an outer query or CTE (sheet 49)
- `percent_rank = (rank - 1) / (rows - 1)`

QUICK SUMMARY

- `row_number` — unique
- `rank` — ties, gaps · `dense_rank` — ties, no gaps
- `ntile` · `percent_rank` · `cume_dist`

Offset & Value Functions

LAG · LEAD · FIRST_VALUE · LAST_VALUE · NTH_VALUE · default offset

LOOKING AT OTHER ROWS

`lag(x)` returns x from the **previous** row in the window's order and **`lead(x)`** from the **next** one; both take an optional offset and a default for when no such row exists. That makes period-over-period changes a one-liner. **`first_value`**, **`last_value`** and **`nth_value`** return a value from a position in the **frame**. Watch `last_value`: the default frame stops at the current row, so you must widen it.

OFFSET FACTS

- `lag(x, n, default)` — `n` rows back
- No previous row → NULL (or the default)
- `lag` / `lead` ignore the frame; value functions don't
- Partition so a customer's lag never reads another's row

NOSQL COMPARISON

- MongoDB: `$shift: { output: '$total', by: -1, default: 0 }` = `lag`
- `$first` / `$last` with a `window` give first / last value
- Both run inside `$setWindowFields`

LAG, LEAD & VALUE FUNCTIONS

```sql
-- Change from the previous order, per customer
SELECT customer_id, order_date, total,
       lag(total)         OVER w AS prev_total,
       total - lag(total) OVER w AS change,
       lead(order_date)   OVER w AS next_date
FROM   orders
WINDOW w AS (PARTITION BY customer_id
             ORDER BY order_date, order_id);

-- Offset and default
SELECT order_id, total,
       lag(total, 2, 0) OVER (ORDER BY order_id) AS two_back
FROM   orders;

-- First, last and 2nd in each department
SELECT first_name, dept_id, salary,
       first_value(first_name) OVER w AS top_paid,
       last_value(first_name)  OVER w AS lowest_paid,
       nth_value(first_name, 2) OVER w AS runner_up
FROM   employees
WINDOW w AS (PARTITION BY dept_id ORDER BY salary DESC
             ROWS BETWEEN UNBOUNDED PRECEDING
                      AND UNBOUNDED FOLLOWING);
```

CODE EXPLAINED

- → `total - lag(total)` — growth from the last order
- → `lag(total, 2, 0)` — two rows back, 0 when missing
- → The full-partition frame makes `last_value` truly last
- → Without it, `last_value` ≈ the current row

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	all five functions
MySQL	all five (8.0+); `RESPECT NULLS` only
SQL Server	no `NTH_VALUE`; `IGNORE NULLS` for `FIRST/LAST_VALUE` (2022)
SQLite	all five (3.25+)

IN AN ORM

```sql
const rows = await prisma.$queryRaw`
  SELECT order_date, total,
         total - lag(total) OVER (ORDER BY order_date)
           AS change
  FROM   orders`;
```

FUNCTIONS

FUNCTION	RETURNS
`lag(x [, n [, d]])`	`x` from `n` rows before (default 1)
`lead(x [, n [, d]])`	`x` from `n` rows after
`first_value(x)`	`x` at the first row of the frame
`last_value(x)`	`x` at the last row of the frame
`nth_value(x, n)`	`x` at row `n` of the frame

LAG OUTPUT

ORDER_DATE	TOTAL	LAG	CHANGE
2026-10-01	40.00	NULL	NULL
2026-10-05	55.00	40.00	15.00
2026-10-09	30.00	55.00	-25.00
2026-10-12	30.00	30.00	0.00

⚠ COMMON MISTAKES

- `last_value` with the default frame
- No `PARTITION BY`, so one customer's lag reads another's order
- Ordering the window differently from the output

✓ TIPS & BEST PRACTICES

- Pass a default to `lag` / `lead` to avoid NULL math
- Use `lag(…) IS DISTINCT FROM x` to spot changes
- Percent change: `(x - lag(x)) / nullif(lag(x), 0)`

QUICK SUMMARY

- `lag` / `lead` — previous / next row
- `first_value` / `last_value` / `nth_value`
- Widen the frame for `last_value`

Running Totals & Moving Averages

SUM() OVER · running total · moving average · cumulative count · percent of total

CUMULATIVE AND SLIDING CALCULATIONS

Add `ORDER BY` to an aggregate's window and it becomes **cumulative**: `sum(x) OVER (ORDER BY d)` is a **running total**. Add a frame like `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` and it becomes a **moving** (sliding) calculation such as a 7-day average. Combined with `GROUP BY`, you can window over aggregates: `sum(sum(total)) OVER (…)` is a running total of daily totals. `sum(x) OVER ()` gives the grand total for **percent of total**.

RUNNING FACTS

- Default frame with `ORDER BY` includes **ties** (peers)
- Use `ROWS` for exact row-count windows
- Moving windows need one row per period: fill gaps first
- Aggregates inside windows: `sum(count(*)) OVER (…)`

NOSQL COMPARISON

- MongoDB: `window: { documents: ['unbounded', 'current'] }` → running total
- `window: { range: [-6, 0], unit: 'day' }` → 7-day window
- Inside `$setWindowFields` with `sortBy`

RUNNING & MOVING

```sql
-- Running total of revenue by day
SELECT order_date, sum(total) AS day_total,
       sum(sum(total)) OVER (ORDER BY order_date)
         AS running
FROM   orders
GROUP  BY order_date;

-- 7-row moving average of daily totals
SELECT order_date, day_total,
       avg(day_total) OVER (ORDER BY order_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM  (SELECT order_date, sum(total) AS day_total
       FROM orders GROUP BY order_date) d;

-- Cumulative count and percent of total
SELECT category, count(*) AS n,
       sum(count(*)) OVER (ORDER BY count(*) DESC) AS cum,
       round(100.0 * count(*) / sum(count(*)) OVER (), 1)
         AS pct
FROM   products
GROUP  BY category;
```

CODE EXPLAINED

- → `sum(sum(total))` — window over the grouped daily sums
- → One row per day, so no ties share a running value
- → `6 PRECEDING` + current row = 7 rows
- → `OVER ()` — grand total for the percentage

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`RANGE BETWEEN INTERVAL '6 days' PRECEDING …`
MySQL	`RANGE INTERVAL 6 DAY PRECEDING` supported
SQL Server	`RANGE` only with `UNBOUNDED` / `CURRENT ROW`
SQLite	`RANGE` with numeric offsets (3.28+)

IN AN ORM

```javascript
let run = 0;
const days = await prisma.order.groupBy({
  by: ['orderDate'], _sum: { total: true },
  orderBy: { orderDate: 'asc' },
});
const withRunning = days.map(d =>
  ({ ...d, running: (run += Number(d._sum.total)) }));
```

PATTERNS

NEED	WINDOW
running total	`sum(x) OVER (ORDER BY d)`
running total per group	`… OVER (PARTITION BY g ORDER BY d)`
moving average, 7 rows	`avg(x) OVER (… ROWS 6 PRECEDING)`
moving sum, 7 days	`… RANGE INTERVAL '6 days' PRECEDING`
percent of total	`x / sum(x) OVER ()`
percent of group	`x / sum(x) OVER (PARTITION BY g)`

EXAMPLE

DAY	TOTAL	RUNNING	MA3
1	10	10	10.0
2	20	30	15.0
3	30	60	20.0
4	0	60	16.7
5	50	110	26.7

⚠ COMMON MISTAKES

- Ties on the sort key making several rows share a running total
- Moving averages over data with missing days
- Integer percent math: `100 * a / b` truncating

✓ TIPS & BEST PRACTICES

- Fill missing days with a date spine first (sheet 34)
- Use `ROWS` unless you want ties grouped
- Round only the displayed result

QUICK SUMMARY

- `sum() OVER (ORDER BY …)` — running
- `ROWS n PRECEDING` — moving
- `OVER ()` — percent of total

Window Frames

ROWS · RANGE · GROUPS · UNBOUNDED PRECEDING · CURRENT ROW · N FOLLOWING · EXCLUDE

WHICH ROWS THE FUNCTION SEES

The **frame** is the slice of the partition an aggregate or value function looks at for the current row. **ROWS** counts physical rows; **RANGE** uses the distance between `ORDER BY` values, so rows with equal values (**peers**) move together; **GROUPS** counts groups of peers. With `ORDER BY` and no frame, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes the current row's peers. Without `ORDER BY` the frame is the whole partition.

FRAME FACTS

- Frames apply to aggregates and value functions only
- `lag`, `lead` and ranking functions ignore frames
- `RANGE n PRECEDING` needs one numeric or date sort key
- `EXCLUDE` drops the current row, its peers or ties

NOSQL COMPARISON

- MongoDB: `window: { documents: [-1, 1] }` = `ROWS 1 PRECEDING AND 1 FOLLOWING`
- `window: { range: [-1, 0] }` = `RANGE`
- Bounds: numbers, `'current'`, `'unbounded'`

FRAMES COMPARED

```
WITH s(d, v) AS (
  VALUES (1, 10), (2, 20), (2, 5), (3, 30))
SELECT d, v,
  sum(v) OVER (ORDER BY d) AS dflt,
  sum(v) OVER (ORDER BY d ROWS
    BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
                          AS rows_run,
  sum(v) OVER (ORDER BY d ROWS
    BETWEEN 1 PRECEDING AND 1 FOLLOWING)
                          AS nbr,
  sum(v) OVER (ORDER BY d RANGE
    BETWEEN 1 PRECEDING AND CURRENT ROW)
                          AS rng,
  sum(v) OVER (ORDER BY d GROUPS
    BETWEEN 1 PRECEDING AND CURRENT ROW)
                          AS grp,
  sum(v) OVER (ORDER BY d ROWS
    BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    EXCLUDE CURRENT ROW)   AS others
FROM s;
```

CODE EXPLAINED

→ The two `d = 2` rows are **peers**

→ `dflt` gives both peers 35; `rows_run` steps 30 → 35

→ `RANGE 1 PRECEDING` at `d = 3` covers `d` 2 to 3

→ `others` = everything except the current row

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`ROWS`, `RANGE` with offsets, `GROUPS`, `EXCLUDE`
MySQL	`ROWS`, `RANGE`; no `GROUPS` or `EXCLUDE`
SQL Server	`ROWS`; `RANGE` only `UNBOUNDED` / `CURRENT ROW`
SQLite	`ROWS`, `RANGE`, `GROUPS`, `EXCLUDE` (3.28+)

IN AN ORM

```
const rows = await prisma.$queryRaw`
  SELECT order_id, avg(total) OVER (ORDER BY order_id
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS ma
  FROM   orders`;
```

- Prisma has no window-function API: use `$queryRaw` or a TypedSQL file

OUTPUT (PEERS IN LISTED ORDER)

D	V	DFLT	ROWS_RUN	NBR	RNG	GRP	OTHERS
1	10	10	10	30	10	10	55
2	20	35	30	35	35	35	45
2	5	35	35	55	35	35	60
3	30	65	65	35	55	55	35

BOUNDS

BOUND	MEANS
`UNBOUNDED PRECEDING`	partition start
`n PRECEDING / FOLLOWING`	n units before / after
`CURRENT ROW`	this row (RANGE: and peers)
`UNBOUNDED FOLLOWING`	partition end

⚠ COMMON MISTAKES

- Relying on the default frame for `last_value` (sheet 46)
- Using `RANGE` and wondering why ties share a value
- SQL Server: `RANGE 6 PRECEDING` isn't supported

✓ TIPS & BEST PRACTICES

- Write the frame out in full when it matters
- `ROWS` is also faster than `RANGE` in SQL Server
- Test frames on a tiny `VALUES` table like this one

QUICK SUMMARY

- `ROWS` · `RANGE` · `GROUPS`
- Default with `ORDER BY`: start → current + peers
- `EXCLUDE` trims the frame

Window Function Patterns

top-N per group · deduplication · gaps & islands · sessionization · QUALIFY

CLASSIC PROBLEMS, SOLVED

Most window recipes follow one shape: **compute** a rank, lag or running value in a CTE or subquery, then **filter** it in the outer query, because window results can't be used in WHERE. **Top-N per group** keeps `row_number() <= n`; **deduplication** keeps `rn = 1`; **gaps and islands** finds runs of consecutive values by subtracting a row number; **sessionization** splits events wherever the gap since the previous one is too long. A few engines add **`QUALIFY`** to filter window results directly.

PATTERN FACTS

- Window results are filtered one level up
- Use `row_number` for exactly N, `dense_rank` to keep ties
- Islands: consecutive values share `value - row_number`
- `QUALIFY` isn't in PostgreSQL, MySQL, SQL Server or SQLite

NOSQL COMPARISON

- MongoDB 5.2+: `$topN` / `$bottomN` give top-N per group inside `$group`
- Or `$setWindowFields` → `$rank`, then `$match`

WINDOW PATTERNS

```sql
-- Top 2 products by price in each category
SELECT name, category, price
FROM  (SELECT p.*,
              row_number() OVER (PARTITION BY category
                                 ORDER BY price DESC) AS rn
       FROM   products p) t
WHERE  rn <= 2;

-- Gaps & islands: runs of consecutive order days
WITH d AS (SELECT DISTINCT order_date FROM orders),
     g AS (SELECT order_date,
                  order_date - (row_number()
                    OVER (ORDER BY order_date))::int AS grp
           FROM d)
SELECT min(order_date) AS run_start,
       max(order_date) AS run_end, count(*) AS days
FROM   g
GROUP  BY grp;

-- QUALIFY (DuckDB, Snowflake, BigQuery)
SELECT * FROM products
QUALIFY row_number() OVER (PARTITION BY category
                           ORDER BY price DESC) <= 2;
```

CODE EXPLAINED

→ `rn <= 2` in the outer query keeps two per category

→ Consecutive days minus 1, 2, 3 … give the same `grp` date

→ `::int` because `date - bigint` isn't defined

→ `QUALIFY` does the outer filter in one level

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	no `QUALIFY`; filter in a CTE or subquery
MySQL	no `QUALIFY`; delete via `JOIN` to a ranked table
SQL Server	no `QUALIFY`; `WITH d AS (…) DELETE FROM d WHERE rn > 1`
SQLite	no `QUALIFY`; same subquery pattern

IN AN ORM

```ts
const top2 = await prisma.$queryRaw`
  SELECT * FROM (
    SELECT p.*, row_number() OVER (
             PARTITION BY category ORDER BY price DESC) AS rn
    FROM products p) t
  WHERE rn <= 2`;
```

RECIPES

PROBLEM	APPROACH
top N per group	`row_number() OVER (PARTITION BY g ORDER BY x DESC)` → `rn <= N`
deduplicate	`row_number() OVER (PARTITION BY key ORDER BY newest)` → delete `rn > 1`
latest row per key	same as dedupe, keep `rn = 1`
gaps & islands	group by `value - row_number()`
sessions	flag `ts - lag(ts) > 30 min`, then running `sum` of flags
change detection	`x IS DISTINCT FROM lag(x)`

ISLANDS EXAMPLE

ORDER_DATE	ROW_NUMBER	DATE - RN
10-01	1	09-30
10-02	2	09-30
10-03	3	09-30
10-06	4	10-02
10-07	5	10-02
→		2 islands: 3 days, 2 days

⚠ COMMON MISTAKES

- Putting `row_number()` in `WHERE`
- Using `rank` for "exactly 2 per group" and getting 3 on ties
- Forgetting `DISTINCT` before gaps & islands on repeated dates

✓ TIPS & BEST PRACTICES

- Keep the ranked CTE: it doubles as a debugging view
- Index `(category, price DESC)` for top-N queries
- Check dedupe with `SELECT` before turning it into `DELETE`

QUICK SUMMARY

- Compute in a CTE, filter outside
- Top-N · dedupe · islands · sessions
- `QUALIFY` only in a few engines

INSERT

ADDING ROWS

`INSERT INTO table (columns) VALUES (…)` adds rows. Always list the **columns**: omitted ones get their **default** (or NULL), and the statement survives new columns being added later. One VALUES list can hold **many rows**, which is far faster than one statement per row. **`INSERT … SELECT`** copies rows produced by a query. **`RETURNING`** hands back generated values such as the new ID, so you don't need a second query.

INSERT FACTS

- Values match the column list by position
- `DEFAULT` uses the column default explicitly
- Identity `ALWAYS` columns can't be given a value
- A failed row fails the whole statement

NOSQL COMPARISON

- MongoDB: `insertOne(doc)` and `insertMany([docs])`
- `_id` is generated client-side, returned as `insertedId`
- No column list: each document carries its own fields

INSERT

```sql
INSERT INTO customers (name, email, city, country)
VALUES ('Ada Lovelace', 'ada@example.com', 'London', 'GB');

-- Several rows in one statement
INSERT INTO products (name, category, price, stock)
VALUES ('Desk Lamp', 'Lighting',   39.99,  25),
       ('Bookshelf', 'Furniture', 129.00,   4),
       ('Notebook',  'Stationery',  4.50, 300);

-- Column default, explicitly
INSERT INTO orders (customer_id, status, total)
VALUES (1, DEFAULT, 59.98);            -- 'pending'

-- Get generated values back
INSERT INTO orders (customer_id, total)
VALUES (1, 59.98)
RETURNING order_id, order_date;

-- Copy rows from a query (same columns)
INSERT INTO archived_orders (order_id, customer_id, total)
SELECT order_id, customer_id, total
FROM   orders
WHERE  order_date < DATE '2025-01-01';
```

CODE EXPLAINED

→ `customer_id` and `created_at` are filled by defaults

→ Three rows, one round trip, one transaction

→ `RETURNING` returns the identity value and default date

→ `INSERT … SELECT` has no `VALUES` keyword

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`RETURNING`; `DEFAULT VALUES`; `OVERRIDING SYSTEM VALUE`
MySQL	no `RETURNING` → `LAST_INSERT_ID()`; `INSERT IGNORE`
SQL Server	`OUTPUT inserted.order_id`; max 1,000 rows per `VALUES`
SQLite	`RETURNING` (3.35+); `INSERT OR IGNORE`

IN AN ORM

```sql
const order = await prisma.order.create({
  data: { customerId: 1, total: 59.98 },
});                                  // returns the row
const made = await prisma.product.createManyAndReturn({
  data: [{ name: 'Lamp', price: 39.99 }, /* … */],
});
```

INSERT FORMS

FORM	USE
`INSERT … VALUES (…)`	one row
`INSERT … VALUES (…), (…)`	many rows, one statement
`INSERT … SELECT …`	copy from a query
`INSERT … DEFAULT VALUES`	all defaults (PG, SQLite, SQL Server)
`INSERT … RETURNING *`	get the new rows back
`INSERT … ON CONFLICT`	upsert (sheet 53)

OMITTED COLUMNS GET…

COLUMN HAS	VALUE
`a DEFAULT`	the default
`an identity / serial`	the next number
`neither, nullable`	NULL
`neither, NOT NULL`	error
`GENERATED … STORED`	computed; can't insert

⚠ COMMON MISTAKES

- `INSERT INTO t VALUES (…)` without columns breaks when a column is added
- Inserting in a loop: thousands of round trips
- Sending numbers as text and relying on implicit casts

✓ TIPS & BEST PRACTICES

- Batch inserts: hundreds to a few thousand rows per statement
- Use parameters, never string-built values (sheet 76)
- For files and huge loads, use `COPY` (sheet 54)

QUICK SUMMARY

- `INSERT INTO t (cols) VALUES (…)`
- Multi-row `VALUES` · `INSERT … SELECT`
- `RETURNING` for generated values

UPDATE

CHANGING ROWS

`UPDATE table SET col = expr WHERE …` changes every row the `WHERE` matches, and **every row in the table** if you leave `WHERE` off. Expressions on the right see the **old** values, so `SET stock = stock - 1` works and two columns can swap. To update from another table, PostgreSQL and SQLite use **`UPDATE … FROM`**, MySQL and SQL Server join in the update. **`RETURNING`** shows the changed rows.

UPDATE FACTS

- No `WHERE` = every row
- All `SET` expressions read the old row
- The result reports how many rows changed
- Run the `WHERE` as a `SELECT` first

NOSQL COMPARISON

- MongoDB: `updateMany({ category: 'Books' }, { $mul: { price: 1.05 } })`
- `$set`, `$inc`, `$unset` change single fields
- `findOneAndUpdate` returns the document, like `RETURNING`

UPDATE

```sql
UPDATE products
SET    price = price * 1.05
WHERE  category = 'Books';

UPDATE products
SET    price = 24.99,
       stock = stock + 10
WHERE  product_id = 7
RETURNING product_id, price, stock;

-- Update from another table (PostgreSQL, SQLite)
UPDATE orders o
SET    total = s.amt
FROM  (SELECT order_id, sum(quantity * unit_price) AS amt
       FROM   order_items
       GROUP  BY order_id) s
WHERE  s.order_id = o.order_id;

-- Portable: correlated subquery
UPDATE orders o
SET    total = (SELECT sum(quantity * unit_price)
                FROM   order_items i
                WHERE  i.order_id = o.order_id);
```

CODE EXPLAINED

- → `price * 1.05` reads each row's current price
- → `RETURNING` lists the rows after the change
- → `FROM` joins the pre-aggregated item totals
- → The subquery form sets NULL for orders with no items

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`UPDATE … FROM`; `RETURNING` (PG 18: `OLD` / `NEW`)
MySQL	`UPDATE orders o JOIN s ON … SET …`; `LIMIT` allowed
SQL Server	`UPDATE o SET … FROM orders o JOIN …`; `OUTPUT`
SQLite	`UPDATE … FROM` (3.33+); `RETURNING` (3.35+)

IN AN ORM

```prisma
await prisma.product.updateMany({
  where: { category: 'Books' },
  data: { price: { multiply: 1.05 } },
});                              // → { count: 12 }
await prisma.product.update({
  where: { id: 7 }, data: { price: 24.99 },
});
```

SET FORMS

WRITE	MEANS
`SET a = 1`	constant
`SET a = a + 1`	based on the old value
`SET (a, b) = (1, 2)`	row form (PG, SQLite)
`SET a = DEFAULT`	reset to the default
`SET a = (SELECT …)`	from a scalar subquery
`SET a = CASE … END`	different value per row (sheet 21)

SAFE UPDATES

STEP	HOW
preview	same `WHERE` in a `SELECT count(*)`
isolate	`BEGIN;` … check … `COMMIT;` (sheet 68)
confirm	`RETURNING` / `OUTPUT` the rows
guard	MySQL `--safe-updates` blocks keyless updates
undo	`ROLLBACK;` before you commit

⚠ COMMON MISTAKES

- Forgetting `WHERE` and updating every row
- Expecting `SET a = b, b = a` to fail (it swaps)
- Correlated update setting NULL where no source row exists

✓ TIPS & BEST PRACTICES

- Wrap risky updates in a transaction and check the count
- Update in batches on huge tables to keep locks short
- Add `AND col IS DISTINCT FROM new` to skip no-op writes

QUICK SUMMARY

- `UPDATE t SET … WHERE …`
- Right side sees old values
- `FROM` / `JOIN` to use other tables

DELETE & TRUNCATE

DELETE · DELETE … USING · TRUNCATE · RESTART IDENTITY · CASCADE · soft delete

REMOVING ROWS

`DELETE FROM table WHERE …` removes matching rows one by one: triggers fire, foreign keys are checked, and it can be rolled back. **`TRUNCATE`** empties a whole table at once, much faster, and can reset identity counters; it has no `WHERE`. Many apps prefer a **soft delete**: set a `deleted_at` timestamp and filter it out, so history and references survive. Removing the table itself is `DROP TABLE` (sheet 59).

DELETE FACTS

- No `WHERE` = every row
- FK `ON DELETE CASCADE` deletes child rows too (sheet 56)
- Deleted space is reused later, not returned at once
- `TRUNCATE` skips row-level triggers

NOSQL COMPARISON

- MongoDB: `deleteMany(filter)`; `drop()` removes the collection
- Soft delete: `{ $set: { deletedAt: new Date() } }`
- A TTL index deletes expired documents automatically

DELETE & TRUNCATE

```sql
DELETE FROM orders
WHERE  status = 'cancelled'
  AND  order_date < DATE '2026-01-01'
RETURNING order_id;

-- Delete using another table (PostgreSQL)
DELETE FROM order_items i
USING  products p
WHERE  p.product_id = i.product_id
  AND  p.category = 'Discontinued';

-- Portable form
DELETE FROM order_items
WHERE  product_id IN (SELECT product_id FROM products
                      WHERE  category = 'Discontinued');

-- Empty tables fast
TRUNCATE order_items, orders RESTART IDENTITY;
TRUNCATE customers CASCADE;  -- + tables referencing it

-- Soft delete: hide instead of destroy
ALTER TABLE customers ADD COLUMN deleted_at TIMESTAMPTZ;
UPDATE customers SET deleted_at = now()
WHERE  customer_id = 42;
SELECT * FROM customers WHERE deleted_at IS NULL;
```

CODE EXPLAINED

- → `RETURNING` lists exactly what was removed
- → `USING` joins `products` to choose which items go
- → `RESTART IDENTITY` resets the ID counters to 1
- → Soft-deleted rows stay; queries filter them out

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`DELETE … USING`; `TRUNCATE` can roll back
MySQL	`DELETE i FROM order_items i JOIN …`; `TRUNCATE` auto-commits
SQL Server	`DELETE i FROM order_items i JOIN …`; no `TRUNCATE … CASCADE`
SQLite	no `TRUNCATE`: `DELETE FROM t` is optimized

IN AN ORM

```ts
await prisma.order.deleteMany({
  where: {
    status: 'cancelled',
    orderDate: { lt: new Date('2026-01-01') },
  },
});
```

- Soft delete: a `$extends` query extension rewrites deletes into updates

DELETE VS TRUNCATE VS DROP

	DELETE	TRUNCATE	DROP TABLE
removes	matching rows	all rows	table + data
WHERE	yes	no	no
row triggers	fire	don't fire	—
speed on big tables	slow	fast	fast
identity reset	no	optional	—
rollback	yes	PG, SQL Server: yes	PG, SQL Server: yes

SOFT DELETE CHECKLIST

ITEM	DO
column	`deleted_at TIMESTAMPTZ NULL`
reads	`WHERE deleted_at IS NULL` everywhere
uniqueness	partial unique index `WHERE deleted_at IS NULL`
convenience	a view of live rows (sheet 64)
purge	hard-delete old rows on a schedule

⚠ COMMON MISTAKES

- `DELETE` without `WHERE` (or with the wrong one)
- Deleting millions of rows in one transaction
- Soft delete without updating unique constraints

✓ TIPS & BEST PRACTICES

- Delete in batches: `WHERE id IN (SELECT … LIMIT 10000)`
- Preview with `SELECT count(*)` and the same `WHERE`
- Use `TRUNCATE` for staging tables between loads

QUICK SUMMARY

- `DELETE … WHERE` — some rows
- `TRUNCATE` — all rows, fast
- Soft delete — `deleted_at`

Upsert & MERGE

ON CONFLICT · DO UPDATE · DO NOTHING · EXCLUDED · ON DUPLICATE KEY UPDATE · MERGE

INSERT OR UPDATE IN ONE STEP

An **upsert** inserts a row, or updates it if one with the same key already exists, in **one atomic statement**, so two concurrent requests can't both insert. PostgreSQL and SQLite write **`INSERT … ON CONFLICT (key) DO UPDATE`**, with the rejected row available as **`EXCLUDED`**; `DO NOTHING` just skips duplicates. MySQL uses **`ON DUPLICATE KEY UPDATE`**. **`MERGE`** is the standard, more general form: compare a source with a target and insert, update or delete per row.

UPSERT FACTS

- The conflict target needs a unique index or PK
- `EXCLUDED.col` = the value you tried to insert
- Add `WHERE` to `DO UPDATE` to update conditionally
- Select-then-insert in app code races; upsert doesn't

NOSQL COMPARISON

- MongoDB: `updateOne(filter, { $set: … }, { upsert: true })`
- `$setOnInsert` sets fields only when inserting
- Needs a unique index on the filter field to stay safe

UPSERT & MERGE

```sql
-- Insert, or update if the email exists
INSERT INTO customers (email, name, city)
VALUES ('ada@example.com', 'Ada Lovelace', 'London')
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name,
              city = EXCLUDED.city;

-- Insert only if missing
INSERT INTO customers (email, name)
VALUES ('bo@example.com', 'Bo')
ON CONFLICT (email) DO NOTHING;
-- Add to an existing quantity
INSERT INTO order_items
       (order_id, product_id, quantity, unit_price)
VALUES (101, 7, 1, 39.99)
ON CONFLICT (order_id, product_id) DO UPDATE
SET    quantity = order_items.quantity + EXCLUDED.quantity;
-- MERGE (PostgreSQL 15+, SQL Server)
MERGE INTO products p
USING staging_prices s ON s.product_id = p.product_id
WHEN MATCHED AND s.price IS NULL THEN DELETE
WHEN MATCHED THEN UPDATE SET price = s.price
WHEN NOT MATCHED THEN
  INSERT (name, price) VALUES (s.name, s.price);
```

CODE EXPLAINED

- → `ON CONFLICT (email)` relies on the `UNIQUE` email
- → `order_items.quantity` = stored value, `EXCLUDED` = new one
- → `WHEN` branches run in order; first match wins
- → `staging_prices` — a loaded table of new prices

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`ON CONFLICT`; `MERGE` (15+) with `RETURNING` (17+)
MySQL	`VALUES (…) AS new ON DUPLICATE KEY UPDATE name = new.name`
SQL Server	`MERGE … ;` (semicolon required); use `HOLDLOCK`
SQLite	`ON CONFLICT … DO UPDATE` (3.24+) with `excluded.`

IN AN ORM

```ts
await prisma.customer.upsert({
  where:  { email: 'ada@example.com' },
  create: { email: 'ada@example.com', name: 'Ada' },
  update: { name: 'Ada' },
});
```

ON CONFLICT PARTS

PART	DOES
`ON CONFLICT (cols)`	which unique key to check
`ON CONFLICT ON CONSTRAINT c`	name the constraint instead
`DO NOTHING`	skip the row
`DO UPDATE SET …`	update the existing row
`EXCLUDED.col`	the proposed new value
`DO UPDATE … WHERE …`	only update when it's newer

MERGE CLAUSES

CLAUSE	ACTION
`WHEN MATCHED THEN`	`UPDATE` or `DELETE`
`WHEN NOT MATCHED THEN`	`INSERT`
`WHEN NOT MATCHED BY SOURCE`	act on target-only rows (SQL Server, PG 17+)
`AND condition`	extra test per branch
`first match wins`	order branches from specific to general

⚠ COMMON MISTAKES

- Upserting on a column without a unique index
- `SELECT` then `INSERT` from app code: race conditions
- SQL Server `MERGE` without `HOLDLOCK` under concurrency

✓ TIPS & BEST PRACTICES

- Use `DO NOTHING` + `RETURNING` to learn which rows were new
- Prefer `ON CONFLICT` for single-row upserts in PostgreSQL
- Use `MERGE` for syncing a staging table into a target

QUICK SUMMARY

- `ON CONFLICT … DO UPDATE / NOTHING`
- `ON DUPLICATE KEY UPDATE` (MySQL)
- `MERGE` — standard sync

Bulk Loading & Export

MOVING DATA IN AND OUT IN BULK

For thousands or millions of rows, use the engine's **bulk path** instead of INSERT: PostgreSQL **COPY**, MySQL **LOAD DATA**, SQL Server **BULK INSERT** / bcp, SQLite's **.import**. They stream rows with far less overhead. Watch **where the file lives**: server-side commands read files on the database server, while client-side ones (`\copy`, `LOAD DATA LOCAL`) read files on your machine. Most of them also export.

BULK FACTS

- Server-side file access needs extra privileges
- `\copy` is a psql command: one line, no `;`
- Load into a staging table, validate, then insert
- CSV quirks: header row, quotes, NULL marker

NOSQL COMPARISON

- `mongoimport --type csv --headerline -d shop -c products products.csv`
- `mongoexport` writes JSON or CSV
- `insertMany(docs, { ordered: false })` for app-side batches

LOAD & EXPORT

```
-- PostgreSQL: file on the server
COPY products (name, category, price, stock)
FROM '/srv/import/products.csv'
WITH (FORMAT csv, HEADER true);

-- psql: file on your machine (one line, no ;)
\copy products FROM 'products.csv' CSV HEADER
\copy (SELECT * FROM orders) TO 'orders.csv' CSV HEADER

-- MySQL
LOAD DATA LOCAL INFILE 'products.csv'
INTO TABLE products
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
IGNORE 1 LINES (name, category, price, stock);

-- SQL Server
BULK INSERT products FROM 'C:\data\products.csv'
WITH (FORMAT = 'CSV', FIRSTROW = 2);

-- SQLite shell
.import --csv --skip 1 products.csv products
```

CODE EXPLAINED

- → `HEADER true` skips the first line on input
- → `\copy` streams through the client connection
- → `IGNORE 1 LINES` — MySQL's header skip
- → `--skip 1` — skip the header in the SQLite shell

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`COPY` (server) / `\copy` (client); csv, text, binary
MySQL	`LOAD DATA [LOCAL]`; `SELECT … INTO OUTFILE`; `local_infile=1`
SQL Server	`BULK INSERT`, bcp, `OPENROWSET(BULK …)`
SQLite	`.import --csv`; export with `.mode csv` + `.once`

IN AN ORM

```
// createMany sends multi-row INSERTs
await prisma.product.createMany({
  data: rows,             // parsed from the CSV
  skipDuplicates: true,   // ON CONFLICT DO NOTHING
});
```

FASTER LOADS

TECHNIQUE	WHY
load into a staging table	validate before touching real data
drop / recreate indexes	build each index once, not per row
one transaction per batch	fewer commits
UNLOGGED table (PG)	skips WAL for scratch data
disable triggers (carefully)	no per-row work
ANALYZE afterwards	fresh statistics (sheet 74)

EXPORT

ENGINE	COMMAND
PostgreSQL	`\copy (query) TO 'f.csv' CSV HEADER`
MySQL	`SELECT … INTO OUTFILE '/tmp/f.csv'`
SQL Server	`bcp "query" queryout f.csv -c -t,`
SQLite	`.headers on .mode csv .once f.csv`

⚠ COMMON MISTAKES

- Using server-side `COPY` with a path on your laptop
- Loading straight into production tables with bad rows
- Forgetting `ANALYZE` after a big load

✓ TIPS & BEST PRACTICES

- Keep loads idempotent: truncate staging, reload, merge (sheet 53)
- Match column order explicitly in the command
- Use `NULL ''` / `NULL AS` options for empty fields

QUICK SUMMARY

- `COPY` / `\copy` · `LOAD DATA` · `BULK INSERT` · `.import`
- Server file vs client file
- Staging table, then merge

CREATE TABLE

CREATE TABLE · column definitions · DEFAULT · IF NOT EXISTS · CREATE TABLE AS · TEMPORARY

DEFINING A TABLE

CREATE TABLE names a table and lists its **columns**, each with a type and optional rules: NOT NULL, a **DEFAULT**, keys and checks. **IF NOT EXISTS** makes scripts safe to re-run. **CREATE TABLE … AS SELECT** builds and fills a table from a query, but copies no keys, defaults or indexes. **Temporary** tables live for one session, which is handy for scratch work.

TABLE FACTS

- Constraints can be per column or listed at the end
- `DEFAULT` applies only when the column is omitted
- `AS SELECT` copies data, not constraints
- Generated columns compute from other columns

NOSQL COMPARISON

- MongoDB creates a collection on first insert
- `createCollection('reviews', { validator: { $jsonSchema: … } })` adds rules
- Without a validator, any document shape is accepted

CREATE TABLE

```sql
CREATE TABLE IF NOT EXISTS reviews (
  review_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id  INT NOT NULL REFERENCES products,
  customer_id INT NOT NULL REFERENCES customers,
  rating      SMALLINT NOT NULL
              CHECK (rating BETWEEN 1 AND 5),
  body        TEXT,
  body_len    INT GENERATED ALWAYS AS
              (length(body)) STORED,
  is_public   BOOLEAN NOT NULL DEFAULT true,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Structure + data from a query
CREATE TABLE big_orders AS
SELECT * FROM orders WHERE total > 1000;

-- Same columns, defaults, indexes; no rows
CREATE TABLE orders_2027 (LIKE orders INCLUDING ALL);

-- Scratch table for this session only
CREATE TEMP TABLE cart (product_id INT, qty INT);
```

CODE EXPLAINED

- → `REFERENCES products` points at its primary key
- → `GENERATED … STORED` — computed on write, read-only
- → `DEFAULT now()` fills the time when `created_at` is omitted
- → `LIKE … INCLUDING ALL` copies structure, not data

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`IF NOT EXISTS`, `LIKE … INCLUDING ALL`, `UNLOGGED`
MySQL	`ENGINE=InnoDB`; `CREATE TABLE t LIKE s`
SQL Server	no `IF NOT EXISTS`; `SELECT … INTO new_t`; `#temp`
SQLite	`STRICT` tables (3.37+); `WITHOUT ROWID`

IN AN ORM

```
model Review {
  id        Int      @id @default(autoincrement())
  rating    Int      @db.SmallInt
  body      String?
  createdAt DateTime @default(now())
}
// prisma migrate dev → CREATE TABLE "Review" (…)
```

COLUMN DEFINITION

PART	EXAMPLE
name + type	`rating SMALLINT`
nullability	`NOT NULL`
default	`DEFAULT 0` · `DEFAULT now()`
inline key	`PRIMARY KEY` · `REFERENCES t`
inline check	`CHECK (rating BETWEEN 1 AND 5)`
generated	`GENERATED ALWAYS AS (expr) STORED`

WAYS TO CREATE

STATEMENT	COPIES
`CREATE TABLE t (…)`	what you define
`… IF NOT EXISTS`	nothing if `t` exists
`CREATE TABLE t AS SELECT`	columns + rows only
`CREATE TABLE t (LIKE s …)`	structure (PG)
`CREATE TEMP TABLE`	session-only table
`CREATE UNLOGGED TABLE`	fast, not crash-safe (PG)

⚠ COMMON MISTAKES

- Expecting `CREATE TABLE AS` to copy keys and indexes
- `IF NOT EXISTS` hiding a table with a different definition
- Forgetting `NOT NULL` and getting half-empty rows

✓ TIPS & BEST PRACTICES

- Make columns `NOT NULL` unless NULL means something
- Keep DDL in versioned migration files
- Name tables plural `snake_case` (sheet 63)

QUICK SUMMARY

- `CREATE TABLE t (col type rules, …)`
- `DEFAULT` · `NOT NULL` · generated
- `AS SELECT` · `LIKE` · `TEMP`

Primary & Foreign Keys

PRIMARY KEY · composite key · FOREIGN KEY · REFERENCES · ON DELETE CASCADE · SET NULL

IDENTITY AND RELATIONSHIPS

A **primary key** uniquely identifies each row and is never NULL; a table has one, made of one column or several (**composite**). A **foreign key** says "this column's value must exist as a key in that table", so orders can't point at missing customers. **`ON DELETE`** decides what happens to child rows when the parent goes: block it, **`CASCADE`** the delete, or **`SET NULL`**.

KEY FACTS

- PK = `UNIQUE` + `NOT NULL`, plus an index
- FK columns allow NULL unless you forbid it
- Default action: `NO ACTION` (blocks the delete)
- Index FK columns yourself in PG and SQL Server

NOSQL COMPARISON

- MongoDB `_id` is the primary key: unique and immutable
- No foreign keys: references are plain values
- Orphans are possible; the app (or embedding) prevents them

KEYS

```sql
CREATE TABLE order_items (
  order_id   INT NOT NULL,
  product_id INT NOT NULL,
  quantity   INT NOT NULL,
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (order_id) REFERENCES orders
    ON DELETE CASCADE,       -- items die with order
  FOREIGN KEY (product_id) REFERENCES products
    ON DELETE RESTRICT       -- keep sold products
);

-- Column shorthand
CREATE TABLE employees (
  emp_id     INT PRIMARY KEY,
  dept_id    INT REFERENCES departments
               ON DELETE SET NULL,
  manager_id INT REFERENCES employees (emp_id)
);

-- Named constraint, added later
ALTER TABLE orders
  ADD CONSTRAINT orders_customer_fk
  FOREIGN KEY (customer_id) REFERENCES customers;

INSERT INTO orders (customer_id, total) VALUES (99999, 10);
-- ERROR: … violates foreign key constraint
--        "orders_customer_fk"
```

CODE EXPLAINED

- → `(order_id, product_id)` — each product once per order
- → `CASCADE` deletes an order's items with the order
- → `SET NULL` keeps employees when a department is removed
- → `manager_id` references the same table (self-reference)

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	FKs aren't indexed automatically; `DEFERRABLE` (sheet 57)
MySQL	InnoDB indexes FK columns automatically
SQL Server	no auto FK index; no multiple cascade paths
SQLite	FKs ignored until `PRAGMA foreign_keys = ON`

IN AN ORM

```ts
await prisma.orderItem.create({
  data: {
    order:   { connect: { id: 101 } },  // FK must exist
    product: { connect: { id: 7 } },
    quantity: 2, unitPrice: 39.99,
  },
});
```

- Schema: `onDelete: Cascade` in `@relation`; composite PK `@@id([orderId, productId])`

ON DELETE / ON UPDATE

ACTION	CHILD ROWS WHEN PARENT CHANGES
`NO ACTION (default)`	error, checked at end of statement
`RESTRICT`	error, checked immediately
`CASCADE`	deleted / updated too
`SET NULL`	FK set to NULL
`SET DEFAULT`	FK set to its default

KEY KINDS

KIND	EXAMPLE
`single-column PK`	`customer_id`
`composite PK`	`(order_id, product_id)`
`foreign key`	`orders.customer_id → customers`
`self-reference`	`employees.manager_id → employees`
`surrogate vs natural`	ID number vs real-world value (sheet 63)

⚠ COMMON MISTAKES

- No index on FK columns: slow joins and cascades
- `CASCADE` on a parent you didn't expect to wipe children
- SQLite without `PRAGMA foreign_keys = ON`

✓ TIPS & BEST PRACTICES

- Name constraints (`orders_customer_fk`) for clear errors
- Use `RESTRICT` for history you must keep
- Load parents before children in data imports

QUICK SUMMARY

- PK — unique, not NULL, one per table
- FK — value must exist in the parent
- `ON DELETE` — restrict, cascade, set null

Constraints

RULES THE DATABASE ENFORCES

Constraints reject bad data no matter which app or script writes it. **NOT NULL** requires a value; **UNIQUE** forbids duplicates (NULLs usually don't count); **CHECK** requires a condition to be true (or NULL); keys link and identify rows (sheet 56). Name constraints with **CONSTRAINT name** so errors are readable. PostgreSQL adds **DEFERRABLE** (check at commit, not per statement) and **EXCLUDE** (no two rows may overlap, such as double-booked rooms).

CONSTRAINT FACTS

- `CHECK` passes when the expression is NULL
- `UNIQUE` creates an index behind the scenes
- A unique **index** on an expression handles case-insensitive rules
- Adding a constraint validates existing rows

NOSQL COMPARISON

- MongoDB: unique indexes enforce `UNIQUE`
- `$jsonSchema` validators cover `required`, `minimum`, `maximum`, `enum`
- Validation can warn instead of reject (`validationAction`)

CONSTRAINTS

```
CREATE TABLE coupons (
  code       VARCHAR(20)  NOT NULL,
  percent    NUMERIC(5,2) NOT NULL,
  starts_on DATE NOT NULL,
  ends_on    DATE,
  CONSTRAINT coupons_code_uq UNIQUE (code),
  CONSTRAINT coupons_pct_chk
    CHECK (percent > 0 AND percent <= 100),
  CONSTRAINT coupons_dates_chk
    CHECK (ends_on IS NULL OR ends_on > starts_on)
);

-- Treat NULLs as equal (PostgreSQL 15+)
ALTER TABLE customers ADD CONSTRAINT email_uq
  UNIQUE NULLS NOT DISTINCT (email);

-- No overlapping bookings per room
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE bookings (
  room   INT,
  during TSTZRANGE,
  EXCLUDE USING gist (room WITH =, during WITH &&)
);
```

CODE EXPLAINED

→ Named checks show up in error messages by name

→ `ends_on IS NULL OR …` allows open-ended coupons

→ `NULLS NOT DISTINCT` allows only one NULL email

→ `&&` means "ranges overlap"; `EXCLUDE` forbids it per room

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	all of them; `NULLS NOT DISTINCT` (15+); `EXCLUDE`
MySQL	`CHECK` enforced since 8.0.16; no `DEFERRABLE`
SQL Server	`UNIQUE` allows one NULL; `WITH NOCHECK` skips old rows
SQLite	`CHECK`, `UNIQUE`, `NOT NULL`; deferrable FKs only

IN AN ORM

```
model Coupon {
  code     String   @unique
  percent  Decimal  @db.Decimal(5, 2)
  startsOn DateTime @db.Date
}
// CHECK constraints: add them to the migration .sql
```

CONSTRAINT TYPES

CONSTRAINT	REJECTS
`NOT NULL`	missing values
`UNIQUE (cols)`	duplicate values or combinations
`CHECK (expr)`	rows where `expr` is false
`PRIMARY KEY`	duplicates and NULLs
`FOREIGN KEY`	references to missing rows
`EXCLUDE (PG)`	rows that conflict by any operator

TIMING & NAMING

OPTION	MEANING
`NOT DEFERRABLE`	checked per statement (default)
`DEFERRABLE INITIALLY DEFERRED`	checked at `COMMIT`
`NOT VALID (PG)`	skip existing rows; `VALIDATE` later
`PG default names`	`orders_pkey`, `orders_customer_id_fkey`
`suggested style`	`<table>_<cols>_<pk|fk|uq|chk>`

⚠ COMMON MISTAKES

- Checking rules only in app code
- Expecting `CHECK (x > 0)` to reject NULL
- Adding a constraint to a huge table at peak time

✓ TIPS & BEST PRACTICES

- Case-insensitive unique: `UNIQUE INDEX … (lower(email))`
- Add big constraints as `NOT VALID`, then `VALIDATE` (PG)
- Use `DEFERRABLE` for circular or swap-style updates

QUICK SUMMARY

- `NOT NULL` · `UNIQUE` · `CHECK`
- Name them: `CONSTRAINT x …`
- `DEFERRABLE` · `EXCLUDE` (PG)

Auto-Increment & Sequences

GENERATING KEY VALUES

An **identity column** fills itself with the next number from a counter. The standard form is **`GENERATED ALWAYS AS IDENTITY`** (no manual values) or **`BY DEFAULT`** (manual values allowed). PostgreSQL's older **`SERIAL`** is a shortcut for an integer plus a **sequence**; MySQL uses **`AUTO_INCREMENT`** and SQL Server **`IDENTITY(1,1)`**. A **sequence** is a standalone counter you call with `nextval`. **UUIDs** are the alternative when IDs must be created anywhere.

SEQUENCE FACTS

- Numbers are never reused, so **gaps are normal**
- Rolled-back inserts still consume a number
- After loading explicit IDs, reset the counter
- Prefer identity over `SERIAL` in new PostgreSQL code

NOSQL COMPARISON

- MongoDB uses `ObjectId` by default; no auto-increment
- Sequential numbers: a counter document updated with `findOneAndUpdate({ $inc: { seq: 1 } })`

IDENTITY & SEQUENCES

```
CREATE TABLE invoices (
  id         BIGINT GENERATED ALWAYS AS IDENTITY
             PRIMARY KEY,
  number     INT GENERATED BY DEFAULT AS IDENTITY
             (START WITH 1000),
  public_id UUID NOT NULL DEFAULT gen_random_uuid()
);
INSERT INTO invoices DEFAULT VALUES RETURNING *;
INSERT INTO invoices (id) VALUES (5);
-- ERROR: cannot insert a non-DEFAULT value into "id"
INSERT INTO invoices (id) OVERRIDING SYSTEM VALUE
VALUES (5);

-- Older style: integer + sequence + default
CREATE TABLE legacy (id SERIAL PRIMARY KEY);

-- Standalone sequence
CREATE SEQUENCE ticket_seq START 1;
SELECT nextval('ticket_seq');   -- 1
SELECT currval('ticket_seq');   -- 1 (this session)

-- Resync after loading explicit ids
SELECT setval(pg_get_serial_sequence('orders',
                                     'order_id'),
              (SELECT max(order_id) FROM orders));
```

CODE EXPLAINED

- → `ALWAYS` rejects manual ids; `BY DEFAULT` accepts them
- → `START WITH 1000` — invoice numbers begin at 1000
- → `currval` is per session: other sessions don't affect it
- → `setval` moves the counter past existing rows

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`GENERATED … AS IDENTITY`, `SERIAL`, sequences
MySQL	`AUTO_INCREMENT`; `LAST_INSERT_ID()`; no sequences
SQL Server	`IDENTITY(1,1)`, `CREATE SEQUENCE`, `NEWSEQUENTIALID()`
SQLite	`INTEGER PRIMARY KEY` = rowid; `AUTOINCREMENT` stops reuse

IN AN ORM

```
model Invoice {
  id       BigInt @id @default(autoincrement())
  publicId String @default(uuid(7)) @db.Uuid
}
// autoincrement() → SERIAL / AUTO_INCREMENT / IDENTITY
```

IDENTITY OPTIONS

OPTION	DOES
`ALWAYS`	only the database sets the value
`BY DEFAULT`	database sets it unless you do
`START WITH n`	first value
`INCREMENT BY n`	step (can be negative)
`CACHE n`	pre-allocate n values (faster, bigger gaps)
`CYCLE`	wrap around at the max (rarely wanted)

INTEGER VS UUID KEYS

	IDENTITY	UUID V4	UUID V7
`size`	4–8 bytes	16 bytes	16 bytes
`insert locality`	sequential	random	time-ordered
`guessable`	yes	no	time is visible
`made by app`	no	yes	yes

⚠ COMMON MISTAKES

- Assuming IDs have no gaps (invoice audits!)
- Bulk-loading ids without `setval` : duplicate key errors later
- Exposing sequential ids in URLs when they shouldn't be guessable

✓ TIPS & BEST PRACTICES

- Need gap-free numbers? Use a counter table with row locking
- Use `BIGINT` identities for large tables
- Expose a UUID publicly, keep the integer internally

QUICK SUMMARY

- `GENERATED … AS IDENTITY` (standard)
- `SERIAL` · `AUTO_INCREMENT` · `IDENTITY(1,1)`
- Sequences: `nextval` / `setval`

ALTER & DROP

ALTER TABLE · ADD COLUMN · DROP COLUMN · RENAME · ALTER COLUMN TYPE · ADD CONSTRAINT · DROP TABLE

CHANGING STRUCTURE

ALTER TABLE changes an existing table: add, drop or rename columns, change types and defaults, and add or remove constraints. **DROP TABLE** removes a table with its data and indexes; it fails if other objects depend on it unless you add **CASCADE**. On busy production tables, the question is not only *what* to change but **how long it locks**: some changes are instant, others rewrite the whole table.

ALTER FACTS

- PostgreSQL DDL is transactional: it can roll back
- MySQL and Oracle DDL commit immediately
- `IF EXISTS` keeps drop scripts re-runnable
- Renames break code that uses the old name

NOSQL COMPARISON

- MongoDB has no table structure to alter
- New fields simply appear; backfill with `updateMany({}, { $set: { pts: 0 } })`
- `$rename` renames a field; `$unset` removes it

ALTER & DROP

```
ALTER TABLE customers ADD COLUMN phone VARCHAR(20);
ALTER TABLE customers
  ADD COLUMN loyalty_pts INT NOT NULL DEFAULT 0;
ALTER TABLE customers DROP COLUMN phone;
ALTER TABLE customers RENAME COLUMN city TO town;
ALTER TABLE customers RENAME TO clients;

ALTER TABLE products
  ALTER COLUMN name TYPE VARCHAR(200);
ALTER TABLE products
  ALTER COLUMN stock TYPE BIGINT USING stock::bigint;
ALTER TABLE products ALTER COLUMN category SET NOT NULL;
ALTER TABLE products ALTER COLUMN stock SET DEFAULT 0;

ALTER TABLE products
  ADD CONSTRAINT price_pos CHECK (price > 0);
ALTER TABLE products DROP CONSTRAINT price_pos;

DROP TABLE IF EXISTS big_orders;
DROP TABLE customers;
-- ERROR: other objects depend on it (orders FK)
DROP TABLE customers CASCADE;  -- also drops the FK
```

CODE EXPLAINED

- → A constant `DEFAULT` with `ADD COLUMN` is instant in PG 11+
- → `USING` tells PG how to convert existing values
- → `SET NOT NULL` scans the table to check existing rows
- → `CASCADE` removes dependent constraints and views

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	transactional DDL; `ALTER COLUMN … TYPE … USING`
MySQL	`MODIFY` / `CHANGE COLUMN`; `ALGORITHM=INSTANT`
SQL Server	`ALTER COLUMN c VARCHAR(200) NOT NULL`; `sp_rename`
SQLite	only add / drop / rename column; else rebuild table

IN AN ORM

```
model Customer {
  phone String? @db.VarChar(20)   // new field
}
// npx prisma migrate dev --name add_phone
// → ALTER TABLE "customers" ADD COLUMN "phone" …
```

ALTER ACTIONS (PG)

ACTION	SYNTAX
add column	`ADD COLUMN c type …`
drop column	`DROP COLUMN c`
rename	`RENAME COLUMN a TO b` · `RENAME TO t2`
change type	`ALTER COLUMN c TYPE t USING …`
null / default	`ALTER COLUMN c SET NOT NULL` · `SET DEFAULT x`
constraint	`ADD CONSTRAINT n …` · `DROP CONSTRAINT n`

LOCK IMPACT (PG)

CHANGE	COST
add nullable column	instant
add column + constant default	instant (11+)
change column type	table rewrite
SET NOT NULL	full scan
add FK / CHECK	use `NOT VALID`, then `VALIDATE`
new index	`CREATE INDEX CONCURRENTLY` (sheet 71)

⚠ COMMON MISTAKES

- Renaming a column the app still reads
- Type changes on huge tables during business hours
- `DROP … CASCADE` removing more than you meant

✓ TIPS & BEST PRACTICES

- Expand → migrate → contract: add new, backfill, then drop old
- Test migrations on a production-size copy
- Set `lock_timeout` so DDL gives up instead of blocking

QUICK SUMMARY

- `ALTER TABLE` add / drop / rename / type
- `DROP TABLE [IF EXISTS] [CASCADE]`
- Mind locks and rewrites

Databases, Schemas & the Catalog

WHERE TABLES LIVE

A server holds **databases**; in PostgreSQL and SQL Server each database holds **schemas** (namespaces such as `public` or `dbo`), and schemas hold tables. Refer to a table as `schema.table`, or let **`search_path`** (PG) or the default schema find it. The **catalog** is the database describing itself: the standard **`information_schema`** views, plus engine-specific ones (`pg_catalog`, `sys`), list every table, column and constraint.

CATALOG FACTS

- MySQL "schema" and "database" mean the same thing
- Cross-database queries: SQL Server yes, PG no
- `information_schema` is portable but slow
- Engine catalogs show more detail

NOSQL COMPARISON

- MongoDB: server → databases → collections
- `use shop`, `show collections`, `db.getCollectionInfos()`
- `db.orders.getIndexes()` lists indexes

DATABASES, SCHEMAS & CATALOG

```
CREATE DATABASE shop;              -- server level
\c shop                            -- psql: connect to it

CREATE SCHEMA sales;               -- namespace in a DB
CREATE TABLE sales.invoices (id INT);
SET search_path TO sales, public;
SELECT * FROM invoices;            -- finds sales.invoices

-- Which tables exist?
SELECT table_schema, table_name
FROM   information_schema.tables
WHERE  table_schema = 'public';

-- Columns of one table
SELECT column_name, data_type, is_nullable, column_default
FROM   information_schema.columns
WHERE  table_name = 'orders'
ORDER  BY ordinal_position;

-- Foreign keys pointing at customers (PG catalog)
SELECT conname, conrelid::regclass AS from_table
FROM   pg_constraint
WHERE  confrelid = 'customers'::regclass;

SELECT pg_size_pretty(pg_total_relation_size('orders'));
```

CODE EXPLAINED

- → `search_path` sets the lookup order for unqualified names
- → `information_schema` — standard views in PG, MySQL, SQL Server
- → `::regclass` turns a table name into its catalog id
- → `pg_total_relation_size` includes indexes and TOAST

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	database → schema → table; `search_path`; `pg_catalog`
MySQL	schema = database; `USE shop;`; `SHOW TABLES`
SQL Server	database → schema (`dbo`) → table; `sys.tables`
SQLite	file = database; `ATTACH 'x.db' AS x`; `sqlite_schema`

IN AN ORM

```
model Invoice {
  id Int @id
  @@schema("sales")        // multiSchema
}
// npx prisma db pull → builds models from the catalog
```

HIERARCHY

ENGINE	LEVELS
PostgreSQL	cluster › database › schema › table
MySQL	server › database (= schema) › table
SQL Server	instance › database › schema › table
SQLite	file (+ attached files) › table

PSQL DESCRIBE COMMANDS

COMMAND	LISTS
`\l`	databases
`\dn`	schemas
`\dt · \dv`	tables · views
`\d orders`	one table: columns, indexes, FKs
`\di · \df`	indexes · functions
`\du · \dx`	roles · extensions

⚠ COMMON MISTAKES

- Creating tables in `public` by accident (wrong `search_path`)
- Hard-coding `dbo.` or `public.` everywhere
- Querying `information_schema` in a hot path

✓ TIPS & BEST PRACTICES

- Group related tables per schema; grant per schema (sheet 75)
- Use the catalog to generate docs and checks
- Keep one database per app, not per customer, unless you must

QUICK SUMMARY

- Database › schema › table
- `search_path` / `USE` for defaults
- `information_schema` + engine catalogs

Normalization

1NF · 2NF · 3NF · BCNF · functional dependency · denormalization

ONE FACT, ONE PLACE

Normalization arranges columns so each fact is stored **once**, which prevents **anomalies**: a city changed in one order row but not the others. It's built on **functional dependency**: A → B means knowing A determines B (a `customer_id` determines the customer's city). Each normal form removes one kind of misplaced dependency. **Denormalization** later copies data back on purpose for speed, as `orders.total` does in the shop schema.

DESIGN FACTS

- Aim for 3NF; it's the practical default
- `order_items.unit_price` is a price **snapshot**, not a 3NF breach
- Each denormalized copy needs a way to stay in sync
- Normalize first, denormalize with evidence

NOSQL COMPARISON

- Document databases **denormalize by design**
- Embed what's read together; copy names into orders
- The cost moves to updates: change a name in many documents

NORMALIZING AN ORDER TABLE

```
-- Unnormalized: repeats and lists
CREATE TABLE orders_flat (
  order_id      INT,
  customer_name TEXT,    -- repeated on every order
  customer_city TEXT,    -- depends on the customer
  products      TEXT,    -- 'Lamp, Desk': not atomic
  prices        TEXT     -- '39.99, 129.00'
);

-- 1NF: one value per cell → one row per item
-- 2NF: product name depends on product_id alone,
--      not the whole (order_id, product_id) key
--      → move to products
-- 3NF: customer_city depends on the customer,
--      not the order → move to customers

-- Result: the shop schema (sheet 04)
--   customers(customer_id, name, city)
--   orders(order_id, customer_id → customers)
--   products(product_id, name, price)
--   order_items(order_id, product_id, qty, price)

-- Deliberate denormalization: cached total
UPDATE orders o
SET    total = (SELECT sum(quantity * unit_price)
                FROM   order_items i
                WHERE  i.order_id = o.order_id);
```

CODE EXPLAINED

→ A list in one cell breaks 1NF: you can't join or index it

→ Partial dependency on part of a composite key breaks 2NF

→ Non-key → non-key (order → customer → city) breaks 3NF

→ `total` duplicates the items; keep it in sync (sheet 67)

DIALECT NOTES

Same in all four: normalization is a design discipline, not syntax. Enforce the result with keys and constraints (sheet 56, sheet 57).

IN AN ORM

```
model Order {
  id    Int         @id
  items OrderItem[]   // 1NF: items in their own table
}
model OrderItem {
  orderId   Int
  productId Int       // name and price live in Product
  @@id([orderId, productId])
}
```

NORMAL FORMS

FORM	RULE	TYPICAL VIOLATION
1NF	atomic values, no repeating groups	`tags = 'a,b,c'`
2NF	1NF + no partial key dependency	product name in `order_items`
3NF	2NF + no non-key → non-key	customer city in `orders`
BCNF	every determinant is a key	rare overlapping-key cases

ANOMALIES PREVENTED

ANOMALY	EXAMPLE
`update`	city changed on some order rows only
`insert`	can't add a product until someone orders it
`delete`	deleting the last order loses the customer
`denormalize when`	reads dominate and the copy is maintained

⚠ COMMON MISTAKES

- Comma-separated lists in a column
- Copying customer details into every order "for convenience"
- Denormalizing before measuring a real problem

✓ TIPS & BEST PRACTICES

- Ask "what does this column depend on?" for every column
- Snapshot values that must not change later (prices, addresses)
- Use materialized views for read-heavy rollups (sheet 64)

QUICK SUMMARY

- 1NF atomic · 2NF whole key · 3NF nothing but the key
- Prevents update anomalies
- Denormalize deliberately

Relationships & ER Modeling

CONNECTING ENTITIES

Entities relate in three ways. **One-to-many** is the workhorse: the foreign key goes on the "many" side (`orders.customer_id`). **One-to-one** puts a unique foreign key (often the primary key itself) on the optional side. **Many-to-many** needs a **junction table** holding two foreign keys, like `order_items` or `product_tags`. An **ER diagram** shows entities as boxes and **cardinality** with crow's-foot marks on the lines.

MODELING FACTS

- The FK lives on the "many" side
- Junction tables can carry data (`quantity`)
- `NOT NULL` FK = mandatory; nullable = optional
- Self-relationships: `employees.manager_id` (sheet 34)

NOSQL COMPARISON

- One-to-many: embed (few, owned) or reference (many, shared)
- Many-to-many: arrays of ids, e.g. `tags: ['sale', 'new']`
- No junction collection unless the link has its own data

RELATIONSHIP TABLES

```sql
-- One-to-many: a customer has many orders
CREATE TABLE orders (
  order_id    INT PRIMARY KEY,
  customer_id INT NOT NULL REFERENCES customers
);
-- One-to-one: optional profile per customer
CREATE TABLE customer_profiles (
  customer_id INT PRIMARY KEY REFERENCES customers,
  birthday    DATE,
  newsletter  BOOLEAN NOT NULL DEFAULT false
);
-- Many-to-many: products ↔ tags
CREATE TABLE tags (tag_id INT PRIMARY KEY, name TEXT);
CREATE TABLE product_tags (
  product_id INT REFERENCES products ON DELETE CASCADE,
  tag_id     INT REFERENCES tags     ON DELETE CASCADE,
  PRIMARY KEY (product_id, tag_id)
);

-- Querying many-to-many: two joins
SELECT p.name, t.name AS tag
FROM   products p
JOIN   product_tags pt USING (product_id)
JOIN   tags t          USING (tag_id);
```

CODE EXPLAINED

→ `customer_id NOT NULL` — every order must have a customer

→ Profile PK = FK, so at most one profile per customer

→ Composite PK stops tagging a product twice

→ `CASCADE` removes links when either side is deleted

DIALECT NOTES

Same in all four. Only the key syntax differs (identity columns, `AUTO_INCREMENT`); relationships are always foreign keys.

IN AN ORM

```
model Product {
  id   Int   @id
  tags Tag[]       // implicit many-to-many
}
model Tag {
  id       Int       @id
  products Product[] // Prisma creates _ProductToTag
}
```

RELATIONSHIP TYPES

TYPE	WHERE THE FK GOES	SHOP EXAMPLE
`one-to-many`	on the many side	customers → orders
`one-to-one`	unique FK on the optional side	customers → customer_profiles
`many-to-many`	junction table, two FKs	orders ↔ products via order_items
`self`	FK to the same table	employees.manager_id

CROW'S FOOT (MERMAID)

MARK	MEANS
`||`	exactly one
`|o o|`	zero or one
`}| |{`	one or many
`}o o{`	zero or many
`CUSTOMER ||--o{ ORDER`	a customer has zero or more orders

⚠ COMMON MISTAKES

- Putting the FK on the "one" side (`customers.order_id`)
- Modeling many-to-many with a comma list
- One-to-one tables without a unique constraint

✓ TIPS & BEST PRACTICES

- Name junction tables after both sides: `product_tags`
- Sketch the ERD before writing DDL
- Generate diagrams from the live schema with tools like DBeaver

QUICK SUMMARY

- 1:N — FK on the many side
- 1:1 — unique FK
- M:N — junction table

Design Conventions

snake_case · surrogate vs natural key · created_at / updated_at · soft delete · FK naming

HOUSE RULES THAT PAY OFF

Consistent conventions make a schema predictable. Use **snake_case** names, which never need quotes. Give most tables a **surrogate key** (a meaningless identity or UUID) and enforce the **natural key** (email, SKU) with a unique constraint, so business values can change without rewriting foreign keys. Add **created_at / updated_at** timestamps, name foreign keys **<table>_id**, and decide up front how deletes work (hard or **soft**).

CONVENTION FACTS

- Pick plural (`orders`) or singular and stick to it
- Booleans as questions: `is_active` , `has_paid`
- Store money as `NUMERIC` , times as `TIMESTAMPTZ`
- Avoid reserved words: `user` , `order` , `group`

NOSQL COMPARISON

- MongoDB convention: `camelCase` fields and the surrogate `_id`
- Mongoose `timestamps: true` adds `createdAt` / `updatedAt`
- Soft delete with `deletedAt` the same way

A TABLE FOLLOWING THE CONVENTIONS

```
CREATE TABLE support_tickets (
  ticket_id   BIGINT GENERATED ALWAYS AS IDENTITY
              PRIMARY KEY,              -- surrogate
  public_ref  TEXT NOT NULL UNIQUE,     -- natural key
  customer_id INT NOT NULL
              REFERENCES customers,     -- <table>_id
  subject     TEXT NOT NULL,
  is_closed   BOOLEAN NOT NULL DEFAULT false,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at  TIMESTAMPTZ               -- soft delete
);
CREATE INDEX support_tickets_customer_id_idx
  ON support_tickets (customer_id);

-- Keep updated_at current
CREATE FUNCTION touch() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;
CREATE TRIGGER support_tickets_touch
  BEFORE UPDATE ON support_tickets
  FOR EACH ROW EXECUTE FUNCTION touch();
```

CODE EXPLAINED

→ `ticket_id` never changes; `public_ref` is what users see

→ FK named after the table it points to: `customer_id`

→ Index name: `<table>_<column>_idx`

→ The trigger stamps `updated_at` on every update (sheet 67)

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	unquoted names fold to lower case: use `snake_case`
MySQL	`ON UPDATE CURRENT_TIMESTAMP` keeps `updated_at` current
SQL Server	PascalCase is common; `dbo` schema; trigger for `updated_at`
SQLite	no `ON UPDATE`; use a trigger

IN AN ORM

```
model SupportTicket {
  id        BigInt    @id @default(autoincrement())
  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt @map("updated_at")
  deletedAt DateTime? @map("deleted_at")
  @@map("support_tickets")
}
```

NAMING

THING	CONVENTION	EXAMPLE
table	plural snake_case	`order_items`
primary key	`<singular>_id` or `id`	`order_id`
foreign key	`<referenced>_id`	`customer_id`
boolean	`is_` / `has_`	`is_closed`
timestamp	`<event>_at`	`shipped_at`
index / constraint	`<table>_<cols>_<kind>`	`orders_status_idx`

SURROGATE VS NATURAL KEY

	SURROGATE	NATURAL
example	`ticket_id` 4812	`email`, SKU, ISBN
changes	never	sometimes
size	small, fixed	varies
meaning	none	business meaning
use	PK + FKs	`UNIQUE` constraint

⚠ COMMON MISTAKES

- Mixed styles: `CustomerID` , `order_id` , `productid`
- Naming a table `user` or `order` and quoting forever
- Email as the primary key, then a customer changes it

✓ TIPS & BEST PRACTICES

- Write the conventions down in the repo
- Let a linter (sqlfluff, squawk) enforce them
- Prisma `@map` / `@@map` bridges camelCase code and snake_case tables

QUICK SUMMARY

- `snake_case` , `<table>_id` , `<event>_at`
- Surrogate PK + unique natural key
- `created_at` / `updated_at` / `deleted_at`

Views & Materialized Views

CREATE VIEW · OR REPLACE · WITH CHECK OPTION · updatable view · MATERIALIZED VIEW · REFRESH · CONCURRENTLY · indexed view

SAVED QUERIES

A **view** is a named query you select from like a table. It stores **no data**: each use runs the query, so it's always current. Views hide complexity, give apps a stable interface and limit which columns or rows a role can see. A simple single-table view is **updatable**; **`WITH CHECK OPTION`** blocks writes the view couldn't see. A **materialized view** stores the result: fast to read, but stale until you **`REFRESH`** it.

VIEW FACTS

- Views can be granted separately from tables (sheet 75)
- `OR REPLACE` can add columns, not drop or retype them
- Filters on a view are pushed into its query
- `CONCURRENTLY` needs a unique index on the MV

NOSQL COMPARISON

- MongoDB: `db.createView('active', 'customers', [pipeline])` — read-only
- On-demand materialized views: an aggregation ending in `$merge`
- Re-run the `$merge` pipeline to refresh

VIEWS

```
CREATE VIEW active_customers AS
SELECT customer_id, name, email, city
FROM   customers
WHERE  deleted_at IS NULL;

SELECT * FROM active_customers WHERE city = 'Oslo';

CREATE OR REPLACE VIEW order_summary AS
SELECT o.order_id, c.name, o.order_date, o.total
FROM   orders o
JOIN   customers c USING (customer_id);

-- Updatable view that rejects out-of-view rows
CREATE VIEW ca_customers AS
SELECT * FROM customers WHERE country = 'CA'
WITH CHECK OPTION;
INSERT INTO ca_customers (name, country)
VALUES ('Zed', 'US');
-- ERROR: new row violates check option

-- Stored result, refreshed on demand
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT date_trunc('month', order_date) AS month,
       sum(total) AS revenue
FROM   orders GROUP BY month;
CREATE UNIQUE INDEX ON monthly_revenue (month);
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue;
```

CODE EXPLAINED

→ `active_customers` hides soft-deleted rows (sheet 52)

→ `WITH CHECK OPTION` — inserts must satisfy `country = 'CA'`

→ The materialized view is computed once and stored

→ `CONCURRENTLY` refreshes without blocking readers

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	views + `MATERIALIZED VIEW`, `REFRESH … CONCURRENTLY`
MySQL	views only; emulate MVs with a table + scheduled event
SQL Server	indexed views (`SCHEMABINDING` + clustered index)
SQLite	read-only views; `INSTEAD OF` triggers to write

IN AN ORM

```
view ActiveCustomer {      // views preview feature
  id    Int    @unique
  name  String
  email String
}
// prisma.activeCustomer.findMany() — read-only
```

VIEW VS MATERIALIZED VIEW

	VIEW	MATERIALIZED VIEW
stores data	no	yes
freshness	always current	as of last refresh
read speed	same as the query	fast; can be indexed
refresh	—	`REFRESH MATERIALIZED VIEW`
best for	abstraction, security	heavy reports, dashboards

UPDATABLE VIEW RULES

SIMPLE VIEW NEEDS	EXAMPLE OF BREAKING IT
one table in `FROM`	a `JOIN`
no aggregates	`GROUP BY`, `sum()`
no `DISTINCT` / `LIMIT`	`SELECT DISTINCT …`
plain columns	`price * 2 AS p` (read-only column)
otherwise	`INSTEAD OF` trigger (sheet 67)

⚠ COMMON MISTAKES

- Stacking views on views until no one can tune the query
- Reading a materialized view as if it were live
- `SELECT *` in a view: columns are fixed at creation time

✓ TIPS & BEST PRACTICES

- Use views as the API for reporting tools
- Schedule MV refreshes (cron, `pg_cron`) after loads
- List columns explicitly in view definitions

QUICK SUMMARY

- View — saved query, no data
- Materialized — stored, needs refresh
- `WITH CHECK OPTION` guards writes

User-Defined Functions

YOUR OWN FUNCTIONS

`CREATE FUNCTION` packages logic you can call inside queries, like a built-in. A **scalar** function returns one value; a **table function** (`RETURNS TABLE`) returns rows and is used in `FROM`. PostgreSQL functions can be plain **`LANGUAGE sql`** (simple, often inlined) or **`plpgsql`** with variables, `IF` and loops. Mark **volatility** honestly: **`IMMUTABLE`** functions can be used in indexes, and the planner optimizes around the label.

FUNCTION FACTS

- Functions run inside the caller's transaction
- They can't `COMMIT` ; procedures can (sheet 66)
- Overloading: same name, different argument types
- `$$ … $$` quotes the body without escaping

NOSQL COMPARISON

- MongoDB has no user-defined query functions in practice
- Logic lives in the app or in Atlas Functions
- `$function` (server-side JS) exists but is discouraged

CREATE FUNCTION

```sql
-- Scalar SQL function (PG 14+ RETURN syntax)
CREATE FUNCTION with_tax(amount NUMERIC,
                         rate NUMERIC DEFAULT 0.08)
RETURNS NUMERIC LANGUAGE sql IMMUTABLE
RETURN round(amount * (1 + rate), 2);

SELECT name, with_tax(price) FROM products;
-- Table function
CREATE FUNCTION customer_orders(p_customer INT)
RETURNS TABLE (order_id INT, order_date DATE, total NUMERIC)
LANGUAGE sql STABLE AS $$
  SELECT order_id, order_date, total
  FROM   orders
  WHERE  customer_id = p_customer
$$;
SELECT * FROM customer_orders(7);

CREATE OR REPLACE FUNCTION loyalty_tier(p_id INT)
RETURNS TEXT LANGUAGE plpgsql STABLE AS $$
DECLARE spent NUMERIC;
BEGIN
  SELECT coalesce(sum(total), 0) INTO spent
  FROM   orders WHERE customer_id = p_id;
  RETURN CASE WHEN spent >= 1000 THEN 'gold'
              WHEN spent >= 250  THEN 'silver'
              ELSE 'bronze' END;
END $$;
```

CODE EXPLAINED

- → `DEFAULT 0.08` — `rate` can be left out
- → `IMMUTABLE` — same inputs, same result, no table reads
- → `STABLE` — reads tables; constant within one statement
- → `SELECT … INTO spent` stores a query result in a variable

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`LANGUAGE sql` / `plpgsql`; volatility labels
MySQL	`CREATE FUNCTION … DETERMINISTIC`; scalar only
SQL Server	scalar + inline table-valued: `RETURNS TABLE AS RETURN …`
SQLite	none in SQL; register functions from the host language

IN AN ORM

```sql
const orders = await prisma.$queryRaw`
  SELECT * FROM customer_orders(${customerId})`;
const [{ tier }] = await prisma.$queryRaw`
  SELECT loyalty_tier(${customerId}) AS tier`;
```

VOLATILITY

LABEL	PROMISE	EXAMPLE
`IMMUTABLE`	same args → same result, forever	`with_tax`, `lower`
`STABLE`	same result within one statement	`loyalty_tier`, `now()`
`VOLATILE`	may change every call (default)	`random()`, inserts

FUNCTION PARTS

PART	EXAMPLE
`arguments`	`(p_id INT, rate NUMERIC DEFAULT 0.08)`
`return type`	`RETURNS NUMERIC` · `RETURNS TABLE (…)`
`language`	`LANGUAGE sql` · `LANGUAGE plpgsql`
`body`	`AS $$ … $$` or `RETURN expr`
`options`	`STRICT` · `SECURITY DEFINER` · `PARALLEL SAFE`

⚠ COMMON MISTAKES

- Labeling a function `IMMUTABLE` when it reads tables
- Calling a slow function per row in a big `SELECT`
- Parameter names that clash with column names in PL/pgSQL

✓ TIPS & BEST PRACTICES

- Prefix parameters (`p_`) and variables (`v_`)
- Prefer `LANGUAGE sql` for simple logic; it can be inlined
- Version functions with your migrations

QUICK SUMMARY

- `CREATE FUNCTION … RETURNS …`
- Scalar or `RETURNS TABLE`
- `IMMUTABLE` · `STABLE` · `VOLATILE`

Stored Procedures

CREATE PROCEDURE · CALL · EXEC · IN / OUT params · DECLARE · IF / LOOP

SERVER-SIDE ROUTINES

A **stored procedure** is a named block of statements stored in the database and run with **CALL** (SQL Server: **EXEC**). Unlike a function it isn't used inside a query, it can return values through **OUT parameters**, and in PostgreSQL 11+ it can **COMMIT** part-way through. Procedures keep multi-step operations ("place an order") in one round trip, close to the data, with variables (DECLARE), branches (IF) and loops.

PROCEDURE FACTS

- Runs on the server: one network round trip
- Errors abort it; `RAISE EXCEPTION` aborts on purpose
- `COMMIT` inside fails if the caller opened a transaction
- Business logic in SQL is harder to test and version

NOSQL COMPARISON

- MongoDB has no stored procedures
- Multi-document work uses driver transactions: `session.withTransaction(…)`
- Logic stays in application code

CREATE PROCEDURE

```
CREATE PROCEDURE place_order(
  p_customer INT, p_product INT, p_qty INT,
  OUT p_order_id INT)
LANGUAGE plpgsql AS $$
DECLARE
  v_price NUMERIC;
BEGIN
  SELECT price INTO v_price
  FROM   products
  WHERE  product_id = p_product
  FOR UPDATE;                       -- lock the row
  IF NOT FOUND THEN
    RAISE EXCEPTION 'product % not found', p_product;
  END IF;
  INSERT INTO orders (customer_id, total)
  VALUES (p_customer, v_price * p_qty)
  RETURNING order_id INTO p_order_id;

  INSERT INTO order_items
  VALUES (p_order_id, p_product, p_qty, v_price);

  UPDATE products SET stock = stock - p_qty
  WHERE  product_id = p_product;
END $$;

CALL place_order(7, 3, 2, NULL);   -- returns p_order_id
```

CODE EXPLAINED

- → `OUT p_order_id` comes back as a result row from `CALL`
- → `FOR UPDATE` stops two orders overselling stock (sheet 70)
- → `NOT FOUND` is set by the previous `SELECT INTO`
- → Pass `NULL` as the placeholder for the `OUT` argument

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`CREATE PROCEDURE` (11+), `CALL`, `OUT` params (14+)
MySQL	`DELIMITER //` … `CREATE PROCEDURE … BEGIN … END`
SQL Server	`CREATE PROCEDURE … AS BEGIN … END`; `EXEC p @x = 1`
SQLite	no stored procedures

IN AN ORM

```
const [row] = await prisma.$queryRaw`
  CALL place_order(${customerId}, ${productId},
                   ${qty}, NULL)`;
// row.p_order_id holds the OUT value
```

- Many teams keep this logic in `prisma.$transaction(async tx => …)` instead

FUNCTION VS PROCEDURE

	FUNCTION	PROCEDURE
called with	`SELECT f()` / in queries	`CALL p()` / `EXEC p`
returns	a value or rows	`OUT` params, result sets
transaction control	no	yes (PG 11+, SQL Server)
use for	calculations, filters	multi-step operations

PL/PGSQL BUILDING BLOCKS

CONSTRUCT	EXAMPLE
variable	`DECLARE v NUMERIC := 0;`
assign	`v := v + 1;` · `SELECT … INTO v`
branch	`IF … ELSIF … ELSE … END IF;`
loop	`FOR r IN SELECT … LOOP … END LOOP;`
error	`RAISE EXCEPTION 'bad %', x;`
catch	`EXCEPTION WHEN unique_violation THEN …`

⚠ COMMON MISTAKES

- Building SQL strings inside procedures (injection, sheet 76)
- Hiding core business rules where the app team can't see them
- Forgetting row locks in read-then-write logic

✓ TIPS & BEST PRACTICES

- Keep procedures small and focused on data integrity
- Keep their source in migrations and code review
- Return clear errors with `RAISE` and SQLSTATE codes

QUICK SUMMARY

- `CREATE PROCEDURE` · `CALL` / `EXEC`
- `IN` / `OUT` params
- Can control transactions

Triggers

CREATE TRIGGER · BEFORE / AFTER · FOR EACH ROW · NEW / OLD · INSTEAD OF · audit trigger

CODE THAT RUNS ON CHANGE

A **trigger** runs automatically when rows are inserted, updated or deleted. **BEFORE** triggers can inspect and change the incoming row (**NEW**) or cancel it; **AFTER** triggers react once the change is made, which suits **audit logs**; **INSTEAD OF** triggers make views writable. `FOR EACH ROW` fires per changed row, `FOR EACH STATEMENT` once per statement. In PostgreSQL the logic lives in a **trigger function**.

TRIGGER FACTS

- `NEW` = row after the change; `OLD` = row before
- Triggers fire for every writer: apps, scripts, ORMs
- They run inside the same transaction
- Hidden side effects surprise people: document them

NOSQL COMPARISON

- MongoDB **change streams** (`collection.watch()`) react after commits
- Atlas Triggers run functions on changes
- No `BEFORE` triggers: validate in the app or with `$jsonSchema`

AUDIT TRIGGER

```
CREATE TABLE price_history (
  product_id INT,
  old_price  NUMERIC(10,2),
  new_price  NUMERIC(10,2),
  changed_at TIMESTAMPTZ DEFAULT now(),
  changed_by TEXT DEFAULT current_user
);

-- Trigger function: runs once per changed row
CREATE FUNCTION log_price() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO price_history
         (product_id, old_price, new_price)
  VALUES (OLD.product_id, OLD.price, NEW.price);
  RETURN NEW;
END $$;

CREATE TRIGGER products_price_audit
AFTER UPDATE OF price ON products
FOR EACH ROW
WHEN (OLD.price IS DISTINCT FROM NEW.price)
EXECUTE FUNCTION log_price();

UPDATE products SET price = 42 WHERE product_id = 7;
SELECT * FROM price_history;   -- one new row
```

CODE EXPLAINED

- → `RETURNS trigger` — a special function type for triggers
- → `UPDATE OF price` — only when `price` is in the `SET` list
- → `WHEN (…)` skips updates that didn't change the value
- → `RETURN NEW` is required for row triggers (ignored in `AFTER`)

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	trigger function + `EXECUTE FUNCTION`; statement-level too
MySQL	inline body `FOR EACH ROW BEGIN … END`; row-level only
SQL Server	statement-level; `inserted` / `deleted` pseudo-tables
SQLite	`BEGIN … END` body; `NEW.` / `OLD.`; `INSTEAD OF` on views

IN AN ORM

```
// Database triggers fire for Prisma writes too.
// App-side hooks run only for Prisma calls:
const db = prisma.$extends({ query: { product: {
  async update({ args, query }) { /* log */ return query(args); },
} } });
```

TIMING

TIMING	CAN	TYPICAL USE
`BEFORE`	change or reject `NEW`	defaults, normalizing, validation
`AFTER`	see the final row	audit logs, counters, notifications
`INSTEAD OF`	replace the write	updatable complex views
`FOR EACH STATEMENT`	run once per statement	bulk bookkeeping

NEW / OLD BY EVENT

EVENT	NEW	OLD
`INSERT`	the new row	—
`UPDATE`	row after	row before
`DELETE`	—	the deleted row
`TRUNCATE`	no row triggers fire	

⚠ COMMON MISTAKES

- Business logic hidden in triggers nobody knows about
- Triggers that update the same table and recurse
- Slow `AFTER` triggers turning bulk updates into crawls

✓ TIPS & BEST PRACTICES

- Use triggers for integrity and auditing, not workflows
- Name them `<table>_<purpose>` and list them in docs
- PG: `pg_trigger_depth()` guards against recursion

QUICK SUMMARY

- `BEFORE` / `AFTER` / `INSTEAD OF`
- `NEW` / `OLD` row values
- Fire for every writer

Transactions

ALL OR NOTHING

A **transaction** groups statements so they succeed or fail **as a unit**. **`BEGIN`** starts one, **`COMMIT`** makes every change permanent and visible, and **`ROLLBACK`** undoes them all. Outside an explicit transaction, most clients run in **autocommit** mode: each statement commits by itself. A **`SAVEPOINT`** marks a point you can roll back to without abandoning the whole transaction. Together this gives the **ACID** guarantees.

TRANSACTION FACTS

- Other sessions don't see your changes until `COMMIT`
- PG: after an error, roll back before continuing
- Keep transactions short: they hold locks
- PG DDL is transactional too (`CREATE` , `ALTER`)

NOSQL COMPARISON

- MongoDB: a single-document write is always atomic
- Multi-document transactions (4.0+): `session.withTransaction(async () => …)`
- Design documents so one write covers one change

BEGIN … COMMIT

```sql
BEGIN;
UPDATE products SET stock = stock - 2
WHERE  product_id = 7;
INSERT INTO orders (customer_id, total)
VALUES (42, 79.98);
COMMIT;              -- both changes become visible

BEGIN;
DELETE FROM order_items WHERE order_id = 101;
ROLLBACK;            -- as if it never happened
-- Partial rollback with a savepoint
BEGIN;
INSERT INTO customers (name, email)
VALUES ('Ada', 'ada@example.com');
SAVEPOINT before_order;
INSERT INTO orders (customer_id, total)
VALUES (99999, 10);  -- ERROR: foreign key violation
ROLLBACK TO SAVEPOINT before_order;
INSERT INTO orders (customer_id, total) VALUES (1, 10);
COMMIT;              -- customer + valid order saved
-- PostgreSQL: an error poisons the transaction
BEGIN;
SELECT 1 / 0;        -- ERROR
SELECT 1;            -- ERROR: transaction is aborted
ROLLBACK;
```

CODE EXPLAINED

→ Stock and order change together, or not at all

→ `ROLLBACK` discards everything since `BEGIN`

→ `ROLLBACK TO SAVEPOINT` undoes only the failed part

→ PG refuses further statements until `ROLLBACK`

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`BEGIN` / `START TRANSACTION`; transactional DDL
MySQL	`START TRANSACTION`; DDL commits implicitly
SQL Server	`BEGIN TRAN`; `SAVE TRAN s`; set `XACT_ABORT ON`
SQLite	`BEGIN IMMEDIATE` to take the write lock up front

IN AN ORM

```ts
await prisma.$transaction(async (tx) => {
  await tx.product.update({
    where: { id: 7 }, data: { stock: { decrement: 2 } },
  });
  await tx.order.create({
    data: { customerId: 42, total: 79.98 },
  });
});  // a thrown error → automatic ROLLBACK
```

ACID

LETTER	GUARANTEE
Atomicity	all statements apply, or none do
Consistency	constraints hold before and after
Isolation	concurrent transactions don't interfere (sheet 69)
Durability	committed data survives a crash

STATEMENTS

STATEMENT	DOES
`BEGIN · START TRANSACTION`	start a transaction
`COMMIT`	save and end
`ROLLBACK`	undo and end
`SAVEPOINT s`	mark a point
`ROLLBACK TO SAVEPOINT s`	undo back to the mark
`RELEASE SAVEPOINT s`	forget the mark

⚠ COMMON MISTAKES

- Leaving a transaction open (idle in transaction) and blocking others
- Calling an external API inside a transaction
- Assuming MySQL can roll back a `CREATE TABLE`

✓ TIPS & BEST PRACTICES

- Do slow work before `BEGIN` ; keep the transaction tight
- Retry on serialization or deadlock errors (sheet 69, sheet 70)
- Set `idle_in_transaction_session_timeout` in PostgreSQL

QUICK SUMMARY

- `BEGIN` → statements → `COMMIT` / `ROLLBACK`
- `SAVEPOINT` for partial undo
- ACID

Isolation Levels

READ COMMITTED · REPEATABLE READ · SERIALIZABLE · dirty read · phantom read · MVCC

WHAT CONCURRENT TRANSACTIONS SEE

The **isolation level** decides how much of other transactions' work you can see while yours runs. Weaker levels allow **anomalies**: a **dirty read** sees uncommitted data, a **non-repeatable read** sees a row change between two reads, a **phantom** is a new row appearing in a repeated query. **READ COMMITTED** (the PostgreSQL and SQL Server default) re-reads fresh data per statement; **REPEATABLE READ** reads one snapshot; **SERIALIZABLE** behaves as if transactions ran one at a time, and may abort yours with a **retryable** error.

ISOLATION FACTS

- **MVCC**: readers see a snapshot; they don't block writers
- PostgreSQL never allows dirty reads
- Higher levels → more retries, not more locks (PG)
- Set the level before the first query in the transaction

NOSQL COMPARISON

- MongoDB transactions use **snapshot** isolation
- `readConcern: 'snapshot'`, `writeConcern: 'majority'`
- Conflicts abort with `TransientTransactionError`: retry

ISOLATION IN ACTION

```
-- A: one snapshot for the whole transaction
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT stock FROM products WHERE product_id = 7;  -- 10
--   B (autocommit): UPDATE products SET stock = 4
--                   WHERE product_id = 7;
SELECT stock FROM products WHERE product_id = 7;  -- 10
COMMIT;
SELECT stock FROM products WHERE product_id = 7;  -- 4
-- Under READ COMMITTED the 2nd read would show 4

-- A: update a row B changed after A's snapshot
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT stock FROM products WHERE product_id = 7;
--   B: UPDATE … stock = stock - 1 … ; COMMIT
UPDATE products SET stock = stock - 1
WHERE  product_id = 7;
-- ERROR: could not serialize access due to
--        concurrent update → ROLLBACK and retry

-- Make it the default for a session
SET SESSION CHARACTERISTICS AS TRANSACTION
  ISOLATION LEVEL SERIALIZABLE;
SHOW transaction_isolation;
```

CODE EXPLAINED

- → `REPEATABLE READ` keeps showing the snapshot value
- → The final read, after `COMMIT`, sees B's change
- → PG stops the lost update with a serialization error
- → Apps must catch SQLSTATE `40001` and retry the transaction

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	default `READ COMMITTED`; true `SERIALIZABLE` (SSI)
MySQL	InnoDB default `REPEATABLE READ`; gap locks
SQL Server	default `READ COMMITTED` (locking); `SNAPSHOT` option
SQLite	effectively `SERIALIZABLE`: one writer at a time

IN AN ORM

```
await prisma.$transaction(
  async (tx) => { /* reads and writes */ },
  { isolationLevel: 'Serializable', maxWait: 5000 },
);
// catch P2034 (write conflict) and retry
```

ANOMALIES ALLOWED (STANDARD)

LEVEL	DIRTY	NON-REPEATABLE	PHANTOM
`READ UNCOMMITTED`	yes	yes	yes
`READ COMMITTED`	no	yes	yes
`REPEATABLE READ`	no	no	yes (PG: no)
`SERIALIZABLE`	no	no	no

PICK A LEVEL

SITUATION	LEVEL
typical web request	`READ COMMITTED`
consistent report over many queries	`REPEATABLE READ`
invariants across rows (no double-booking)	`SERIALIZABLE` + retry
single-row read-modify-write	`READ COMMITTED` + `FOR UPDATE` (sheet 70)

⚠ COMMON MISTAKES

- Using `SERIALIZABLE` without retry logic
- Assuming every engine's `REPEATABLE READ` behaves the same
- Reading, computing in the app, then writing back blindly

✓ TIPS & BEST PRACTICES

- Prefer atomic updates: `SET stock = stock - 1 WHERE stock > 0`
- Wrap transactions in a retry loop for `40001` / `40P01`
- Keep long reports on a replica or snapshot

QUICK SUMMARY

- RC (default) · RR (snapshot) · SERIALIZABLE
- Anomalies: dirty, non-repeatable, phantom
- MVCC + retries

Locking & Deadlocks

FOR UPDATE · FOR SHARE · SKIP LOCKED · NOWAIT · LOCK TABLE · deadlock · advisory lock

COORDINATING WRITERS

Writers lock the rows they change until `COMMIT`. When you **read a row and then update it** based on what you saw, take the lock up front with **`SELECT … FOR UPDATE`** so no one changes it in between. **`NOWAIT`** errors instead of waiting; **`SKIP LOCKED`** skips rows others hold, which turns a table into a safe **job queue**. A **deadlock** happens when two transactions each wait for the other's lock; the database aborts one of them.

LOCK FACTS

- Plain `SELECT` takes no row locks (MVCC)
- Locks are held until the transaction ends
- Lock rows in a consistent order to avoid deadlocks
- Advisory locks are app-defined, named by a number

NOSQL COMPARISON

- MongoDB: single-document atomic updates avoid read-then-write locks:
- `findOneAndUpdate({ _id, stock: { $gte: 1 } }, { $inc: { stock: -1 } })`
- SQL can do the same: `UPDATE … WHERE stock >= 1`

ROW LOCKS

```sql
-- Read-then-write: lock the row first
BEGIN;
SELECT stock FROM products
WHERE  product_id = 7
FOR UPDATE;                    -- others wait here
UPDATE products SET stock = stock - 1
WHERE  product_id = 7;
COMMIT;                        -- lock released

-- Fail fast instead of waiting
SELECT * FROM orders
WHERE  order_id = 101 FOR UPDATE NOWAIT;

-- Workers each claim a different paid order
UPDATE orders SET status = 'shipped'
WHERE  order_id = (SELECT order_id FROM orders
                   WHERE  status = 'paid'
                   ORDER  BY order_date
                   LIMIT  1
                   FOR UPDATE SKIP LOCKED)
RETURNING order_id;

-- Deadlock: A locks 1 then 2, B locks 2 then 1
-- → ERROR: deadlock detected (one is aborted)

LOCK TABLE products IN SHARE MODE;  -- whole table
SELECT pg_advisory_xact_lock(42);   -- app mutex
```

CODE EXPLAINED

- → `FOR UPDATE` blocks other writers and other `FOR UPDATE` s
- → `NOWAIT` raises an error if the row is already locked
- → `SKIP LOCKED` hands each worker an unlocked row
- → `pg_advisory_xact_lock` is released at commit

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`FOR UPDATE / SHARE`, `NOWAIT`, `SKIP LOCKED`, advisory
MySQL	`FOR UPDATE / SHARE`, `NOWAIT`, `SKIP LOCKED`; `GET_LOCK()`
SQL Server	hints `WITH (UPDLOCK, ROWLOCK, READPAST)`
SQLite	no row locks; one writer (`BEGIN IMMEDIATE`)

IN AN ORM

```ts
await prisma.$transaction(async (tx) => {
  const [p] = await tx.$queryRaw`
    SELECT stock FROM products
    WHERE  product_id = ${id} FOR UPDATE`;
  // check p.stock, then update with tx
});
```

ROW LOCK MODES (PG)

CLAUSE	BLOCKS
`FOR UPDATE`	updates, deletes, other row locks
`FOR NO KEY UPDATE`	same, but allows FK checks (`UPDATE` default)
`FOR SHARE`	updates and deletes; other `FOR SHARE` OK
`FOR KEY SHARE`	only deletes and key changes

AVOIDING DEADLOCKS

DO	WHY
`lock rows in a fixed order (by id)`	no circular waits
`keep transactions short`	fewer overlapping locks
`index FK columns`	avoid wide lock scans
`retry on 40P01`	the victim can safely rerun
`lock_timeout`	give up instead of hanging

⚠ COMMON MISTAKES

- Read, decide in the app, write: two buyers get the last item
- Locking rows in different orders in different code paths
- `LOCK TABLE` on a busy table during peak hours

✓ TIPS & BEST PRACTICES

- Prefer one atomic `UPDATE … WHERE` over read-then-write
- Use `SKIP LOCKED` for queues instead of polling flags
- Watch `pg_locks` / `pg_stat_activity` when things hang

QUICK SUMMARY

- `FOR UPDATE` · `FOR SHARE`
- `NOWAIT` · `SKIP LOCKED`
- Deadlock → one aborts; retry

Index Basics

FINDING ROWS WITHOUT SCANNING

An **index** is a separate, sorted structure (usually a **B-tree**) that maps column values to rows, so the database can jump to matches instead of reading the whole table. It speeds up `WHERE`, joins, `ORDER BY` and `MIN`/`MAX`, at the cost of **slower writes** and extra disk. A **composite index** sorts by several columns in order; **`INCLUDE`** adds payload columns so a query can be answered from the index alone. Primary keys and unique constraints create indexes automatically.

INDEX FACTS

- Every `INSERT` / `UPDATE` maintains every index
- Foreign keys are **not** indexed automatically (PG)
- Small tables don't need indexes; scans are fine
- Confirm use with `EXPLAIN` (sheet 73)

NOSQL COMPARISON

- MongoDB: `createIndex({ customerId: 1, orderDate: -1 })`
- Same compound-prefix rule and the same write cost
- `_id` is indexed automatically

CREATING INDEXES

```
CREATE INDEX orders_customer_id_idx
  ON orders (customer_id);

-- Composite: equality column first, then sort column
CREATE INDEX orders_cust_date_idx
  ON orders (customer_id, order_date DESC);

SELECT order_id, order_date
FROM   orders
WHERE  customer_id = 42
ORDER  BY order_date DESC
LIMIT  10;                      -- index finds + sorts

-- Unique index (like a UNIQUE constraint)
CREATE UNIQUE INDEX products_name_uq
  ON products (lower(name));

-- Covering: extra columns for index-only scans
CREATE INDEX orders_status_idx
  ON orders (status) INCLUDE (total);

-- Build without blocking writes (PostgreSQL)
CREATE INDEX CONCURRENTLY customers_city_idx
  ON customers (city);

DROP INDEX IF EXISTS customers_city_idx;
```

CODE EXPLAINED

- → `customer_id = 42` narrows to one customer's entries
- → `order_date DESC` in the index means no sort step
- → `INCLUDE (total)` lets `SELECT total … WHERE status = …` skip the table
- → `CONCURRENTLY` is slower but doesn't lock out writes

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	B-tree default; `INCLUDE`; `CONCURRENTLY`
MySQL	InnoDB rows are clustered by the PK; FKs auto-indexed
SQL Server	one clustered index per table; `INCLUDE`; `ONLINE = ON`
SQLite	B-tree indexes; no `INCLUDE`

IN AN ORM

```
model Order {
  customerId Int
  orderDate  DateTime
  @@index([customerId, orderDate(sort: Desc)])
}
```

WHAT A B-TREE HELPS

QUERY SHAPE	EXAMPLE
equality	`WHERE email = 'a@x.com'`
range	`WHERE order_date >= '2026-10-01'`
prefix match	`WHERE name LIKE 'Desk%'`
sorting	`ORDER BY order_date DESC LIMIT 10`
min / max	`SELECT max(order_date)`
joins	`JOIN orders o ON o.customer_id = c.customer_id`

WHAT INDEXES COST

COST	EFFECT
writes	every index updated per row
space	often 10–50% of the table each
memory	hot indexes compete for cache
unused ones	pure cost: find and drop them

⚠ COMMON MISTAKES

- Indexing every column "just in case"
- Composite index in the wrong column order
- Forgetting FK indexes, making joins and deletes slow

✓ TIPS & BEST PRACTICES

- Index what your `WHERE`, `JOIN` and `ORDER BY` use
- PG: `pg_stat_user_indexes.idx_scan = 0` finds unused indexes
- Use `CONCURRENTLY` on production tables

QUICK SUMMARY

- `CREATE [UNIQUE] INDEX name ON t (cols)`
- Faster reads, slower writes
- Composite order matters

Index Types & Strategies

partial index · expression index · leftmost prefix · GIN · GiST · BRIN · hash

CHOOSING THE RIGHT INDEX

A composite B-tree helps queries that use its **leftmost columns**: `(customer_id, status)` serves `customer_id` alone or both, but not `status` alone. A **partial index** covers only rows matching a WHERE, so it stays small. An **expression index** stores `lower(email)` so a query on `lower(email)` can use it. PostgreSQL also offers other index **types**: **GIN** for arrays, JSONB and text search, **GiST** for ranges and geometry, **BRIN** for huge naturally ordered tables, **Hash** for equality only.

STRATEGY FACTS

- Equality columns first, then range / sort columns
- The query must repeat the partial index's condition
- The expression in `WHERE` must match the index exactly
- PG 18 skip scan can use `(a, b)` for `b` alone, sometimes

NOSQL COMPARISON

- MongoDB: `partialFilterExpression` = partial index
- Also text, wildcard, `2dsphere` and TTL indexes
- Compound indexes follow the same prefix rule

INDEX STRATEGIES

```
-- Partial: only the rows you query
CREATE INDEX orders_pending_idx
  ON orders (order_date)
  WHERE status = 'pending';

-- Expression: matches lower(email) lookups
CREATE INDEX customers_email_lower_idx
  ON customers (lower(email));
SELECT * FROM customers
WHERE  lower(email) = 'ada@example.com';

-- Leftmost prefix
CREATE INDEX orders_cust_status_idx
  ON orders (customer_id, status);
-- WHERE customer_id = 1                  ✓
-- WHERE customer_id = 1 AND status = 'x' ✓
-- WHERE status = 'paid'                  ✗ mostly

-- GIN trigram: fast '%lamp%' searches
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX products_name_trgm
  ON products USING gin (name gin_trgm_ops);

-- BRIN: tiny index for huge, date-ordered tables
CREATE INDEX orders_date_brin
  ON orders USING brin (order_date);
```

CODE EXPLAINED

→ The partial index skips every shipped and paid order

→ `WHERE lower(email) = …` matches the indexed expression

→ A phone book sorted by last name can't find first names

→ BRIN stores min/max per block range: KBs, not GBs

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	B-tree, Hash, GIN, GiST, SP-GiST, BRIN; partial + expression
MySQL	B-tree, `FULLTEXT`, `SPATIAL`; functional (8.0.13+); no partial
SQL Server	filtered indexes (`WHERE`), columnstore, computed columns
SQLite	partial and expression indexes; B-tree only

IN AN ORM

```
model Product {
  name String
  @@index([name(ops: raw("gin_trgm_ops"))], type: Gin)
}
// partial / expression indexes: edit the migration SQL
```

INDEX TYPES (POSTGRESQL)

TYPE	GOOD FOR	OPERATORS
B-tree	most columns; sorting	`= < > BETWEEN LIKE 'a%'`
Hash	equality only	`=`
GIN	arrays, jsonb, full text, trigram	`@> ? @@ %`
GiST	ranges, geometry, nearest	`&& <->`
BRIN	huge append-only tables	`< > BETWEEN`

STRATEGY CHECKLIST

SITUATION	TRY
`most rows excluded by one flag`	partial index
`function in WHERE`	expression index
`filter + sort`	composite `(filter, sort)`
`query reads few columns`	`INCLUDE` / covering index
`'%word%' search`	trigram GIN
`append-only log by time`	BRIN

⚠ COMMON MISTAKES

- Putting a range column before an equality column
- Partial index whose condition queries don't repeat
- GIN indexes on write-heavy tables without measuring

✓ TIPS & BEST PRACTICES

- Design indexes from real queries, not from tables
- One good composite beats three single-column indexes
- Re-check plans after adding an index (sheet 73)

QUICK SUMMARY

- Leftmost prefix rule
- Partial · expression · covering
- GIN · GiST · BRIN · Hash

Reading Query Plans

EXPLAIN · EXPLAIN ANALYZE · Seq Scan · Index Scan · Nested Loop · Hash Join · cost

HOW THE DATABASE RUNS YOUR QUERY

EXPLAIN shows the **plan** the optimizer chose: a tree of steps such as scans, joins and sorts, each with an estimated **cost** and **row count**. **EXPLAIN ANALYZE** actually runs the query and adds real times and row counts, so you can compare estimates with reality. Read it inside-out: the most indented nodes run first. Large gaps between estimated and actual rows usually mean stale **statistics** (sheet 74).

PLAN FACTS

- `ANALYZE` executes the query: careful with writes
- Wrap write statements in `BEGIN … ROLLBACK`
- Cost is in arbitrary units; compare, don't read literally
- A seq scan isn't always bad: small tables, big results

NOSQL COMPARISON

- MongoDB: `db.orders.find({ … }).explain('executionStats')`
- `COLLSCAN` = full scan, `IXSCAN` = index
- Compare `totalDocsExamined` with `nReturned`

EXPLAIN

```
EXPLAIN
SELECT * FROM orders WHERE customer_id = 42;
-- Index Scan using orders_customer_id_idx on orders
--   (cost=0.29..8.31 rows=5 width=40)
--   Index Cond: (customer_id = 42)

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.name, sum(o.total)
FROM   customers c
JOIN   orders o USING (customer_id)
WHERE  o.order_date >= '2026-10-01'
GROUP  BY c.name;
-- HashAggregate (actual time=4.1..4.3 rows=120)
--   -> Hash Join (rows=900) (actual rows=1150)
--        Hash Cond: (o.customer_id = c.customer_id)
--        -> Seq Scan on orders o
--             Filter: (order_date >= '2026-10-01')
--             Rows Removed by Filter: 48850
--        -> Hash
--             -> Seq Scan on customers c
-- Planning Time: 0.3 ms
-- Execution Time: 4.6 ms        (output trimmed)
```

CODE EXPLAINED

→ `cost=0.29..8.31` — startup cost .. total cost

→ `rows=900` estimate vs `actual rows=1150` : close enough

→ 48,850 rows removed by a filter → an index candidate

→ `Hash Join` builds a hash of customers, probes with orders

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`
MySQL	`EXPLAIN`, `EXPLAIN ANALYZE` (8.0.18+), `FORMAT=TREE`
SQL Server	execution plans in SSMS; `SET STATISTICS IO, TIME ON`
SQLite	`EXPLAIN QUERY PLAN SELECT …`

IN AN ORM

```
const prisma = new PrismaClient({
  adapter, log: [{ emit: 'event', level: 'query' }],
});
prisma.$on('query', (e) => console.log(e.duration, e.query));
// copy a slow query and EXPLAIN ANALYZE it in psql
```

PLAN NODES

NODE	MEANS	WATCH FOR
`Seq Scan`	read the whole table	big table + few rows wanted
`Index Scan`	walk index, fetch rows	many rows → random I/O
`Index Only Scan`	answer from the index	good; needs vacuumed table
`Bitmap Heap Scan`	collect matches, read pages	medium selectivity
`Nested Loop`	inner lookup per outer row	large outer side
`Hash Join`	hash one side, probe other	hash spilling to disk
`Sort / HashAggregate`	order / group rows	`external merge` = disk

NUMBERS TO READ

FIELD	TELLS YOU
`cost=a..b`	startup .. total estimate
`rows=`	estimated rows out
`actual time=a..b`	ms to first .. last row
`loops=`	times the node ran (multiply!)
`Rows Removed by Filter`	wasted reads
`Buffers: shared hit / read`	cache hits / disk reads

⚠ COMMON MISTAKES

- Tuning from `EXPLAIN` estimates without `ANALYZE`
- Running `EXPLAIN ANALYZE DELETE …` outside a rolled-back transaction
- Ignoring `loops` : per-loop time × loops = real time

✓ TIPS & BEST PRACTICES

- Paste plans into explain.dalibo.com or explain.depesz.com
- Fix the node with the biggest actual time first
- Compare plans before and after each change

QUICK SUMMARY

- `EXPLAIN` — plan · `ANALYZE` — real run
- Read inner nodes first
- Estimates vs actuals → statistics

Query Optimization

MAKING QUERIES FAST

Most slow queries come from a few causes. **Non-SARGable** predicates wrap the column in a function or arithmetic, so its index can't be used; rewrite them as ranges on the bare column. **SELECT ∗** drags unneeded columns and blocks index-only scans. The **N+1** pattern runs one query per row from app code instead of one set-based query. The planner relies on **statistics**, refreshed by **ANALYZE**; PostgreSQL's **VACUUM** clears dead row versions so tables and indexes stay lean.

OPTIMIZATION FACTS

- Measure first: `EXPLAIN ANALYZE` (sheet 73)
- SARGable = Search ARGument able: usable by an index
- Autovacuum normally runs `VACUUM` and `ANALYZE` for you
- Fewer round trips beat faster single queries

NOSQL COMPARISON

- Same rules in MongoDB: index the filter fields
- Project only needed fields: `find(q, { name: 1 })`
- Avoid unanchored `$regex` and `$where` on big collections

COMMON FIXES

```sql
-- ✗ function on the column: index unusable
SELECT order_id FROM orders
WHERE  extract(YEAR FROM order_date) = 2026;
-- ✓ range on the bare column
SELECT order_id FROM orders
WHERE  order_date >= '2026-01-01'
  AND  order_date <  '2027-01-01';

-- ✗ math on the column     ✓ math on the constant
WHERE  price * 1.08 > 100    -- →  WHERE price > 100 / 1.08

-- ✗ every column            ✓ only what's needed
SELECT * FROM orders WHERE customer_id = 42;
SELECT order_id, total FROM orders WHERE customer_id = 42;

-- ✗ N+1 from app code: one query per customer
SELECT * FROM orders WHERE customer_id = $1;  -- × 500
-- ✓ one query for all of them
SELECT * FROM orders WHERE customer_id = ANY ($1);

-- Fresh statistics; clean up dead rows (PG)
ANALYZE orders;
VACUUM (ANALYZE) orders;
```

CODE EXPLAINED

→ `extract(…)` hides `order_date` from its index

→ Moving the math to the constant side keeps the index usable

→ `= ANY ($1)` takes an array of ids in one round trip

→ `VACUUM (ANALYZE)` — reclaim space and refresh stats

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`ANALYZE`, `VACUUM`; autovacuum on by default
MySQL	`ANALYZE TABLE`, `OPTIMIZE TABLE`
SQL Server	`UPDATE STATISTICS`; index rebuild / reorganize
SQLite	`ANALYZE`, `VACUUM`, `PRAGMA optimize`

IN AN ORM

```sql
// ✗ N+1: one query per customer
for (const c of customers)
  c.orders = await prisma.order.findMany(
    { where: { customerId: c.id } });
// ✓ one batched query for all customers
await prisma.customer.findMany({ include: { orders: true } });
```

SARGABLE REWRITES

✗ SLOW	✓ INDEX-FRIENDLY
`date(ts) = '2026-10-18'`	`ts >= '2026-10-18' AND ts < '2026-10-19'`
`lower(email) = 'x'`	expression index on `lower(email)` (sheet 72)
`name LIKE '%lamp'`	trigram index or full-text (sheet 80)
`coalesce(x, 0) = 5`	`x = 5`
`col::text = '42'`	`col = 42`
`a = 1 OR b = 2`	`UNION` of two indexed queries

CHECKLIST

CHECK	LOOK AT
is an index used?	`EXPLAIN` node types
estimates right?	estimated vs actual rows
round trips?	query log / APM
fetching too much?	`SELECT *`, missing `LIMIT`
table bloated?	`n_dead_tup` in `pg_stat_user_tables`

⚠ COMMON MISTAKES

- Adding indexes before reading the plan
- Fixing N+1 with caching instead of a better query
- Turning off autovacuum to "save resources"

✓ TIPS & BEST PRACTICES

- Enable `pg_stat_statements` to find the costliest queries
- Batch writes; paginate reads (sheet 16)
- Re-run `ANALYZE` after big loads or deletes

QUICK SUMMARY

- Keep predicates SARGable
- Select only needed columns
- Kill N+1; keep stats fresh

Users, Roles & Privileges

CREATE ROLE · CREATE USER · GRANT · REVOKE · least privilege · ALTER DEFAULT PRIVILEGES

WHO CAN DO WHAT

Databases control access with **roles**. A role that can log in is a user; a role that can't is a group you grant to others. **GRANT** gives privileges (SELECT, INSERT, UPDATE, DELETE, EXECUTE …) on tables, schemas, sequences or functions; **REVOKE** takes them back. Follow **least privilege**: the app's login gets only the data access it needs, migrations run as a separate owner role, and analysts get read-only access.

PRIVILEGE FACTS

- Grants on existing tables don't cover future ones
- `ALTER DEFAULT PRIVILEGES` handles new tables
- Table owners and superusers bypass grants
- Row-level security filters rows per user (PG)

NOSQL COMPARISON

- MongoDB RBAC: `db.createUser({ user, pwd, roles: [{ role: 'read', db: 'shop' }] })`
- Built-in roles: `read`, `readWrite`, `dbAdmin`
- Custom roles can grant actions per collection

ROLES & GRANTS

```
-- Group role: read-only access to the schema
CREATE ROLE readonly NOLOGIN;
GRANT CONNECT ON DATABASE shop TO readonly;
GRANT USAGE  ON SCHEMA public TO readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public
  TO readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO readonly;   -- future

-- A login that inherits it
CREATE ROLE analyst LOGIN PASSWORD 'change-me';
GRANT readonly TO analyst;

-- App login: data only, no DDL
CREATE ROLE app_rw LOGIN PASSWORD 'from-vault';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON ALL TABLES IN SCHEMA public TO app_rw;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public
  TO app_rw;
REVOKE DELETE ON orders FROM app_rw;

-- Only some columns
GRANT SELECT (customer_id, name, city)
  ON customers TO support;
```

CODE EXPLAINED

- → `NOLOGIN` — a group role; nobody signs in as it
- → `USAGE ON SCHEMA` is needed before any table access
- → Sequence `USAGE` lets inserts take identity values
- → Column grants hide `email` from the support role

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	roles only (`CREATE USER` = role + `LOGIN`); RLS policies
MySQL	`CREATE USER 'app'@'%' IDENTIFIED BY …`; roles (8.0+)
SQL Server	logins (server) + users (database); `db_datareader`
SQLite	no users: protect the file instead

IN AN ORM

```
# The app connects as the least-privileged role
DATABASE_URL="postgresql://app_rw:…@db:5432/shop"
# Run `prisma migrate deploy` with an owner role,
# in CI, not with the app's credentials
```

COMMON PRIVILEGES

PRIVILEGE	ALLOWS
`SELECT`	read rows (and columns, if listed)
`INSERT · UPDATE · DELETE`	change data
`TRUNCATE · REFERENCES`	empty table · create FKs to it
`USAGE (schema, sequence)`	look inside · take values
`CREATE (schema, database)`	create objects in it
`EXECUTE (function)`	call it
`CONNECT (database)`	open a connection

LEAST-PRIVILEGE SETUP

ROLE	GETS
`owner / migrator`	owns objects; runs DDL
`app_rw`	DML on tables, sequence usage
`readonly`	`SELECT` (and future tables)
`support`	selected columns or a view (sheet 64)
`per-tenant users`	row-level security policies

⚠ COMMON MISTAKES

- The app connecting as a superuser or owner
- Granting on `ALL TABLES` but forgetting future tables
- Shared logins that make audits meaningless

✓ TIPS & BEST PRACTICES

- Grant to group roles, then add people to groups
- Keep passwords in a secret store; rotate them
- PG RLS: `CREATE POLICY … USING (tenant_id = …)`

QUICK SUMMARY

- Roles = users + groups
- `GRANT` / `REVOKE` privileges
- Least privilege per role

SQL Injection & Parameterized Queries

parameterized query · $1 · ? · @param · PREPARE · EXECUTE · dynamic SQL · quote_ident

NEVER BUILD SQL FROM INPUT

SQL injection happens when user input is pasted into SQL text, so the input can change the query itself: `' OR '1'='1` returns every row, and worse inputs drop tables. The fix is **parameters**: the SQL text contains placeholders ($1, ?, `@email`) and the values travel **separately**, so they are always treated as data. Parameters work for **values only**; table or column names must come from a whitelist or be quoted with **`quote_ident` / `format('%I')`**.

INJECTION FACTS

- Escaping by hand is fragile; parameters aren't
- Prepared statements also cache the plan
- Dynamic SQL in functions is just as vulnerable
- Least privilege limits the damage (sheet 75)

NOSQL COMPARISON

- NoSQL injection: a body of `{ "email": { "$ne": null } }` matches everyone
- Cast inputs to expected types before building filters
- Reject keys starting with `$`; avoid `$where`

PARAMETERS, NOT CONCATENATION

```
-- ✗ App code that builds SQL from input:
--   "… WHERE email = '" + input + "'"
--   input:  ' OR '1'='1   → every customer
--   input:  '; DROP TABLE orders; --

-- ✓ Placeholders; values are sent separately
SELECT * FROM customers WHERE email = $1;      -- PG
SELECT * FROM customers WHERE email = ?;       -- MySQL
SELECT * FROM customers WHERE email = @email; -- SQL Svr

-- Server-side prepared statement
PREPARE find_customer (text) AS
  SELECT * FROM customers WHERE email = $1;
EXECUTE find_customer('ada@example.com');
DEALLOCATE find_customer;

-- Dynamic SQL inside the database
CREATE FUNCTION count_rows(tbl text) RETURNS bigint
LANGUAGE plpgsql AS $$
DECLARE n bigint;
BEGIN
  EXECUTE format('SELECT count(*) FROM %I', tbl) INTO n;
  RETURN n;
END $$;
-- %I quotes identifiers; pass values with USING
```

CODE EXPLAINED

→ The injected quote closes the string and adds `OR true`

→ A placeholder value can't change the query's structure

→ `PREPARE` parses once; `EXECUTE` supplies values

→ `format('%I', tbl)` = `quote_ident(tbl)`

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`$1, $2`; `PREPARE` / `EXECUTE`; `format('%I %L')`
MySQL	`?`; `PREPARE s FROM '…'; EXECUTE s USING @v`
SQL Server	`@name`; `sp_executesql N'…', N'@e nvarchar(255)', @e`
SQLite	`?`, `?1`, `:name`, `@name`, `$name`

IN AN ORM

```
// ✓ tagged template → sent as $1
await prisma.$queryRaw`
  SELECT * FROM customers WHERE email = ${email}`;
// ✗ string building → injectable
await prisma.$queryRawUnsafe(
  `SELECT * FROM customers WHERE email = '${email}'`);
```

PLACEHOLDERS BY DRIVER

DRIVER	STYLE
`node-postgres (pg)`	`$1, $2`
`mysql2`	`?`
`Python DB-API (psycopg)`	`%s · %(name)s`
`JDBC`	`?`
`.NET (SqlClient)`	`@name`
`Go database/sql`	`$1` (pq) · `?` (mysql)

NOT A PARAMETER? DO THIS

DYNAMIC PART	SAFE APPROACH
`table / column name`	whitelist, or `quote_ident` / `%I`
`sort direction`	map `'asc'`/`'desc'` to fixed SQL
`IN list`	one array param: `= ANY ($1)`
`LIMIT / OFFSET`	a normal parameter
`LIKE pattern`	parameter + escape `%` and `_`

⚠ COMMON MISTAKES

- Template strings with `${…}` passed to raw-unsafe APIs
- Escaping quotes by hand and missing a case
- Trusting "internal" input (headers, cookies, CSV files)

✓ TIPS & BEST PRACTICES

- Ban `$queryRawUnsafe` and friends in code review
- Run the app with a role that can't drop tables
- Test with `'`, `--` and `;` in every input

QUICK SUMMARY

- Values → parameters, always
- Names → whitelist / `quote_ident`
- `$1` · `?` · `@name`

Backup & Restore

pg_dump · pg_restore · mysqldump · .backup · BACKUP DATABASE · point-in-time recovery

GETTING DATA BACK

A **logical backup** (`pg_dump`, `mysqldump`) writes SQL or an archive you can restore anywhere, even into a newer version. A **physical backup** copies the data files and, together with the transaction log (PostgreSQL WAL, MySQL binlog, SQL Server log backups), allows **point-in-time recovery**: restore to the second before a bad DELETE. A backup only counts once you've **restored it successfully**.

BACKUP FACTS

- Logical dumps are consistent snapshots
- Roles and settings need separate dumps (`pg_dumpall -g`)
- Big databases: physical backups restore faster
- Store copies off the database server

NOSQL COMPARISON

- MongoDB: `mongodump` / `mongorestore` (BSON)
- Atlas: continuous backups with point-in-time restore
- `mongoexport` isn't a backup: JSON loses types

BACKUP & RESTORE

```
# PostgreSQL: one database, compressed custom format
pg_dump -Fc -d shop -f shop.dump
pg_restore -d shop_restored --clean --if-exists shop.dump
pg_dump -d shop --schema-only > schema.sql
pg_dump -d shop -t orders --data-only > orders.sql
pg_dumpall --globals-only > roles.sql

# MySQL
mysqldump --single-transaction --routines shop > shop.sql
mysql shop_restored < shop.sql

# SQLite: safe copy of a live database
sqlite3 shop.db ".backup 'shop-2026-10-18.db'"
sqlite3 shop.db "VACUUM INTO 'shop-compact.db'"

# PostgreSQL point-in-time recovery (config)
#   archive_mode = on
#   archive_command = 'cp %p /archive/%f'
#   restore: recovery_target_time = '2026-10-18 09:59'

# SQL Server (T-SQL, run in sqlcmd)
# BACKUP DATABASE shop TO DISK = '/backup/shop.bak';
# RESTORE DATABASE shop FROM DISK = '/backup/shop.bak'
#   WITH REPLACE;
```

CODE EXPLAINED

- → `-Fc` — custom format: compressed, selective restore
- → `--clean --if-exists` drops objects before recreating
- → `--single-transaction` — consistent InnoDB dump, no locks
- → `.backup` copies safely while the database is in use

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`pg_dump` / `pg_restore`; PITR = base backup + WAL
MySQL	`mysqldump`; MySQL Shell `util.dumpInstance()`; binlog PITR
SQL Server	full / differential / log; `RESTORE … WITH STOPAT`
SQLite	`.backup`, `VACUUM INTO`, or copy the file when idle

IN AN ORM

```
# Prisma moves schema, not data
npx prisma migrate deploy        # schema on a fresh DB
pg_restore --data-only -d shop shop.dump
```

BACKUP TYPES

TYPE	WHAT	RESTORES TO
`logical dump`	SQL / archive of objects + rows	the dump time
`physical / base`	copy of data files	the backup time
`+ log archive`	WAL / binlog / log backups	any point in time
`disk snapshot`	volume snapshot (cloud)	the snapshot time
`replica`	live copy on another server	not a backup: copies mistakes

GOOD PRACTICE

RULE	WHY
`3-2-1`	3 copies, 2 media, 1 off-site
`test restores`	untested backups fail when needed
`encrypt`	dumps contain all your data
`automate + monitor`	alerts when a job fails
`document RPO / RTO`	how much loss, how long down
`keep several generations`	a corrupt night doesn't overwrite the good one

⚠ COMMON MISTAKES

- Never trying a restore until the outage
- Treating a replica as a backup
- Dumping data but not roles, extensions or settings

✓ TIPS & BEST PRACTICES

- Restore into a scratch database on a schedule
- Keep dumps from the version you'll restore with
- Use managed PITR in the cloud when you can

QUICK SUMMARY

- Logical: `pg_dump` · `mysqldump` · `.backup`
- Physical + logs → point in time
- Test the restore

JSON

DOCUMENTS INSIDE ROWS

JSON columns hold flexible attributes that don't deserve their own columns: product specs, settings, raw API payloads. PostgreSQL's **jsonb** stores parsed, binary JSON that can be indexed; **->** returns a JSON value, **->>** returns text, and **@>** tests containment. Change part of a document with **jsonb_set** or ||. The SQL standard's **JSON_VALUE** / **JSON_QUERY** / **JSON_TABLE** work in PostgreSQL 17+, MySQL and SQL Server.

JSON FACTS

- `jsonb` drops duplicate keys and whitespace
- Keep fields you filter or join on as real columns
- A GIN index speeds up `@>` and `?`
- Cast `->>` text before comparing numbers

NOSQL COMPARISON

- This is MongoDB's home ground: documents are the data
- `find({ 'size.w': { $gt: 20 } })` queries nested fields
- `$set: { 'size.w': 35 }` updates one field in place

JSONB

```
ALTER TABLE products ADD COLUMN attrs JSONB;
UPDATE products
SET    attrs = '{"color": "black",
                 "size": {"w": 30, "h": 45},
                 "tags": ["desk", "led"]}'
WHERE  product_id = 7;

SELECT attrs -> 'size'                 AS size_json,
       attrs ->> 'color'               AS color,
       attrs #>> '{size,w}'            AS width,
       (attrs -> 'size' ->> 'h')::int  AS height,
       JSON_VALUE(attrs, '$.size.w')   AS w_std  -- PG 17
FROM   products
WHERE  product_id = 7;

-- Containment and key tests (GIN-indexable)
SELECT name FROM products
WHERE  attrs @> '{"color": "black"}';
SELECT name FROM products WHERE attrs -> 'tags' ? 'led';

UPDATE products
SET    attrs = jsonb_set(attrs, '{color}', '"white"')
WHERE  product_id = 7;
UPDATE products SET attrs = attrs - 'tags';
```

CODE EXPLAINED

- `->` keeps JSON; `->>` gives text
- `#>>` follows a path: `{size,w}` = `size.w`
- `@>` — does the document contain this fragment?
- `jsonb_set` replaces one path; `- 'key'` removes a key

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`jsonb`, `-> ->> @> ?`; `JSON_TABLE` (17+)
MySQL	`JSON` type; `->`, `->>`, `JSON_EXTRACT`, `JSON_TABLE`
SQL Server	`JSON_VALUE`, `OPENJSON`; native `json` type (2025)
SQLite	`json` / `jsonb` functions (3.45+), `->` and `->>`

IN AN ORM

```
await prisma.product.findMany({
  where: { attrs: { path: ['color'], equals: 'black' } },
});
await prisma.product.update({
  where: { id: 7 },
  data: { attrs: { color: 'white', size: { w: 30 } } },
});   // replaces the whole JSON value
```

JSONB OPERATORS

OPERATOR	DOES	EXAMPLE
`-> ->>`	get key as JSON / text	`attrs ->> 'color'`
`#> #>>`	get path as JSON / text	`attrs #>> '{size,w}'`
`@>`	contains	`attrs @> '{"color":"black"}'`
`? ?| ?&`	key exists / any / all	`attrs ? 'tags'`
`||`	merge objects	`attrs || '{"new":true}'`
`- #-`	remove key / path	`attrs - 'tags'`

JSON VS JSONB (PG)

	JSON	JSONB
stored as	text, as written	parsed binary
key order / duplicates	kept	normalized
indexing	no	GIN
operators	basic	full set
use	exact original text	almost always

⚠ COMMON MISTAKES

- Putting core, frequently filtered fields only inside JSON
- Comparing `->>` text to numbers without a cast
- Using `json` when you meant `jsonb`

✓ TIPS & BEST PRACTICES

- Promote hot JSON keys to generated columns + indexes
- Validate shape with `CHECK (jsonb_typeof(attrs) = 'object')`
- Index with `USING gin (attrs)` for `@>` and `?` lookups

QUICK SUMMARY

- `jsonb` — indexable JSON
- `->` JSON · `->>` text · `@>` contains
- `jsonb_set` · `||` · `-` to modify

Arrays & Generated Series

LISTS IN A COLUMN, ROWS FROM NOTHING

PostgreSQL columns can hold **arrays** (`text[]`, `int[]`). Test membership with **`= ANY (arr)`**, containment with **`@>`** and overlap with **`&&`**. **`unnest`** turns an array into rows, and **`array_agg`** turns rows back into an array (sheet 31). Arrays are perfect for passing a list as **one parameter**. **`generate_series`** produces rows from nothing: number ranges, or one row per day for reports.

ARRAY FACTS

- Arrays are **1-based**: `tags[1]` is the first
- Out-of-range index → NULL, not an error
- Arrays suit small tag-like lists, not relations (sheet 62)
- Other engines use JSON arrays instead

NOSQL COMPARISON

- MongoDB arrays are native: `find({ tags: 'led' })` matches an element
- `$all`, `$elemMatch`, `$push`, `$addToSet`
- `$unwind` = `unnest`; `$push` in `$group` = `array_agg`

ARRAYS & SERIES

```
ALTER TABLE products ADD COLUMN tags TEXT[] DEFAULT '{}';
UPDATE products SET tags = ARRAY['desk', 'led']
WHERE  product_id = 7;

SELECT name, tags[1] AS first_tag,
       cardinality(tags) AS n_tags
FROM   products;

SELECT name FROM products WHERE 'led' = ANY (tags);
SELECT name FROM products WHERE tags @> ARRAY['desk'];
SELECT name FROM products
WHERE  tags && ARRAY['sale', 'new'];    -- any overlap

-- Array → rows
SELECT p.name, t.tag
FROM   products p, unnest(p.tags) AS t(tag);

-- A list as one parameter
SELECT * FROM orders WHERE order_id = ANY ($1::int[]);

-- Rows from nothing
SELECT generate_series(1, 5);              -- 1 … 5
SELECT d::date
FROM   generate_series(DATE '2026-10-01',
                       DATE '2026-10-07',
                       INTERVAL '1 day') AS d;
```

CODE EXPLAINED

→ `'{}'` is an empty array literal

→ `= ANY (tags)` — true if any element equals `'led'`

→ `@>` all listed present · `&&` at least one shared

→ `$1::int[]` — the driver sends one array value

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	native arrays; `ANY`, `unnest`, `@>`, `&&`
MySQL	JSON arrays: `JSON_TABLE`, `MEMBER OF()` (8.0.17+)
SQL Server	`STRING_SPLIT`, `OPENJSON`; `GENERATE_SERIES` (2022)
SQLite	`json_each()`; recursive CTE for series (sheet 42)

IN AN ORM

```
// schema: tags String[]  (PostgreSQL only)
await prisma.product.findMany({
  where: { tags: { has: 'led' } },     // hasSome, hasEvery
});
await prisma.order.findMany({
  where: { id: { in: [101, 102, 103] } },  // → = ANY($1)
});
```

ARRAY TOOLKIT

WRITE	MEANS
`ARRAY[1, 2, 3]` · `'{1,2,3}'`	array literal
`arr[1]` · `arr[2:3]`	element · slice
`x = ANY (arr)`	x is in arr
`arr @> ARRAY[x]`	arr contains all of these
`arr && ARRAY[x, y]`	arr shares any element
`array_append(arr, x)`	add an element
`cardinality(arr)`	element count

GENERATE_SERIES USES

NEED	CALL
numbers 1–100	`generate_series(1, 100)`
every 5th	`generate_series(0, 100, 5)`
every day this month	`generate_series(d1, d2, '1 day')`
hourly buckets	`… INTERVAL '1 hour'`
test data	`INSERT … SELECT … FROM generate_series(1, 1e5)`

⚠ COMMON MISTAKES

- Using arrays where a junction table belongs
- Off-by-one: arrays start at 1
- `NOT ('x' = ANY (arr))` with NULL elements → UNKNOWN

✓ TIPS & BEST PRACTICES

- GIN-index array columns queried with `@>` or `&&`
- Pass id lists as one array parameter, not many `?`s
- Left-join a series to fill reporting gaps (sheet 34)

QUICK SUMMARY

- `text[]` · `= ANY` · `@>` · `&&`
- `unnest` ↔ `array_agg`
- `generate_series` for numbers and dates

Full-Text Search

SEARCHING WORDS, NOT CHARACTERS

`LIKE '%lamp%'` matches characters; **full-text search** matches **words**. PostgreSQL turns text into a **`tsvector`**: normalized word stems ("jumped" → `jump`) with positions, minus stop words like "the". A **`tsquery`** describes what to find (`desk & lamp`), and **`@@`** tests a match. Store the vector in a generated column, index it with **GIN**, and sort by **`ts_rank`**. MySQL uses `FULLTEXT` + `MATCH … AGAINST`, SQLite the **FTS5** extension.

SEARCH FACTS

- The language config (`'english'`) controls stemming
- `websearch_to_tsquery` accepts Google-style input
- `setweight` ranks title hits above body hits
- Typos need trigram fuzzy matching (`pg_trgm`)

NOSQL COMPARISON

- MongoDB: `createIndex({ name: 'text' }) + find({ $text: { $search: 'desk lamp' } })`
- Atlas Search (Lucene): relevance, fuzzy, autocomplete

TSVECTOR & TSQUERY

```
SELECT to_tsvector('english',
                   'The quick brown foxes jumped');
-- 'brown':3 'fox':4 'jump':5 'quick':2

SELECT name FROM products
WHERE  to_tsvector('english', name)
       @@ to_tsquery('english', 'desk & lamp');

-- Store a weighted vector and index it
ALTER TABLE products ADD COLUMN search tsvector
  GENERATED ALWAYS AS (
    setweight(to_tsvector('english', name), 'A') ||
    setweight(to_tsvector('english',
              coalesce(category, '')), 'B')
  ) STORED;
CREATE INDEX products_search_gin
  ON products USING gin (search);
-- Ranked search with user-style input
SELECT name, ts_rank(search, q) AS rank
FROM   products,
       websearch_to_tsquery('english',
                            'led lamp -wood') AS q
WHERE  search @@ q
ORDER  BY rank DESC
LIMIT  10;
```

CODE EXPLAINED

→ Stop word "the" is dropped; "foxes" stems to `fox`

→ `&` = AND inside `to_tsquery`

→ Name matches (weight A) outrank category matches (B)

→ `-wood` excludes; quotes would require a phrase

DIALECT NOTES

ENGINE	SYNTAX / BEHAVIOR
PostgreSQL	`tsvector`, `tsquery`, `@@`, GIN; `pg_trgm` for typos
MySQL	`FULLTEXT` index; `MATCH (col) AGAINST ('…' IN BOOLEAN MODE)`
SQL Server	full-text catalog; `CONTAINS`, `FREETEXT`
SQLite	`CREATE VIRTUAL TABLE … USING fts5(…)`; `MATCH`, `bm25()`

IN AN ORM

```
// preview feature: fullTextSearchPostgres
await prisma.product.findMany({
  where: { name: { search: 'desk & lamp' } },
});
// ranked search with weights → $queryRaw
```

TSQUERY SYNTAX

WRITE	MATCHES
`desk & lamp`	both words
`desk | lamp`	either word
`!wood`	not wood
`led <-> lamp`	"led" right before "lamp"
`lam:*`	words starting with lam
`websearch: "led lamp" -wood`	phrase, minus wood

WHICH SEARCH?

NEED	USE
`exact prefix ('Desk%')`	B-tree + `LIKE`
`substring anywhere`	trigram GIN (`pg_trgm`)
`words, stemming, ranking`	full-text (`tsvector`)
`typo tolerance`	trigram `similarity()` / `%`
`large-scale relevance tuning`	a search engine (Elasticsearch, Meilisearch)

⚠ COMMON MISTAKES

- Using `LIKE '%word%'` on large text columns
- Mixing language configs between vector and query
- Computing `to_tsvector` per row at query time on big tables

✓ TIPS & BEST PRACTICES

- Store the vector in a generated column with a GIN index
- Use `websearch_to_tsquery` for user input
- `ts_headline` highlights matches for display

QUICK SUMMARY

- `tsvector @@ tsquery`
- GIN index + `ts_rank`
- MySQL `MATCH` · SQLite FTS5

Sheet 81

Quick Reference: Query Syntax

clause order · logical order · JOIN types · aggregates · window syntax · DML / DDL skeletons

CLAUSE ORDER

WRITTEN	RUNS	#
`SELECT … AS a`	5th	13
`FROM t JOIN u ON …`	1st	32
`WHERE cond`	2nd	14
`GROUP BY cols`	3rd	28
`HAVING agg_cond`	4th	29
`window functions`	after 5	44
`DISTINCT`	6th	17
`ORDER BY … LIMIT n`	7th, 8th	15

JOINS

SYNTAX	DOES	#
`JOIN … ON a = b`	matches only	32
`LEFT JOIN`	+ unmatched left	33
`FULL JOIN`	+ unmatched both	33
`CROSS JOIN`	every combination	34
`JOIN … USING (k)`	same-named key	32
`WHERE EXISTS (…)`	semi-join	36
`WHERE NOT EXISTS (…)`	anti-join	36
`JOIN LATERAL (…)`	subquery per row	37

FILTERS

SYNTAX	DOES	#
`= <> < > <= >=`	compare	18
`BETWEEN a AND b`	inclusive range	18
`IN (…) · NOT IN`	list membership	18
`LIKE 'a%' · '_b'`	pattern	19
`~ · ILIKE`	regex · no case (PG)	19
`IS [NOT] NULL`	null test	11
`IS DISTINCT FROM`	NULL-safe <>	11
`AND · OR · NOT`	AND binds first	14

AGGREGATES & GROUPING

SYNTAX	DOES	#
`count(*) · count(c)`	rows · non-NULL	27
`sum · avg · min · max`	skip NULLs	27
`count(DISTINCT c)`	unique values	17
`agg FILTER (WHERE …)`	conditional	29
`string_agg(c, ',')`	join text	31
`GROUP BY ROLLUP (a, b)`	subtotals	30

WINDOW SYNTAX

SYNTAX	DOES	#
`f() OVER ()`	whole result	44
`OVER (PARTITION BY g)`	per group	44
`OVER (… ORDER BY d)`	running / ranked	47
`… ROWS n PRECEDING`	moving frame	48
`row_number() · rank()`	ranking	45
`lag(x) · lead(x)`	prev · next row	46

SUBQUERIES & SETS

SYNTAX	DOES	#
`(SELECT avg(x) FROM t)`	scalar value	38
`FROM (SELECT …) AS d`	derived table	40
`WITH c AS (…) SELECT …`	CTE	41
`WITH RECURSIVE …`	hierarchy	42
`UNION [ALL]`	stack rows	43
`INTERSECT · EXCEPT`	common · difference	43

DML SKELETONS

SYNTAX	DOES	#
`INSERT INTO t (a, b) VALUES (1, 2), (3, 4)`	add rows	50
`INSERT INTO t (a) SELECT x FROM s`	copy rows	50
`UPDATE t SET a = a + 1 WHERE …`	change rows	51
`UPDATE t SET a = s.x FROM s WHERE …`	update from table	51
`DELETE FROM t WHERE …`	remove rows	52
`… ON CONFLICT (k) DO UPDATE SET a = EXCLUDED.a`	upsert	53
`… RETURNING *`	get changed rows	50

DDL & TRANSACTIONS

SYNTAX	DOES	#
`CREATE TABLE t (id … PRIMARY KEY, …)`	new table	55
`ALTER TABLE t ADD COLUMN c TEXT`	add column	59
`CREATE INDEX t_c_idx ON t (c)`	index	71
`CREATE VIEW v AS SELECT …`	saved query	64
`DROP TABLE IF EXISTS t`	remove table	59
`BEGIN; … COMMIT; / ROLLBACK;`	transaction	68
`SELECT … FOR UPDATE`	lock rows	70

NULL & CASE

SYNTAX	DOES	#
`coalesce(a, b, 'x')`	first non-NULL	26
`nullif(a, 0)`	NULL when equal	26
`CASE WHEN … THEN … END`	if / else	21
`x / nullif(y, 0)`	no divide error	26
`greatest(a, b)`	largest arg	26

STRINGS & DATES

SYNTAX	DOES	#
`lower(trim(s))`	normalize text	22
`split_part(s, '@', 2)`	nth field	23
`date_trunc('month', d)`	bucket by month	25
`now() - INTERVAL '7 days'`	date math	25
`CAST(x AS INT) · x::int`	convert	12

SORTING & PAGING

SYNTAX	DOES	#
`ORDER BY a DESC NULLS LAST`	sort	15
`LIMIT n OFFSET k`	page	16
`FETCH FIRST n ROWS ONLY`	standard limit	16
`WHERE (d, id) < (…)`	keyset page	16
`DISTINCT ON (k)`	first per key (PG)	17

Quick Reference: Dialect Differences

PostgreSQL · MySQL · SQLite · SQL Server · LIMIT vs TOP · concatenation · upsert · identity

QUERY BASICS

FEATURE	POSTGRESQL	MYSQL 8.4	SQL SERVER	SQLITE	#
first n rows	`LIMIT n`	`LIMIT n`	`TOP (n)` / `FETCH`	`LIMIT n`	16
concatenate	`a || b`	`CONCAT(a, b)`	`a + b`	`a || b`	20
quote a name	`"name"`	`` `name` ``	`[name]`	`"name"`	03
string list per group	`string_agg`	`GROUP_CONCAT`	`STRING_AGG`	`group_concat`	31
case-insensitive match	`ILIKE`	`LIKE` (collation)	`LIKE` (collation)	`LIKE` (ASCII)	19
NULL-safe equals	`IS NOT DISTINCT FROM`	`<=>`	`IS NOT DISTINCT FROM`	`IS`	11
if / else function	`CASE`	`IF(c, a, b)`	`IIF(c, a, b)`	`iif(c, a, b)`	21
int / int	integer	decimal	integer	integer	20

TYPES & KEYS

FEATURE	POSTGRESQL	MYSQL 8.4	SQL SERVER	SQLITE	#
auto key	`GENERATED … AS IDENTITY`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`INTEGER PRIMARY KEY`	58
boolean	`BOOLEAN`	`TINYINT(1)`	`BIT`	0 / 1	10
UUID	`UUID`	`BINARY(16)`	`UNIQUEIDENTIFIER`	`TEXT` / `BLOB`	10
timestamp with zone	`TIMESTAMPTZ`	`TIMESTAMP` (UTC)	`DATETIMEOFFSET`	ISO `TEXT`	09
JSON	`jsonb`	`JSON`	`NVARCHAR` / `json` (2025)	`json` functions	78
arrays	`int[]`	JSON arrays	`OPENJSON`	`json_each`	79
now / today	`now()` · `current_date`	`NOW()` · `CURDATE()`	`SYSDATETIME()`	`datetime('now')`	25

WRITING DATA

FEATURE	POSTGRESQL	MYSQL 8.4	SQL SERVER	SQLITE	#
upsert	`ON CONFLICT … DO UPDATE`	`ON DUPLICATE KEY UPDATE`	`MERGE`	`ON CONFLICT`	53
return changed rows	`RETURNING`	—	`OUTPUT`	`RETURNING`	50
update from a join	`UPDATE … FROM`	`UPDATE t JOIN …`	`UPDATE … FROM … JOIN`	`UPDATE … FROM`	51
delete with a join	`DELETE … USING`	`DELETE t FROM t JOIN …`	`DELETE t FROM t JOIN …`	`IN (subquery)`	52
empty a table	`TRUNCATE`	`TRUNCATE` (commits)	`TRUNCATE`	`DELETE FROM t`	52
bulk load	`COPY` / `\copy`	`LOAD DATA`	`BULK INSERT`	`.import`	54

FEATURE SUPPORT

FEATURE	POSTGRESQL	MYSQL 8.4	SQL SERVER	SQLITE	#
FULL OUTER JOIN	yes	no	yes	3.39+	33
INTERSECT / EXCEPT	yes	yes	yes	yes	43
LATERAL	yes	yes	APPLY	no	37
FILTER (WHERE …)	yes	no	no	3.30+	29
DISTINCT ON	yes	no	no	no	17
materialized views	yes	no	indexed views	no	64
transactional DDL	yes	no	yes	yes	68
stored procedures	yes	yes	yes	no	66